



INSTITUTO FEDERAL
Rio Grande do Norte

Campus Ceará-Mirim

Turma 2025.2 – Técnico de Nível Médio em Informática, na Forma Integrada

Disciplina: Programação Orientada a Objetos

Leandro Silva Monteiro de Oliveira

Conteúdo

1. Introdução à orientação a objetos.
 - a. Classes e objetos.
 - b. Estado, comportamento, identidade, abstração e encapsulamento.
 - c. Atributos, métodos e construtores.
 - d. Atributos estáticos e métodos estáticos.**
 - e. Sobrecarga e sobrescrita de métodos.
2. Introdução a modelagem de sistemas através do paradigma de orientação a objetos.
 - a. Leitura e compreensão de diagrama de classes.**
 - b. Relacionamento de classes.**
 - i. Associação, agregação e composição.**
 - ii. Coesão e acoplamento.**
 - iii. Herança, classes abstratas, interfaces e polimorfismo.
3. Coleções.
 - a. Listas, conjuntos e mapas.
4. Tratamento de exceções.
5. Serialização e persistência de objetos.
6. Noções de interface gráfica com o usuário.

Agenda

1 Bimestre

1. Introdução à Programação Orientada a Objeto
2. Classes, Atributos e Métodos
3. Encapsulamento
4. Composição e Agregação - 07/04
5. Herança e Herança Múltipla - 14/04
6. Polimorfismo - 22/04
7. Classes Abstratas e Interfaces - 28/04
8. Coleções - 05/05
9. Tratamento de exceções - 12/05

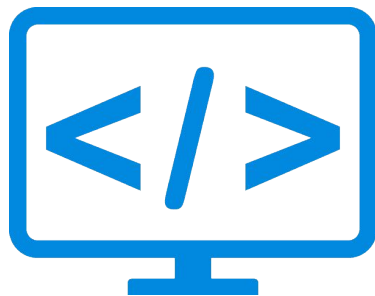
Prova - 19/05

2 Bimestre

10. Serialização de objetos - 26/05
11. Leitura e compreensão de diagrama de classes. - 02/06
12. Persistência de objetos com Peewee e MySQL - 09/06
13. Introdução ao Tkinter - 16/06
14. Tkinter - Widgets Básicos - 23/06
15. Widgets Básicos - Parte 2 - 30/06
16. Gerenciamento de Layout - 07/07
17. Menus e Barra de Ferramentas - 08/07

Prova - 14/07

Prova Final - 21/07



Introdução à Programação Orientada a Objeto



Introdução

- Programas eram lineares e com poucos módulos (Programação Estruturada)
- Aumento da complexidade dos sistemas e difícil **reusabilidade** de componentes
- Criou-se um novo paradigma para análise e desenvolvimento de sistemas: a **Programação Orientada a Objetos**.

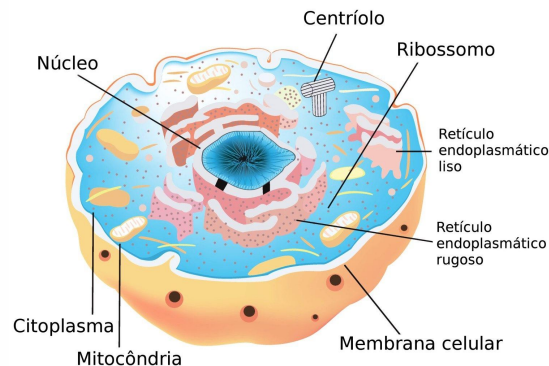
Paradigma da Orientação a Objetos

- O paradigma OO surgiu no fim dos anos 60;
- Alan Kay, um dos pais desse paradigma, formulou a chamada analogia biológica:



Analogia Biológica

- Cada célula interagiria com outras células através do envio de mensagens para realizar um objetivo comum
- Adicionalmente, cada célula se comportaria como uma unidade autônoma
- De uma forma mais geral, Alan Kay pensou em como construir um sistema de software a partir de **agentes autônomos** que **interagem entre si**



Fundamentos da Orientação a Objetos



CONCEITOS DA PROGRAMAÇÃO ORIENTADA A OBJETOS

A POO está sedimentada sobre quatro pilares derivados do princípio da abstração, são eles: **Encapsulamento**, **Herança**, **Composição** e **Polimorfismo**.



OBJETOS

São usados para representar aqueles elementos e abstrações que fazem parte da solução do programa que estamos desenvolvendo;

Possuem características e comportamentos;

As características também podem ser chamadas de dados ou atributos.

Os comportamentos também podem ser chamados de operações ou métodos.

OBJETOS



CARACTERÍSTICAS

(dados, atributos)

Tipo: Ferrari

Cor: Vermelho

Placa: KZE-1018

Número de portas: 2

COMPORTAMENTOS

(operações, métodos)

Ligar

Desligar

Acelerar

Frear



CARACTERÍSTICAS

(dados, atributos)

Tipo: Camila

Cor do cabelo: Negro

Biotipo: Magro

COMPORTAMENTOS

(operações, métodos)

Andar

Correr

Dirigir carro

CLASSES

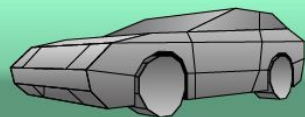
A modelagem e programação de um conjunto de objetos que possuem características (atributos) e comportamentos (métodos) é feita utilizando o conceito de Classe;

Cada classe funciona no fundo como um molde para a criação de um dado objeto;

Os objetos são vistos como representações concretas (instâncias) das classes.

CLASSES

CLASSE



Tipo: ?
Cor: ?
Placa: ?
Número de portas: ?

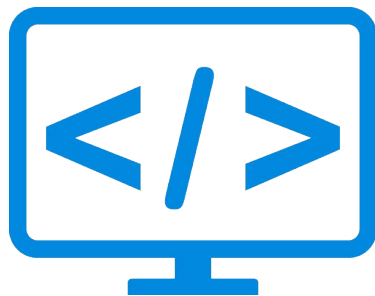
OBJETOS



Tipo: Porsche
Cor: Branco
Placa: MHZ-4345
Número de portas: 4



Tipo: Ferrari
Cor: Vermelho
Placa: JKL-0001
Número de portas: 2



Classes, Atributos e Métodos

Classes, métodos, atributos e objetos

Classe Cachorro:



Objeto: Ralph
Atributo raça: Pug



Objeto: Paçoca
Atributo raça: Salsicha

Método: sentar



Atributo: Pelagem mista
(múltiplas cores)

Classes, métodos, atributos e objetos

Dessa forma, podemos dizer que as classes têm como objetivo definir um determinado grupo de dados de um único tipo ao especificar os **elementos de dados que um objeto contém** (atributos) e o **comportamento** pertencente ao objeto (métodos)

Construtor: Inicializando um objeto



```
class Cachorro:  
    def __init__():  
        pass
```

class: definir uma classe

def __init__ : definir construtor

Definindo uma classe: O parâmetro self e a criação de atributos

```
class Cachorro
```



```
class Cachorro:
    def __init__(self, raca):
        self.raca = raca
```

self: representação da instância.

possibilidade de ter atributos distintos e/ou únicos em cada instância.

Definindo uma classe: O parâmetro self e a criação de atributos

```
class Cachorro
```



```
class Cachorro:  
    def __init__(self, raca):  
        self.raca = raca
```

self: representação da instância.

possibilidade de ter atributos distintos e/ou únicos em cada instância.

Definindo uma classe: Métodos



```
class Cachorro:

    def __init__(self, raca):
        self.raca = raca

    def late(self):
        print("Au au!")
```

def: criação do
método late()

Objeto

```
class Cachorro

class Cachorro:
    def __init__(self, raca):
        self.raca = raca
    def late(self):
        print("Au au!")

Toto = Cachorro("dálmata")
```

Instanciando nosso
primeiro doguinho,
digo, objeto: **Totó**



Interação com o objeto

```
class Cachorro

class Cachorro:
    def __init__(self, raca):
        self.raca = raca
    def late(self):
        print("Au au!")

Toto = Cachorro("dálmata")

print(Toto.raca) #dálmata

Toto.late() #"Au Au!"
```

Atributos de classe

```
class Cachorro:
    quantidade = 0
    cachorros = []

    def __init__(self, raca, idade):
        self.raca = raca
        self.idade = idade

    def late(self):
        print("Au au!")
```

Quantidade = atributo de classe*
para armazenar quantidade de
objetos

*Atributo de classe é utilizado para
armazenar informações
relevantes à classe como, por
exemplo, a quantidade de objetos
de Cachorro que instanciamos.

Mais sobre métodos

```
class Cachorro:
    quantidade = 0
    cachorros = []

    def __init__(self, raca, idade):
        self.raca = raca
        self.idade = idade

    def late(self):
        print("Au au!")
```

```
def mudaIdade(self):
    self.idade = self.idade + 1 #Incrementação

Toto = Cachorro("dálmata", 2)
print(Toto.idade) #2
Toto.mudaIdade()
print(Toto.idade) #3
```

mudalidade: método capaz de alterar atributo "idade".

Classes default

O que vimos até agora foi a criação de classes personalizadas, onde definimos as características que uma instância de uma classe deve ter e seus métodos.

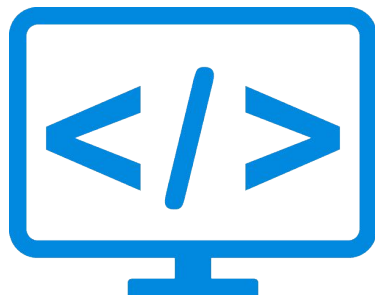
Mas você sabia que existem classes definidas pelo próprio Python? São elas `str`, `int`, `float`, `list` etc.

`str (string)` = "Hello World"

`int (integer)` = 1

`float` = 1.54

`list` = [1, 2, 3]



Encapsulamento

Encapsulamento

Objetivo: esconder de todos os usuários de uma classe algumas informações que não são necessárias ao uso da classe.

Importância: permite que a implementação das funcionalidades de uma classe sejam alteradas sem que o código que a usa precise mudar.

Encapsulamento

```
class Conta:

    def __init__(self, saldo):
        self._saldo = saldo

    def get_saldo(self):
        return self._saldo

    def set_saldo(self, saldo):
        if(saldo < 0):
            print("O Saldo não pode ser negativo")
        else:
            self.saldo = saldo
```

Problema: Como nossa classe é projetada com atributos públicos, podemos quebrar a interface e simplesmente acessar e alterar os atributos que deveriam ser protegidos.

Encapsulamento

Solução: Mais sobre o property:

Possui métodos extras, como um getter e um setter. Quando aplicado em um objeto, nos retorna uma cópia dele com as suas funcionalidades. Logo, somos capazes de chamar nossos métodos sem os parênteses e como se fossem atributos públicos.

```
class Conta:

    def __init__(self, saldo=0.0):
        self._saldo = saldo

    @property
    def saldo(self):
        return self._saldo

    @saldo.setter
    def saldo(self, saldo):
        if(saldo < 0):
            print("O Saldo não pode ser negativo")
        else:
            self._saldo = saldo
```

Encapsulamento

Mais sobre o property:

Possui métodos extras, como um getter e um setter. Quando aplicado em um objeto, nos retorna uma cópia dele com as suas funcionalidades. Logo, somos capazes de chamar nossos métodos sem os parênteses e como se fossem atributos públicos.

```
decorator property

@property
def func(self):
    return self.func

#é equivalente a:

def func(self):
    return self._func

func = property(func)
```

Exemplo Prático: Protegendo a Classe Conta

2. Getter disfarçado:
Acesso limpo e direto
de leitura

```
class Conta:
    def __init__(self, saldo=0.0):
        self._saldo = saldo

    @property
    def saldo(self):
        return self._saldo

    @saldo.setter
    def saldo(self, saldo):
        if (saldo < 0):
            print("O Saldo não pode ser negativo")
        else:
            self._saldo = saldo
```

1. Atributo interno
protegido por convenção (_)

3. Setter inteligente:
Barreira de validação na
escrita contra saldos
negativos

Encapsulamento

```
Printando um objeto

class Gato(SerVivo):
    def __init__(self, nome, nacionalidade):
        super().__init__(nome, nacionalidade)
        self.tipo = 'Gato'

gato1 = Gato("Garfield", "estadunidense")
print(gato1) #<__main__.Gato object at 0x7f8780ab8cd0>
```

O que acontece se
printarmos um
objeto?

Encapsulamento

```
Printando um objeto

class Gato(SerVivo):
    #__init__
    def __str__(self):
        return f"""
        -----
        Sou um {self.tipo} {self.nacionalidade}!
        Mas você pode me chamar de {self.nome}
        -----
        """

gato1 = Gato("Garfield", "estadunidense")
print(gato1) #vai chamar o __str__
```

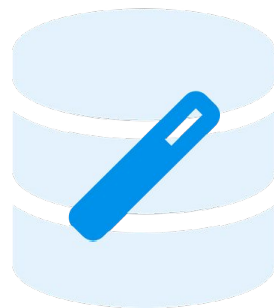
método `__str__`:
útil para printar as
informações do seu
objeto

Matriz de Diagnóstico: Modificadores de Acesso

Nível de Acesso	Sintaxe	Significado / Intenção	Bloqueio Real do Python
Público	<code>self.nome</code>	Acesso livre para qualquer parte do código	Nenhum (Acesso direto)
Protegido	<code>self._nome</code>	Uso interno da classe e subclasses	Convenção (Confiança)
Privado	<code>self.__nome</code>	Prevenção de conflitos de nomenclatura	Mecanismo (Name Mangling)

O que são Data Classes?

- ✓ Introduzidas no Python 3.7 pela **PEP 557**.
- ✓ Classes voltadas principalmente para o **armazenamento de dados**.
- ✓ O decorador **@dataclass** automatiza a criação de métodos especiais.
- ✓ Tornam o código mais **conciso, legível** e menos propenso a erros.



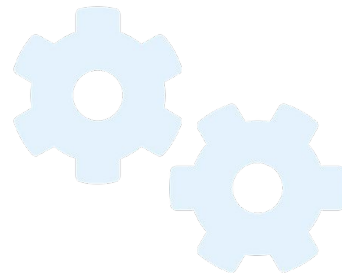
Redução de Código Repetitivo (Boilerplate)

O Trabalho Manual do Programador:

✖ Em classes tradicionais, o programador deve escrever manualmente métodos como:

- `__init__`: Inicialização de atributos
- `__repr__`: Representação textual amigável
- `__eq__`: Comparação de igualdade

✍ Com **Data Classes**, todo esse código é gerado automaticamente.



O Passado (Classe Comum)

```
class InventoryItem:

    def __init__(self, name: str,
                  unit_price: float,
                  quantity_on_hand: int = 0):
        self.name = name
        self.unit_price = unit_price
        self.quantity_on_hand =
            quantity_on_hand
```



O Futuro (Data Class)

```
@dataclass
class InventoryItem:
    name: str
    unit_price: float
    quantity_on_hand: int = 0
```

Redução drástica de complexidade visual e digitação, mantendo exatamente as mesmas funcionalidades e tipagem forte.

Métodos Gerados Automaticamente



`__init__`

Cria o construtor que aceita argumentos para os campos definidos.



`__repr__`

Retorna uma string clara:

`Produto(nome='Café', preco=15.0)`



`__eq__`

Compara instâncias como se fossem tuplas de seus valores.

Importante

A ordem dos campos nos métodos gerados segue exatamente a **ordem de declaração** na classe.

Se o método já estiver definido na classe, o decorador não o sobrescreverá.

Configurações do Decorador

init=True

(Padrão) Gera automaticamente o método construtor `__init__`.

repr=True

(Padrão) Gera o método `__repr__` para representação textual.

eq=True

(Padrão) Gera o método `__eq__` para comparação de igualdade.

order=False

Se True, gera métodos de ordenação como `__lt__` e `__gt__`.

Dica

O decorador aceita diversos outros parâmetros para controle fino, como **`unsafe_hash`**, **`match_args`** e **`kw_only`**.

O Superpoder da Imutabilidade



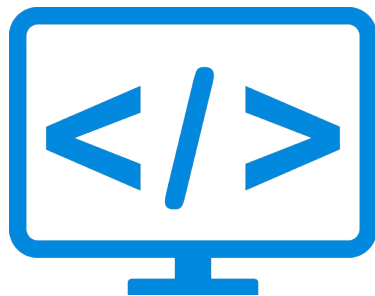
```
@dataclass(frozen=True)
class InventoryItem:
    name: str
    unit_price: float
```

Emula instâncias congeladas de leitura apenas (read-only). Dados que não mudam são dados seguros e previsíveis.

```
item = InventoryItem('Placa', 500)
```

```
item.unit_price = 450
```

```
>_ FrozenInstanceError: cannot assign
to field 'unit_price'
```



Composição e Agregação

Composição

Classe **Conta** criada para um sistema de *internet banking*.

```
class Conta

class Conta:
    def __init__(self, numero, cliente, saldo):
        self.numero = numero
        self.saldo = saldo
        self.limite = limite

    def transfere(self, destino, valor):
        print(f"Transferindo R${valor} para {destino}")
```

Composição

Como toda conta possui histórico de transações, foi criada também a Classe **Histórico**.

```
class Historico:

    def __init__(self):
        self.transacoes = []
        self.dataAbertura = datetime.datetime.today()

    def imprime(self):
        print(f"data de abertura: {self.dataAbertura}")
        print("Transações: ")

        for transacao in self.transacoes:
            print(transacao)
```

Composição

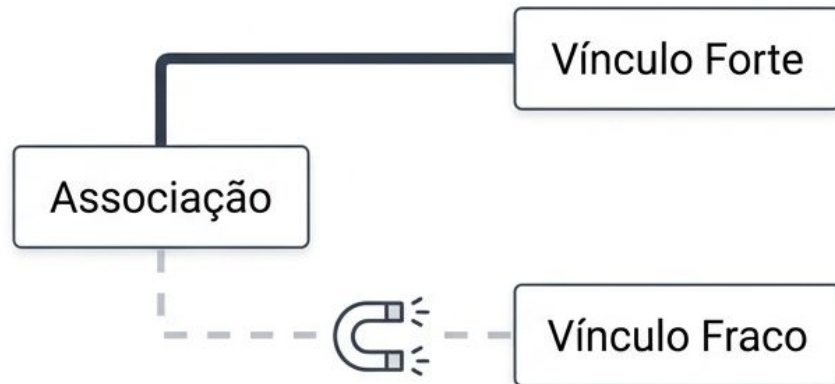
E é nesse momento que usaremos a composição! Ela fará com que cada objeto de **Conta** possua um **histórico**. Assim, criamos um atributo em **Conta** que recebe um objeto de **Histórico**.

```
class Conta

class Conta:
    def __init__(self, numero, cliente, saldo):
        self.numero = numero
        self.saldo = saldo
        self.limite = limite
        self.historico = Historico()
```


Objetos não vivem isolados

Na Programação Orientada a Objetos, construir sistemas complexos exige que classes colaborem entre si. Embora a Herança seja o relacionamento mais famoso (“é um”), a maior parte da arquitetura de software real é construída sobre Associações (“tem um”).



A grande decisão arquitetural não é se os objetos vão interagir, mas sim como eles compartilham seu ciclo de vida.

O Vínculo Forte: Composição



**Na Composição, a classe "filha" não existe sem a classe "pai".
É uma relação de dependência existencial.**

A parte pertence estritamente ao todo.

O ciclo de vida é unificado: se o objeto principal for destruído na memória, suas partes também serão.

Exemplo Clássico: Uma Conta bancária e seu Historico de transações. Um histórico não tem sentido ou existência sem uma conta atrelada.

As Peças do Quebra-Cabeça

```
class Conta:
    def __init__(self, numero, cliente, saldo):
        self.numero = numero
        self.saldo = saldo
        self.limite = limite

    def transfere(self, destino, valor):
        print(f"Transferindo R${valor} para {destino}")
```

```
class Historico:
    def __init__(self):
        self.transacoes = []
        self.dataAbertura = datetime.datetime.today()

    def imprime(self):
        print(f"Data de abertura: {self.dataAbertura}")
```

O Momento Exato da Composição

```
class Conta:
    def __init__(self, numero, cliente, saldo):
        self.numero = numero
        self.saldo = saldo
        self.limite = limite
        self.historico = Historico() <--- O Vínculo
```

E é nesse momento que usaremos a composição! Ela fará com que cada objeto de Conta possua um histórico. Assim, criamos um atributo em Conta que recebe um objeto de Histórico (instanciado internamente).

O Vínculo Fraco: Agregação



Na Agregação, as partes podem existir independentemente do todo. É uma relação de associação por referência.

O todo precisa da parte para funcionar plenamente, mas a parte tem vida própria.

O ciclo de vida é independente: destruir o objeto principal não destrói os objetos agregados.

Exemplo Clássico: Um Carrinho de Compras e Produtos. Se o usuário exclui o carrinho, os produtos continuam existindo no banco de dados.

A Entidade Independente (Produto)

Primeiro, criamos um objeto que possui seu próprio significado no sistema, independentemente de onde será usado.

```
class Produto:
    def __init__(self, nome, preco):
        self.nome = nome
        self.preco = preco

    def exibir_detalhes(self):
        print(f"{self.nome} - R${self.preco}")
```

Este objeto vive no catálogo da loja. Ele não sabe o que é um carrinho de compras.

O Momento Exato da Agregação

```
class CarrinhoDeCompras:
    def __init__(self):
        self.itens = [] ← # Lista vazia, pronta para agregar

    def adicionar_produto(self, produto): ←--- Recebe de fora
        self.itens.append(produto)
```

Na agregação, o objeto "parte" (produto) é instanciado fora do objeto "todo" (CarrinhoDeCompras) e passado como parâmetro. O carrinho apenas guarda uma referência a um objeto que já existe.

A Diferença na Sintaxe (Lado a Lado)

Composição (Instanciação Interna)

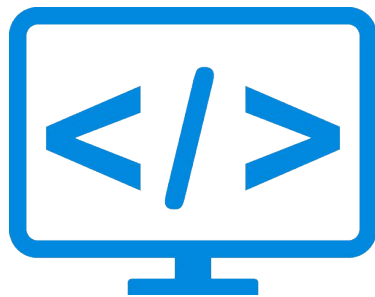
```
self.historico = Historico()
```

O construtor da classe principal cria a dependência. A classe principal tem controle total sobre o nascimento da parte.

Agregação (Injeção Externa)

```
carrinho.adicionar(produto)
```

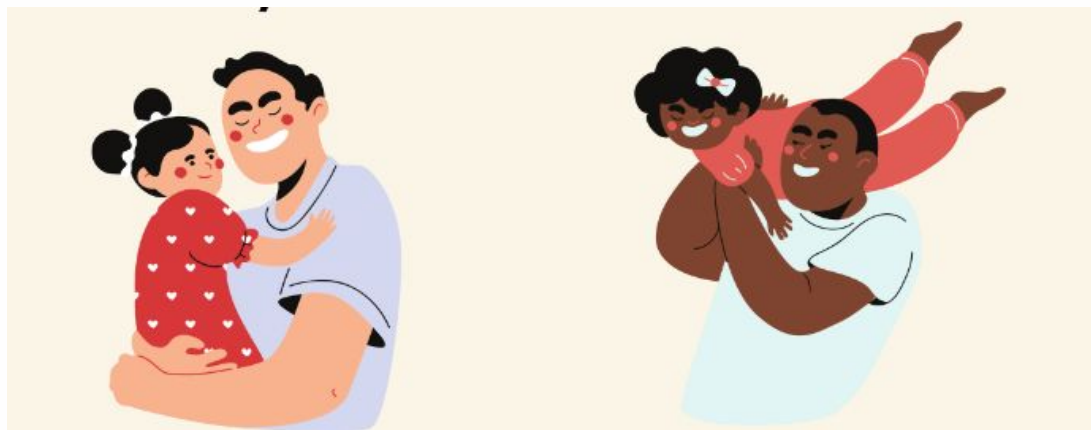
A classe principal recebe a dependência de fora. A classe principal apenas "pega emprestado" o objeto.



Herança

Herança

Todo filho tem algumas características dos pais e chamamos isso de herança genética! Inspirando-se neste conceito de **herança**, foi criado o conceito de **herança** na programação e então implementado em Python.



Herança

Sabemos que tanto o gato quanto o humano são seres vivos e portanto nascem e amadurecem. Observe a classe **SerVivo**:

```
class SerVivo:
    def __init__(self):
        self.tipo = 'Ser Vivo'
        self.idade = 0

    def nascer(self, nome):
        self.nome = nome
        print(f'Eu sou do tipo {self.tipo}. Meu nome é {self.nome} e acabei de nascer!')

    def amadurecer(self):
        self.idade += 1
        print(f'Eu, {self.nome}, amadureci! Minha idade é {self.idade}.')
```

Herança

Então as classes **Humano** e **Gato** podem herdar as propriedades de **SerVivo**!

```
class Humano(SerVivo):
    def __init__(self):
        super().__init__()
        self.tipo = 'Humano'

    def sorrir(self):
        print('(: Eu sou humano e sei rir :)')

class Gato(SerVivo):
    def __init__(self):
        super().__init__()
        self.tipo = 'Gato'

    def miar(self):
        print('Miau! Sou um gatinho e sei miar +^-^+')
```

Herança

Podemos mandar o objeto Humano nascer? Claro!

`humano.nascer('Wilson')`

```
class Humano(SerVivo):  
    def __init__(self):  
        super().__init__()  
        self.tipo = 'Humano'  
  
    def sorrir(self):  
        print('(: Eu sou humano e sei rir :)')
```

OBS: o método `nascer()` está em `SerVivo`!

Herança

E se pedirmos ao objeto Gato para sorrir ou ao objeto Humano para miar? **erro de atribuição!**

```
class Humano(SerVivo):
    def __init__(self):
        super().__init__()
        self.tipo = 'Humano'

    def sorrir(self):
        print('(: Eu sou humano e sei rir :)')

class Gato(SerVivo):
    def __init__(self):
        super().__init__()
        self.tipo = 'Gato'

    def miar(self):
        print('Miau! Sou um gatinho e sei miar +^-^+')
```

Herança

Vamos agora criar uma nova classe filha de **SerVivo** (ou apenas subclasse): **Cachorro**.

```
class Cachorro

class Cachorro(SerVivo):
    def __init__(self):
        self.tipo = 'Cachorro'
```

Você consegue identificar o que está faltando?

Herança

Perceba que diferentemente das outras classes, não chamamos `super().__init__()` em seu construtor. Por conta disso, o interpretador não inicia a nova classe com o construtor da classe mãe. Dessa maneira, nosso objeto Cachorro não herdará nunca o atributo idade, somente os métodos.



Herança

Se quisermos, além de usar os atributos da classe mãe, adicionar novos atributos, é somente preciso adicionar ao `__init__` da classe filha.

```
class Criança

class Criança(Humano):
    def __init__(self):
        super().__init__()
        self.dentes = 0
        self.inocencia = True
```

`Criança` vai herdar atributos e métodos tanto de `Humano` quanto de `SerVivo`.

Herança

E se quisermos um `__init__` na classe filha que receba pelo menos um input do usuário **diferente da classe mãe?**

```
class CriancaBrincalhona

class CriancaBrincalhona(Crianca):
    def __init__(self, brinquedoFavorito):
        super().__init__() #perceba que não recebe brinquedoFavorito
        self.brinquedoFavorito = brinquedoFavorito
```

Ela recebe **brinquedoFavorito** diferentemente da sua classe mãe

Herança

Em adição, a herança nos possibilita o uso de **métodos sobrescritos**: se um filho tem um método com o mesmo nome de um método do pai, o interpretador executa o do filho.

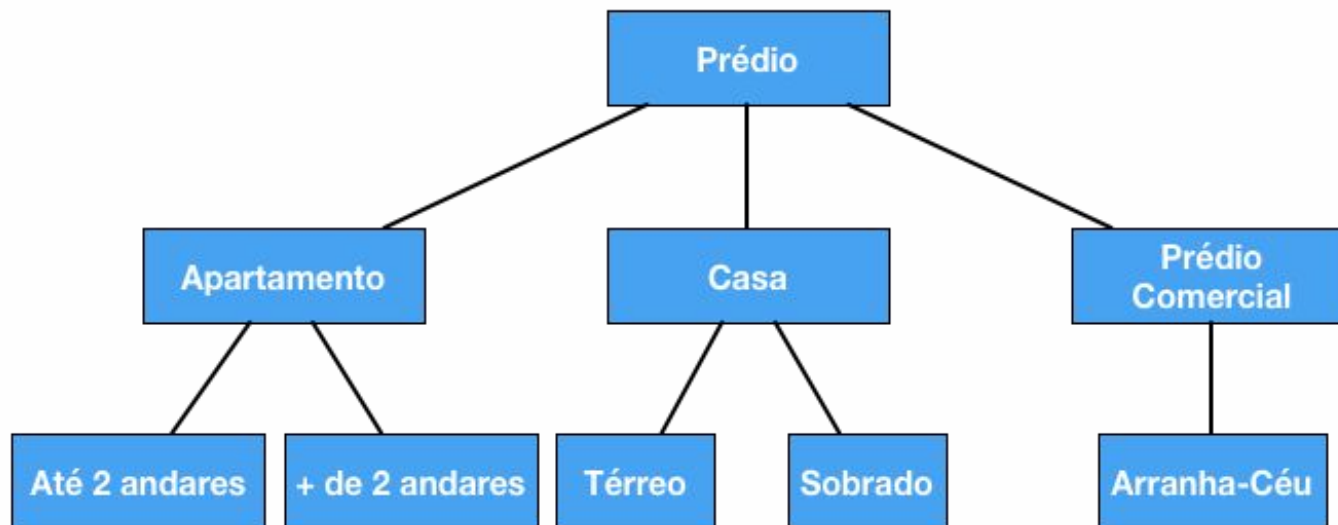
```
class GatoBranco

class GatoBranco(Gato):
    def __init__(self):
        super().__init__()

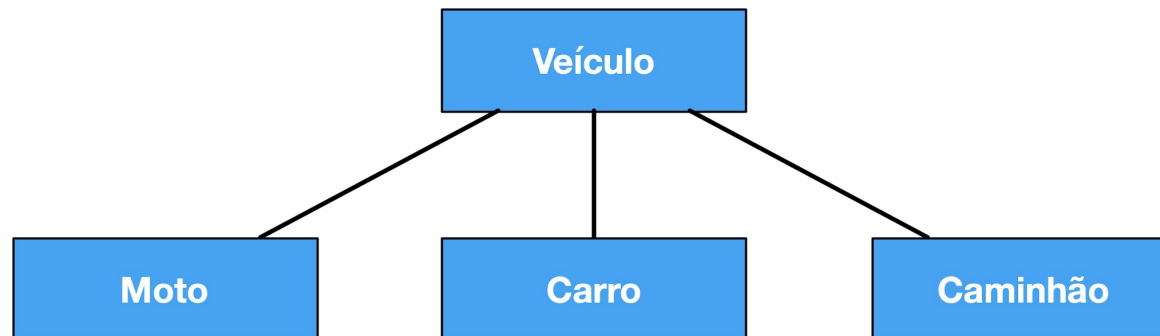
    def miar(self):
        print('Eu sou um gato branco e sou rei do mundo! feed me /\-q-\')

```

Modelo Hierárquico

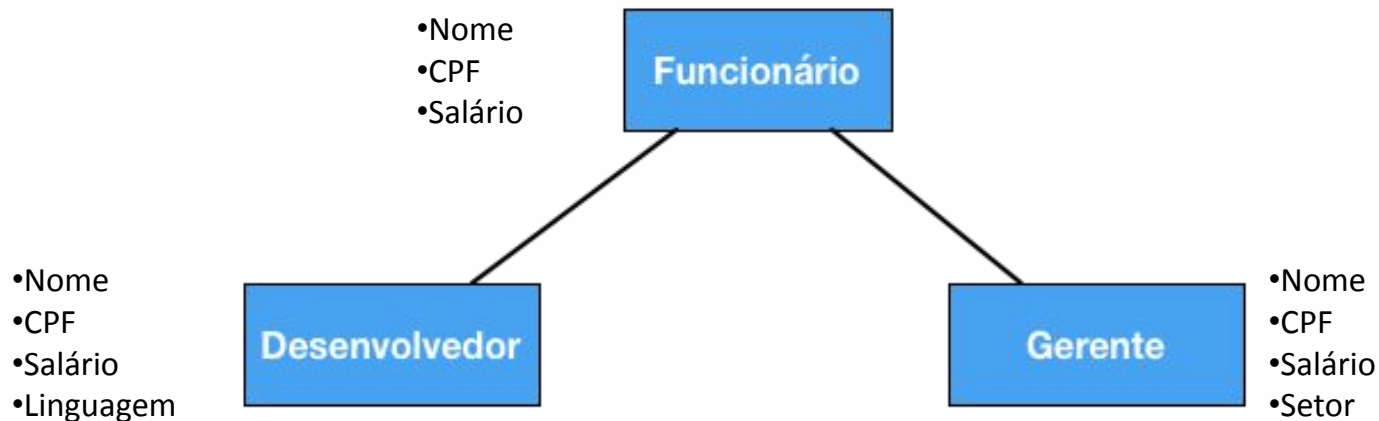


Modelo Hierárquico



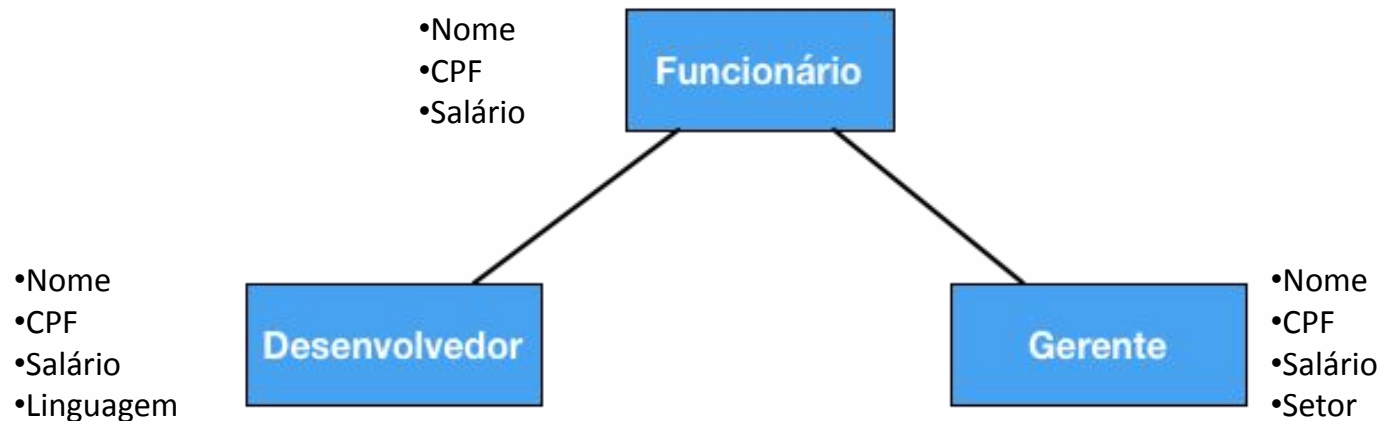
Herança

- Exemplo: dois funcionários



Herança

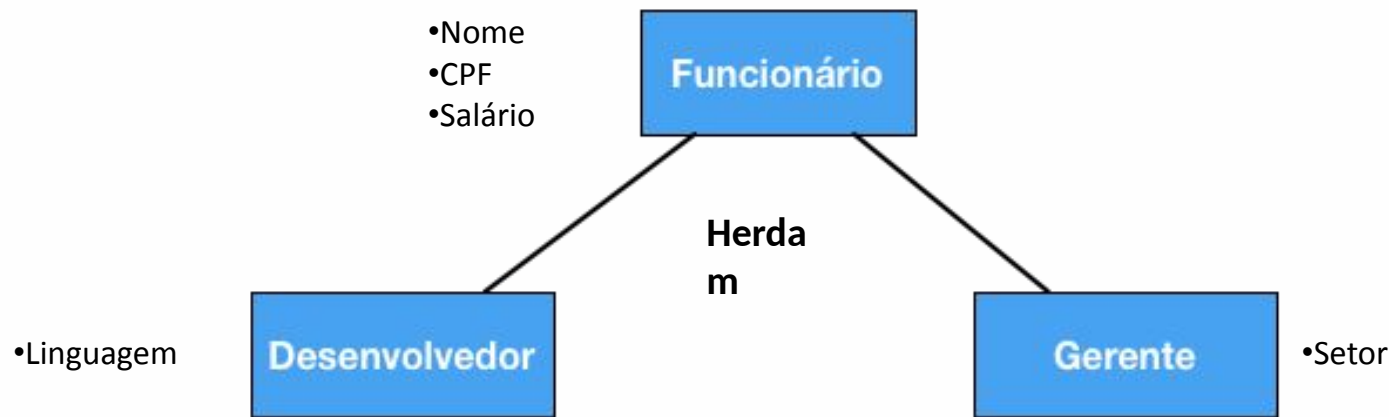
- Exemplo: dois funcionários



- Problema: repetição de alguns atributos para todos os funcionários
- Como resolver?

Herança

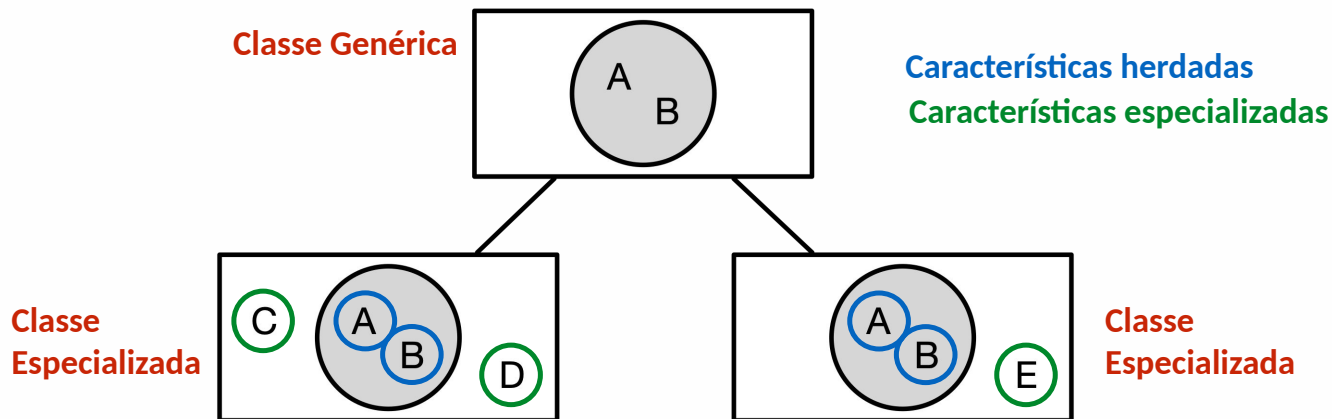
• Solução: **herança**



• Desenvolvedor e Gerente **herdam** atributos e métodos de Funcionário

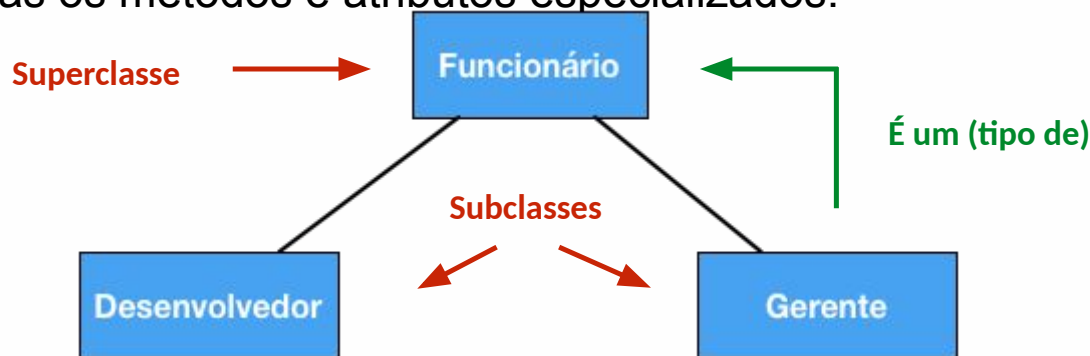
Herança

- Estrutura hierárquica e modular;
- Classes genéricas podem ser especializadas em classes mais particulares;
- Classes especializadas reutilizam o código das mais genéricas.



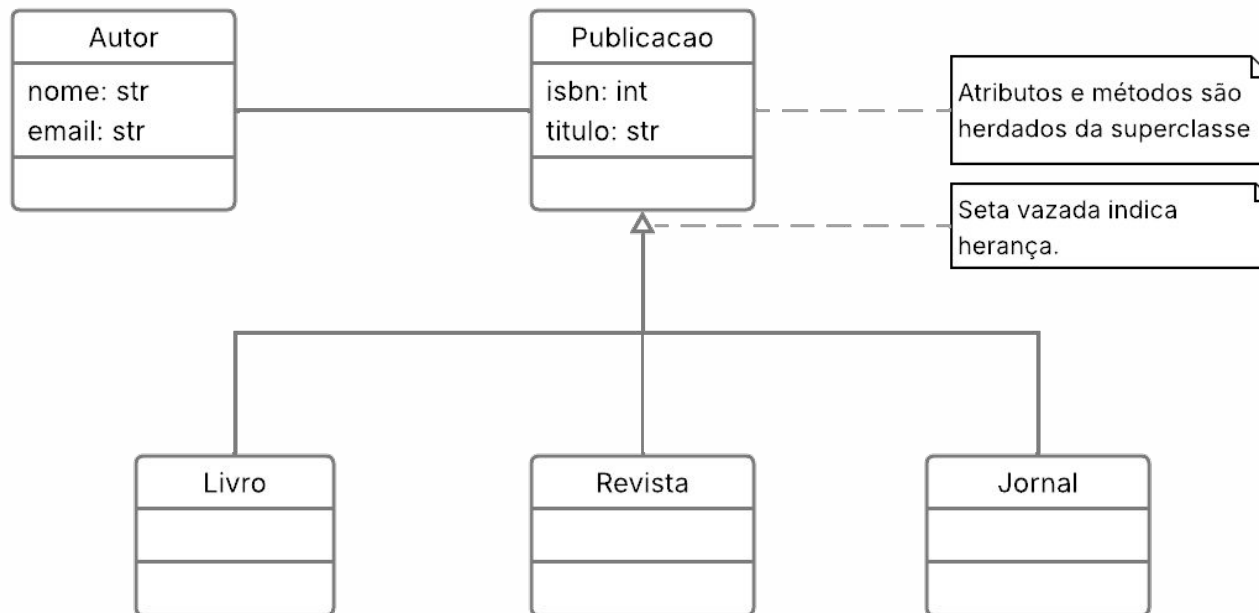
Herança

- Classe genérica, classe base, **superclasse** ou pai:
 - Define variáveis de instância “genéricas” e métodos.
- Classe especializada, derivada, **subclasse** ou filha:
 - Especializa, estende ou **herda** os métodos e atributos “genéricos” de uma superclasse;
 - Define apenas os métodos e atributos especializados.

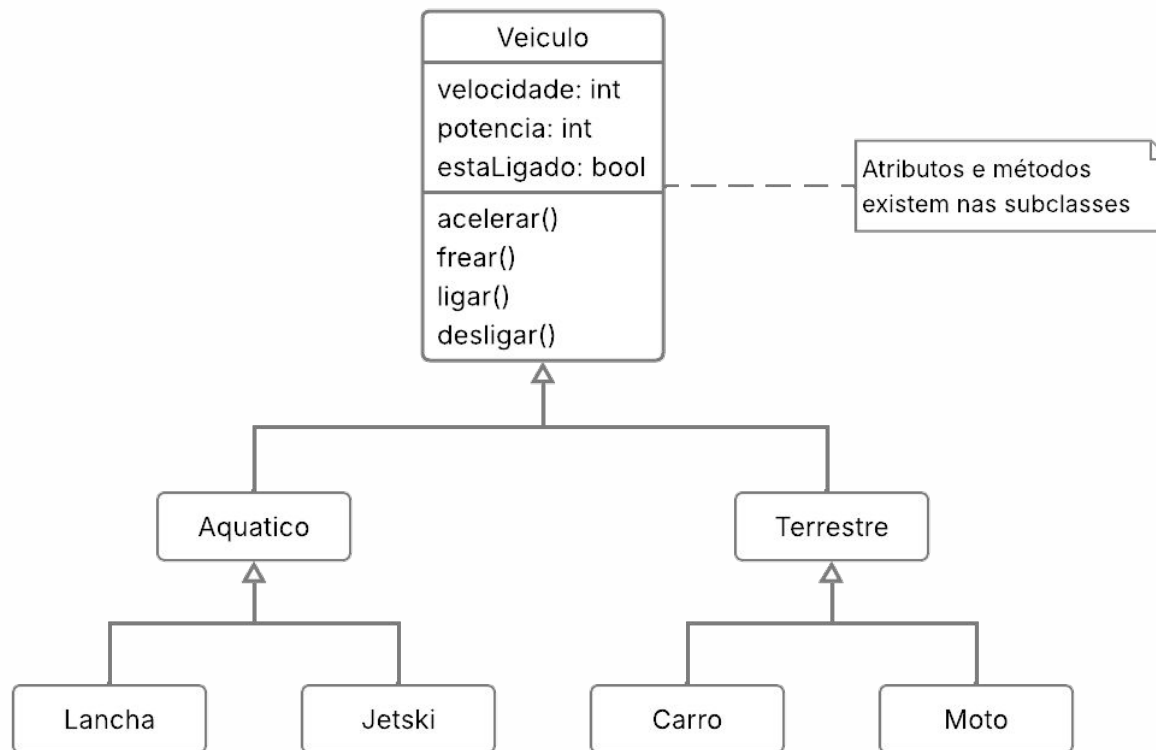


Herança

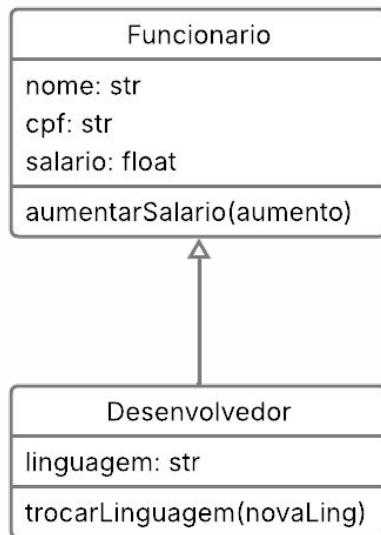
- Na UML



Herança



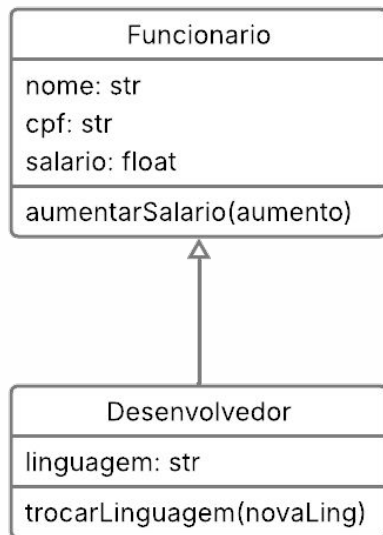
Herança - Implementação



```
class Funcionario:
    def __init__(self, nome, cpf, sal):
        self.nome = nome
        self.cpf = cpf
        self.salario = sal

    def aumentarSalario(self, aumento):
        self.salario = self.salario + aumento
```

Herança - Implementação



```
class Funcionario:
    def __init__(self, nome, cpf, sal):
        self.nome = nome
        self.cpf = cpf
        self.salario = sal

    def aumentarSalario(self, aumento):
        self.salario = self.salario + aumento

from funcionario import Funcionario
class Desenvolvedor(Funcionario):
    def __init__(self, nome, cpf, sal, ling):
        super().__init__(nome, cpf, sal)
        self.linguagem = ling

    def trocarLinguagem(self, novaLing):
        self.linguagem = novaLing
```

Herança - Implementação

```
class Funcionario:  
    def __init__(self, nome, cpf, sal):  
        self.nome = nome  
        self.cpf = cpf  
        self.salario = sal
```

```
    def aumentarSalario(self, aumento):  
        self.salario = self.salario + aumento
```

Indica que Desenvolvedor é uma subclasse de Funcionario.

```
class Desenvolvedor(Funcionario):  
    def __init__(self, nome, cpf, sal, ling):  
        super().__init__(nome, cpf, sal)  
        self.linguagem = ling  
  
    def trocarLinguagem(self, novaLing):  
        self.linguagem = novaLing
```

`super()` é usado para chamar o construtor da superclasse Funcionario. Depois, `__init__` inicializa as variáveis da superclasse.

Herança - Implementação

```
from desenvolvedor import Desenvolvedor

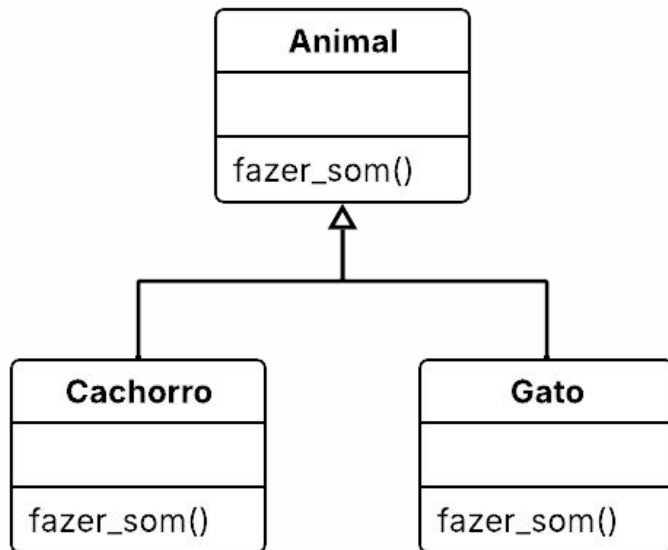
d = Desenvolvedor("Leo Silva", "123.456.789-00", 4500, "Python")
print(d.nome, d.cpf, d.salario, d.linguagem)
# Leo Silva 123.456.789-00 4500 Python

d.aumentarSalario(500)
print(d.nome, d.cpf, d.salario, d.linguagem)
# Leo Silva 123.456.789-00 5000 Python
```


Herança - Sobrescrita de Métodos

- Ocorre quando a **subclasse define um método com o mesmo nome** de um método da superclasse
- Quando uma instância da subclasse chama esse método, a implementação da subclasse é executada ao invés da implementação da superclasse
- Esse mecanismo **permite que a subclasse substitua ou altere o comportamento de um método herdado** da superclasse, adicionando a sua própria lógica específica.

Herança - Sobreescrita de Métodos



```
class Animal:
    def __init__(self, nome):
        self.nome = nome
```

```
    def fazer_som(self):
        return "Som genérico"
```

```
class Cachorro(Animal):
    def fazer_som(self):
        return "Au Au!"
```

```
class Gato(Animal):
    def fazer_som(self):
        return "Miau!"
```

Herança - Sobrescrita de Métodos

```
class Animal:  
    def __init__(self, nome):  
        self.nome = nome
```

```
    def fazer_som(self):  
        return "Som genérico"
```

```
class Cachorro(Animal):  
    def fazer_som(self):  
        return "Au Au!"
```

```
class Gato(Animal):  
    def fazer_som(self):  
        return "Miau!"
```

O construtor da superclasse é usado pelas subclasses.

Método fazer_som() sobrescrito nas subclasses, implementando comportamentos diferentes.

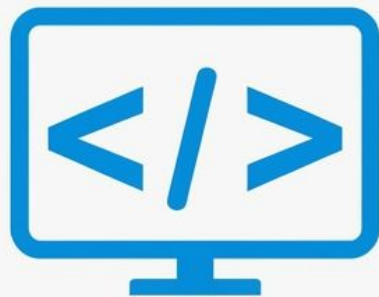


Herança - Sobrescrita de Métodos

```
from cachorro import Cachorro
from gato import Gato

cachorro = Cachorro("Rex")
gato = Gato("Mimi")

print(cachorro.nome)
print(cachorro.fazer_som())
print(gato.nome)
print(gato.fazer_som())
```



Herança Múltipla

Na vida real, herdamos de ambos os pais



Na vida real, uma criança herda traços genéticos de ambos os pais (pai e mãe). Em Python, uma classe também pode herdar características e comportamentos de múltiplas classes base ao mesmo tempo!

Construindo componentes independentes

```
class Relogio:  
    def ver_hora(self):  
        print('14:30 - Hora de codar!')
```

```
class Calendario:  
    def ver_data(self):  
        print('Hoje é dia de aprender Python.')
```

Vamos traduzir isso para código. Imagine duas classes base distintas, cada uma com sua própria especialidade.

A convergência: criando uma subclasse múltipla

```
class Smartwatch(Relogio, Calendario):  
    pass
```



Ao criar nosso `Smartwatch`, passamos ambas as classes base separadas por vírgula. Ele acaba de herdar de ambas!

O objeto herda todas as funcionalidades



```
meu_watch = Smartwatch()
```

```
meu_watch.ver_hora()
```

```
meu_watch.ver_data()
```

```
> 14:30 - Hora de codar!
```

```
> Hoje é dia de aprender Python.
```

O objeto da subclasse tem acesso total. Podemos chamar métodos do Relógio e do Calendário usando a mesma instância.

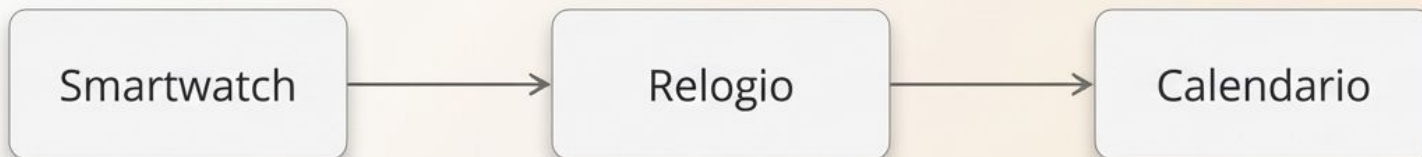
Mas e se houver um conflito de métodos?



```
# Se adicionarmos isso no Relogio:  
def ajustar(self):  
    print('Ajustando a hora...')  
  
# E adicionarmos isso no Calendario:  
def ajustar(self):  
    print('Ajustando o dia e mês...')  
  
# O que acontece aqui?  
meu_watch.ajustar()
```

Se ambas as classes pai possuírem um método com o exato mesmo nome, qual deles o Python vai executar?

MRO: Ordem de Resolução de Métodos



```
print(Smartwatch.__mro__)
```

```
# Output: (<class 'Smartwatch'>, <class 'Religio'>, <class 'Calendario'>, <class 'object'>)
```

O Python resolve conflitos usando o MRO. Ele busca primeiro na própria classe, depois na primeira classe pai (da esquerda para a direita). O atributo especial `__mro__` revela essa ordem exata de busca.

Navegando a hierarquia com `super()`

```
class Smartwatch(Rologio, Calendario):  
    def __init__(self):  
        super().__init__()  
        self.bateria = 100
```

Na herança múltipla, o `super()` se torna vital. Ele não chama apenas 'o pai'; ele segue rigorosamente a ordem do MRO para que todas as classes base sejam inicializadas na sequência correta, sem duplicações.

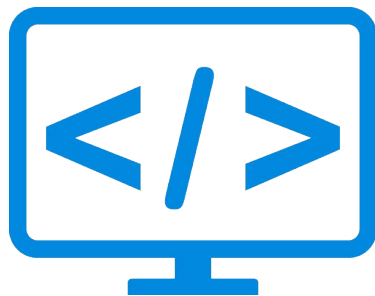
O complexo "Problema do Diamante"



Cuidado! A herança múltipla é poderosa, mas pode gerar códigos extremamente confusos e difíceis de manter. Use com moderação. Se a arquitetura ficar muito complexa, considere usar Composição de objetos em vez de Herança.

Flexibilidade com responsabilidade

Conceito	Resolução	Boas Práticas
Combina comportamentos de múltiplas fontes usando <code>class Sub(Pai, Mae)`</code> .	O Python usa o MRO (esquerda para a direita) para decidir qual método executar em caso de conflitos.	Use <code>super()</code> para inicialização segura e evite hierarquias profundas para manter o código limpo.

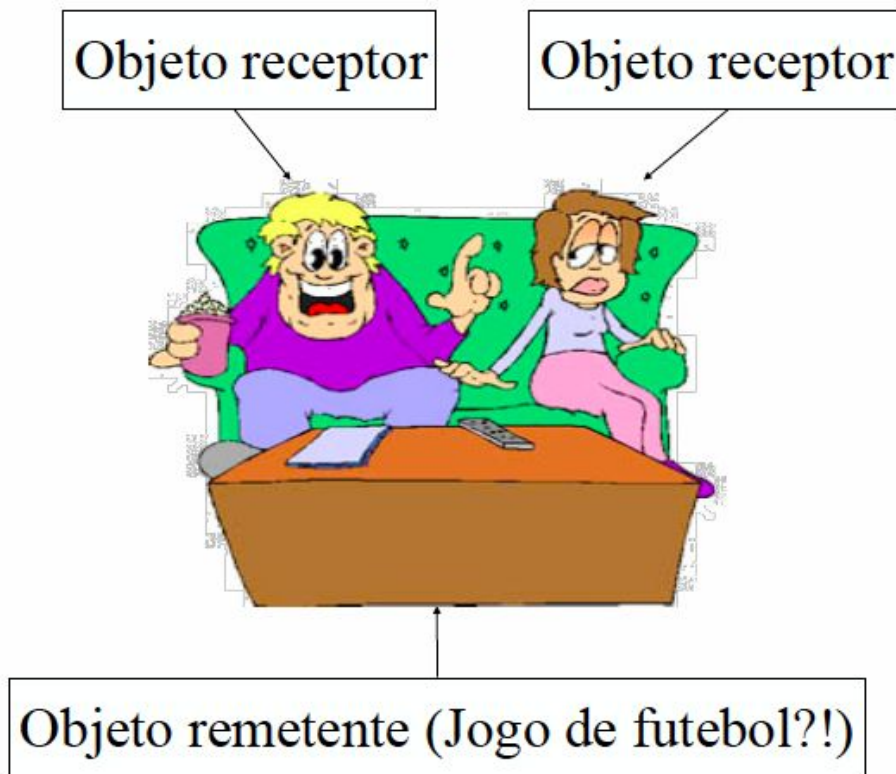


Polimorfismo

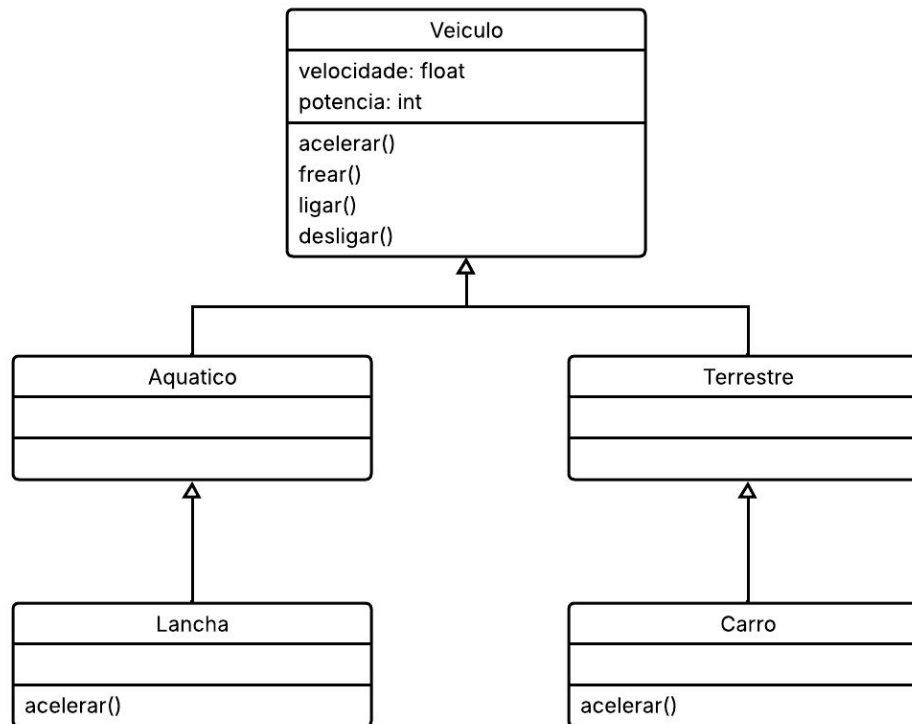
Polimorfismo

- . Princípio fundamental da orientação a objetos
- . “Várias formas”
- . É a habilidade de objetos de classes diferentes responderem a mesma mensagem de maneiras diferentes

Polimorfismo



Polimorfismo



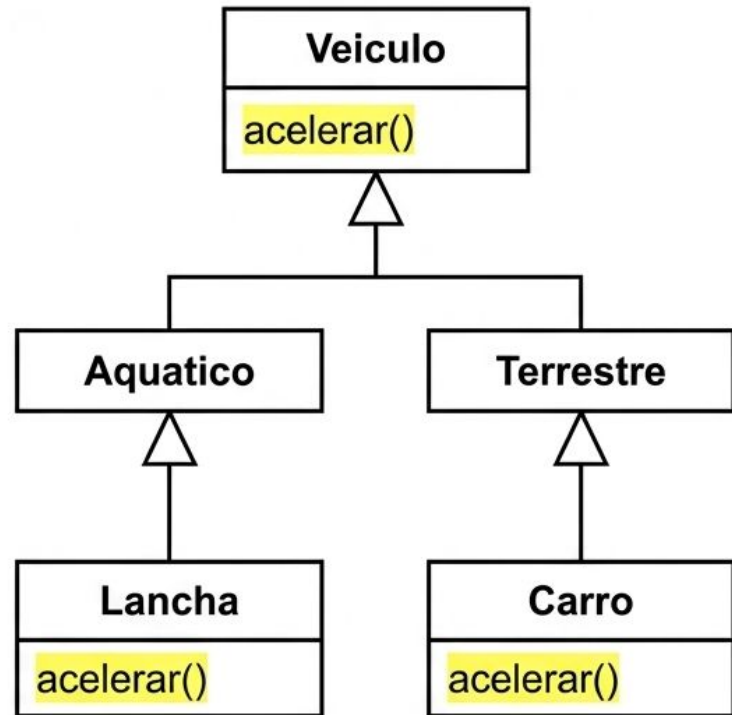
1. Polimorfismo por Sobreposição (Sobrescrita)

Onde Ocorre: Através da Herança (relacionamento entre Superclasse e Subclasse).

A Regra: O método é redefinido na subclasse, mas obrigatoriamente mantém a mesma assinatura (nome e parâmetros).

O Comportamento: Altera ou estende o comportamento definido na superclasse.

A Resolução: Ocorre em tempo de execução (Dynamic Binding). A decisão de qual método rodar é feita na hora.





Sobrescrita de Métodos

- . Um método pode ser redefinido na subclasse
 - Sobrescrita
 - Altera o comportamento definido na superclasse
 - Mantém a assinatura do método

```
class Animal:
    def emitir_som(self):
        return "Algum som genérico"
class Cachorro(Animal):
    def emitir_som(self):
        return "Cachorro latindo"
class Gato(Animal):
    def emitir_som(self):
        return "Gato miando"
# comportamento polimorfico
animais = [Cachorro(), Gato(), Animal()]
for animal in animais:
    print(animal.emitir_som()) # Chama o metodo
                              # sobrescrito baseado no tipo do objeto
```

Sobrescrita de Métodos

- É possível chamar um método da superclasse
 - `super().metodo()`
- Pode ser usado em qualquer método
 - O comportamento de um método pode ser sobrescrito, mas mantendo a funcionalidade existente.

```
class Gato(Animal):  
    def emitir_som(self):
```

- **`som_original = super().emitir_som()`**
- **`print(som_original)`**
return "Gato miando" Cachorro latindo

Algum som genérico

Gato miando

Algum som genérico

A Sobrescrita na Prática (Python)

```
class Animal:
    def emitir_som(self):
        return 'Algum som genérico'

class Cachorro(Animal):
    def emitir_som(self):
        return 'Cachorro latindo'

class Gato(Animal):
    def emitir_som(self):
        return 'Gato miando'
```

 É possível usar `super().emitir_som()` para invocar o comportamento original antes de sobrescrevê-lo.

```
animais = [Cachorro(), Gato(), Animal()]

for animal in animais:
    # A mesma chamada executa métodos diferentes
    print(animal.emitir_som())
```

Cachorro
latindo

Gato
miando

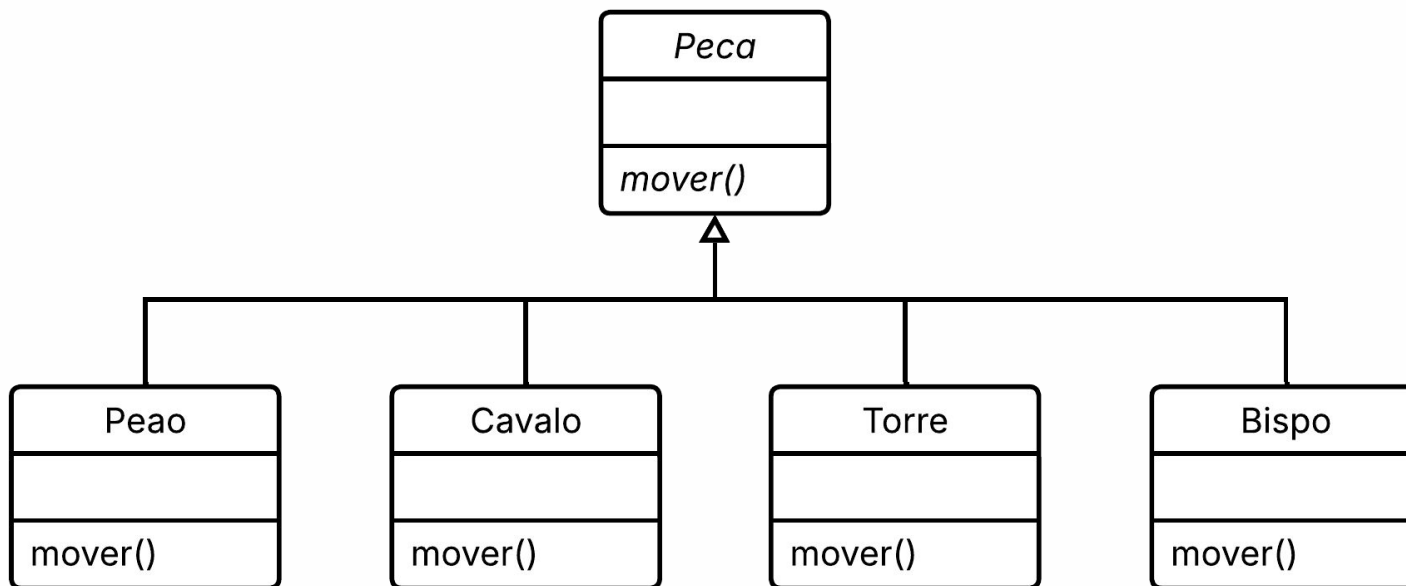
Algum som
genérico

Polimorfismo

- . Método executado depende da classe do objeto
- . Mesma chamada executa métodos diferentes
 - `obj.metodo()` executa método definido na classe do objeto referenciado por `obj`
- . Permite executar métodos de subclasses mesmo sem conhece-las
- . É possível através da sobrescrita
- . A decisão da execução do método é tomada em tempo de execução

Polimorfismo

- Exemplo: jogo de xadrez





Polimorfismo

- Exemplo: jogo de xadrez

```
from abc import ABC, abstractmethod
class Peca:
    @abstractmethod
    def mover(self):
        pass
class Peao(Peca):
    def mover(self):
        return "movendo o peao..."
```

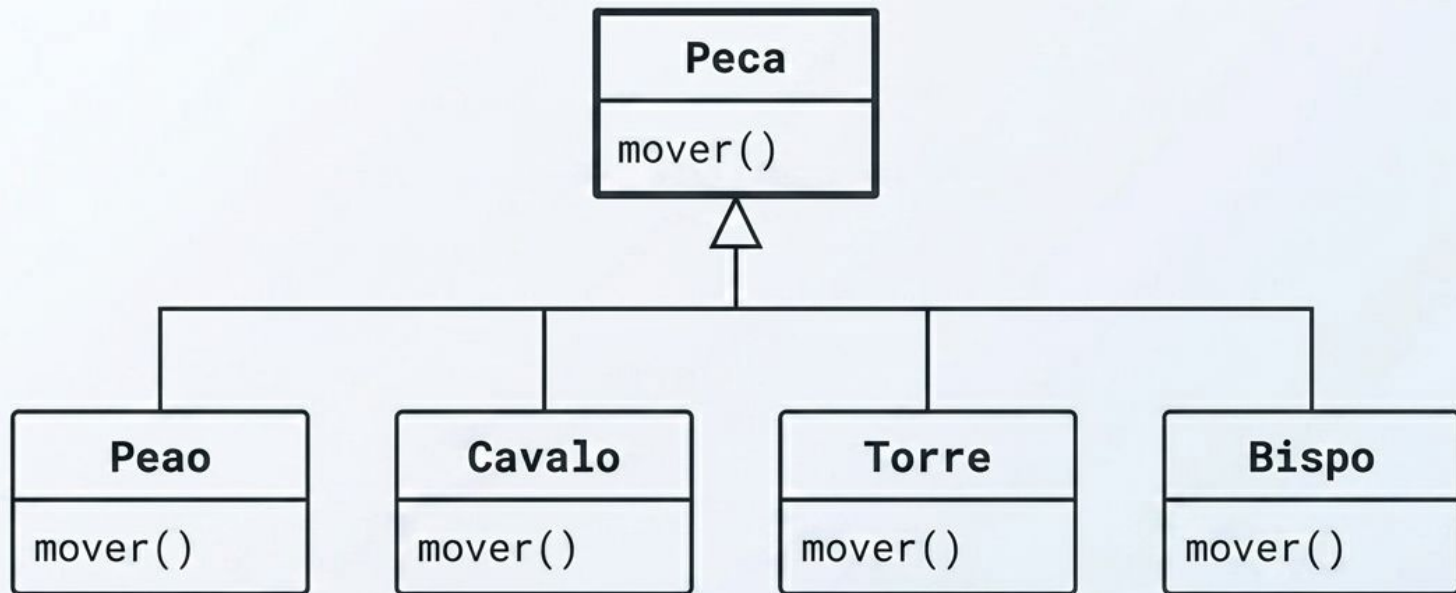
```
class Cavalo(Animal):
    def mover(self):
        return "movendo o cavalo..."
class Torre(Animal):
    def mover(self):
        return "movendo a torre..."
class Bispo(Animal):
    def mover(self):
        return "movendo o bispo..."
```



Polimorfismo

- . Exemplo: jogo de xadrez

```
# fazer importacoes...  
peao = Peao();  
cavalo = Cavalo();  
torre = Torre();  
bispo = Bispo();  
peao.mover(); // movendo peao...  
cavalo.mover(); // movendo cavalo...  
torre.mover(); // movendo torre...  
bispo.mover(); // movendo bispo...
```



Um tabuleiro possui 32 peças de vários tipos. Sem polimorfismo, mover uma peça exigiria um labirinto lógico. Com polimorfismo, é apenas `peca.mover()`.

Traduzindo o Modelo para Python: Polimorfismo na Prática

Contrato (Abstração)

```
from abc import ABC, abstractmethod

class Peca(ABC):
    @abstractmethod
    def mover(self):
        pass
```

Implementação (Concreto)

```
class Peao(Peca):
    def mover(self):
        return 'movendo o peão...'

class Cavalo(Peca):
    def mover(self):
        return 'movendo o cavalo...'

...
```

```
> cavalo = Cavalo()
> cavalo.mover()
// movendo o cavalo...
> █
```

Exercício: Eliminando a Parede de ifs

O Anti-Padrão (Código Rígido)

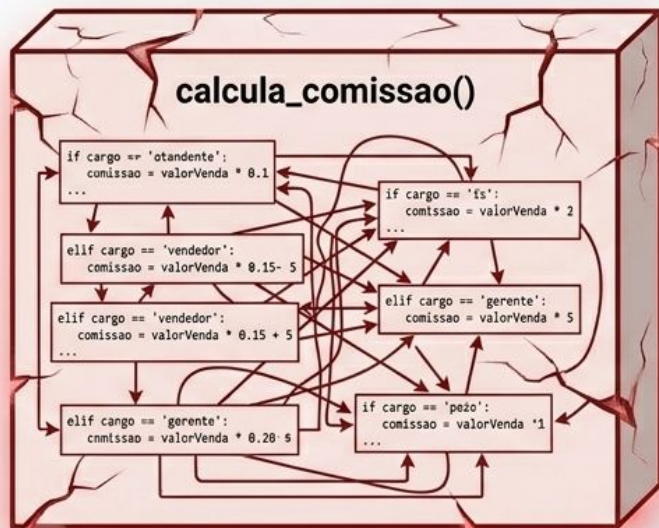
```
if cargo == 'atendente':  
    comissao = valorVenda * 0.1  
elif cargo == 'vendedor':  
    comissao = valorVenda * 0.15 + 5  
elif cargo == 'gerente':  
    comissao = valorVenda * 0.20 + 10
```

Acoplamento Alto:
Qualquer novo cargo
exige modificar este
bloco.

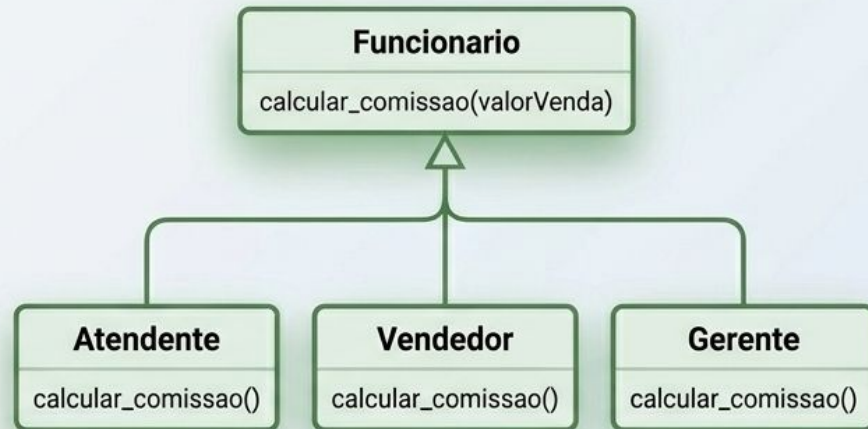
Difícil manutenção.

Desafio: Arquitetura Polimórfica vs. Anti-Padrão Monolítico

O Problema (Anti-Padrão)



A Arquitetura Alvo (Polimórfica)



O desafio: Crie o código para a arquitetura ao lado e substitua as condicionais por uma chamada única: **`funcionario.calcular_comissao(valor)`**.



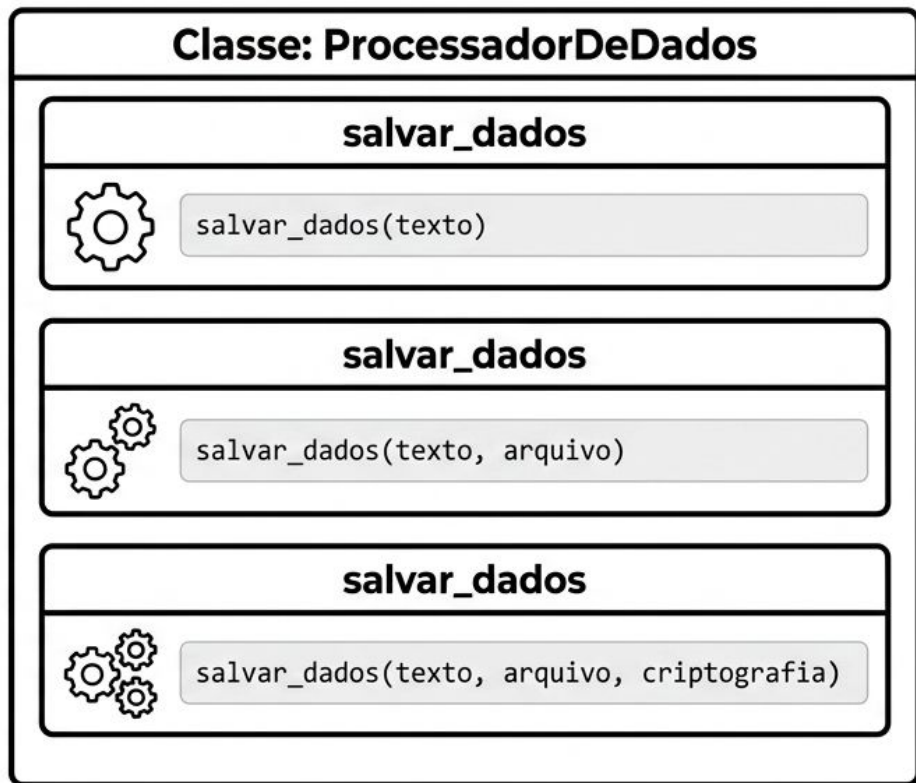
2. Sobrecarga de Métodos (Informação Externa)

Onde Ocorre: Geralmente dentro da mesma classe (não requer herança).

A Regra: Métodos possuem o mesmo nome, mas assinaturas diferentes (quantidade ou tipo de parâmetros distintos).

O Comportamento: Cria opções alternativas para inicializar ou processar dados no mesmo objeto.

A Resolução: Geralmente resolvida em tempo de compilação (Static Binding em linguagens tipadas como Java/C++).



Matriz de Síntese: Sobrescrita vs. Sobrecarga

	Sobreposição (Sobrescrita)	Sobrecarga de Métodos
Localização	Requer herança (Múltiplas classes)	Ocorre na mesma classe
Assinatura	Exatamente a mesma (nome e parâmetros)	Deve ser diferente (parâmetros distintos)
Comportamento	Substitui/estende o método da classe pai	Cria opções alternativas de uso para a mesma ação
Resolução	Tempo de Execução (Dynamic Binding)	Tempo de Compilação (Static Binding)

O objetivo de ambos é criar códigos mais limpos e intuitivos, mantendo a coerência na nomenclatura das ações.

Atenção: A Nuance da Sobrecarga em Python

Diferente de Java ou C++, o **Python puro não suporta sobrecarga nativa**. Se você definir dois métodos com o mesmo nome na mesma classe, a última definição **sobrescreve** a primeira.

A Abordagem Falha



```
class Calculadora:
    def somar(self, a, b):
        return a + b

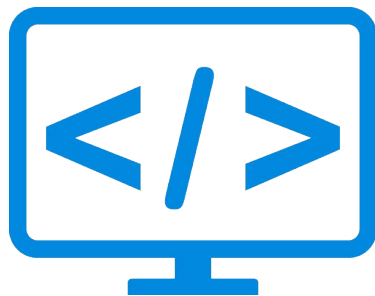
# Python ignora o primeiro método!
def somar(self, a, b, c):
    return a + b + c
```

A Simulação Pythonica



```
class Calculadora:
    # Uso de argumentos padrão (default)
    def somar(self, a, b, c=0):
        return a + b + c

# Ambos funcionam perfeitamente:
calc = Calculadora()
calc.somar(2, 3)      # Simula a assinatura 1
calc.somar(2, 3, 4)  # Simula a assinatura 2
```



Classes Abstratas e Interfaces

Classes Abstratas

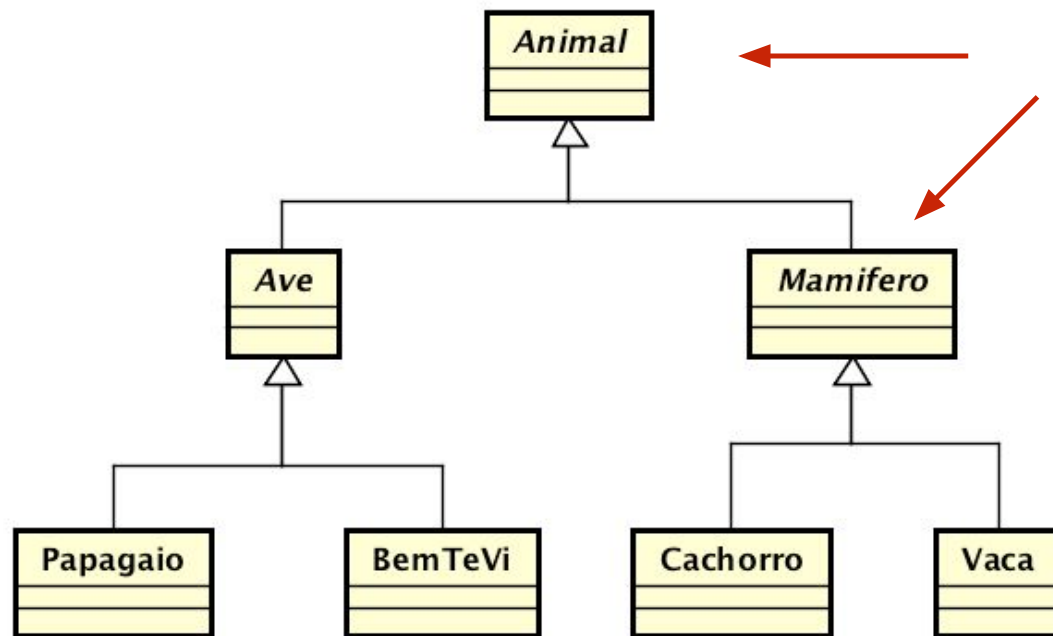
- Há casos em que não fazem sentido existir um objeto, apesar de a classe existir
- Exemplo:
 - Pessoa, PessoaJuridica, PessoaFisica
 - Em um relatório de clientes, existem apenas pessoas físicas e pessoas jurídicas
 - Utilização da classe Pessoa apenas para tirar proveito da herança e polimorfismo
 - Não faz sentido permitir instanciar um objeto da classe Pessoa



Classes Abstratas

- São classes que **não podem** ser instanciadas
- São usadas com herança
- Define-se uma superclasse com características e comportamentos comuns e cada subclasse implementa suas especificidades
- Utilizada apenas para a derivação de novas classes

Classes Abstratas



Na UML, nome da classe em itálico

Classes Abstratas

- Vantagens:
 - Impor a implementação de métodos
 - Incentivar a reutilização de código
 - Melhora a legibilidade
 - Melhora a capacidade de manutenção
 - Incentiva o polimorfismo

Métodos Abstratos

- Os métodos também podem ser abstratos:
 - Não possuem implementação
 - Deve, **obrigatoriamente**, ser implementado nas subclasses concretas (não abstratas)
 - Caso a subclasse seja abstrata, não é necessário que esta forneça implementação para os métodos abstratos
- Toda classe em Python que possui pelo menos um método abstrato, é abstrata
- Classes abstratas podem ter métodos com implementação

Classes Abstratas

- Em Python, são implementadas através do módulo **abc** (abstract base classes) e o decorador **@abstractmethod**

```
from abc import ABC, abstractmethod
```

```
class Forma(ABC):
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def perimetro(self):
```

```
        pass
```

```
from forma import Forma
```

```
class Circulo(Forma):
```

```
    def __init__(self, raio):
```

```
        self.raio = raio
```

```
    def area(self):
```

```
        return 3.14159 * self.raio ** 2
```

```
    def perimetro(self):
```

```
        return 2 * 3.14159 * self.raio
```




Classes Abstratas

- Quando usar:
 - Ao projetar os sistemas em que um determinado comportamento precisa ser imposto
 - Implementar APIs consistentes em todas as classes
 - Fornecimento de uma estrutura básica para hierarquia de classes



Dissecando o Código

O Molde (Abstrato)

```
class Forma(ABC):  
    @abstractmethod  
    def area(self):  
        pass
```



Contrato
Cumprido

A Realização (Concreto)

```
class Circulo(Forma):  
    def __init__(self, raio):  
        self.raio = raio  
  
    def area(self):  
        return 3.14159 * self.raio ** 2
```

Por que utilizar Classes Abstratas?



Impor Implementação

Fornecer uma estrutura básica (API consistente) e obrigar o comportamento de métodos vitais nas subclasses.



Reutilização de Código

Evita duplicação ao compartilhar lógica funcional e atributos comuns diretamente na superclasse.



Fácil Manutenção

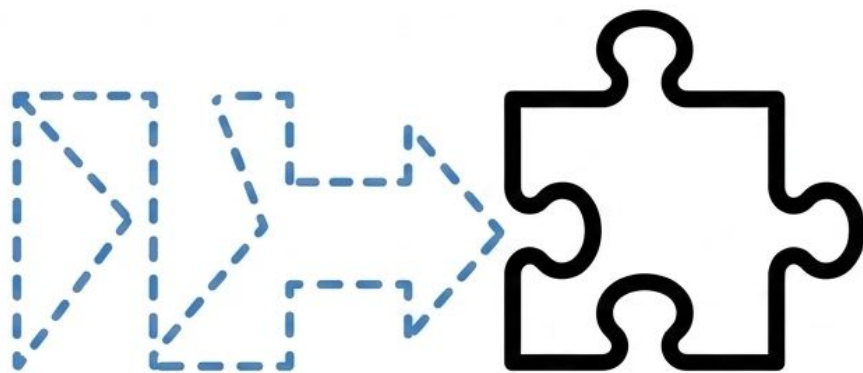
Melhora imensamente a legibilidade do sistema ao centralizar as regras de negócio em um único molde.



Polimorfismo

Permite que o sistema trate diferentes subclasses concretas de forma totalmente homogênea e padronizada.

A Evolução: Além da Herança de Código



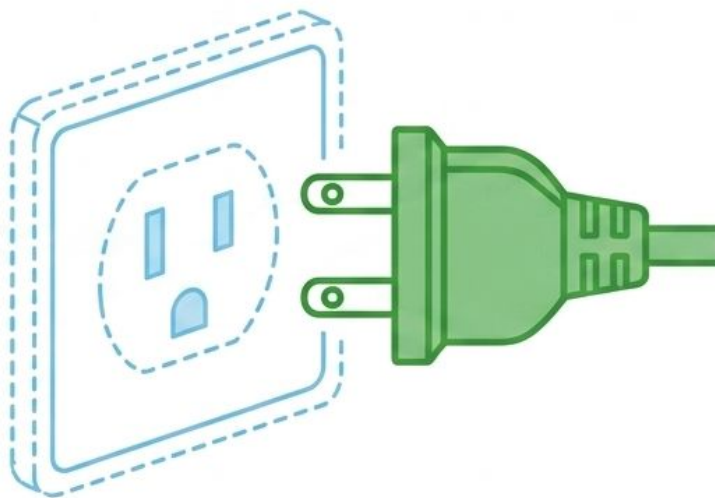
Até agora, vimos que classes abstratas podem **misturar métodos implementados** com métodos vazios. Mas e se o design do sistema exigir o compartilhamento apenas de regras, com **absolutamente zero** implementação?

Interfaces em Python (O Contrato Puro)

Python não tem uma palavra-chave 'interface' nativa. Em Python, uma Interface é simplesmente uma **Classe Abstrata** onde 100% dos métodos são abstratos.

A Interface (O Contrato)

Dita o formato e as regras, mas não possui lógica própria.



A Classe Concreta

Fornece a energia, assinando e cumprindo o contrato.

Key Concept: Uma interface pura não guarda estado (variáveis) nem código funcional. Se a sua classe assina o contrato, ela garante ao sistema que sabe executar aquelas ações.

Definindo uma Interface na Prática

```
class Imprimivel(ABC):  
    @abstractmethod  
    def imprimir(self):  
        pass  
  
    @abstractmethod  
    def formatar_saida(self):  
        pass
```

Zero implementação.
Apenas assinaturas.

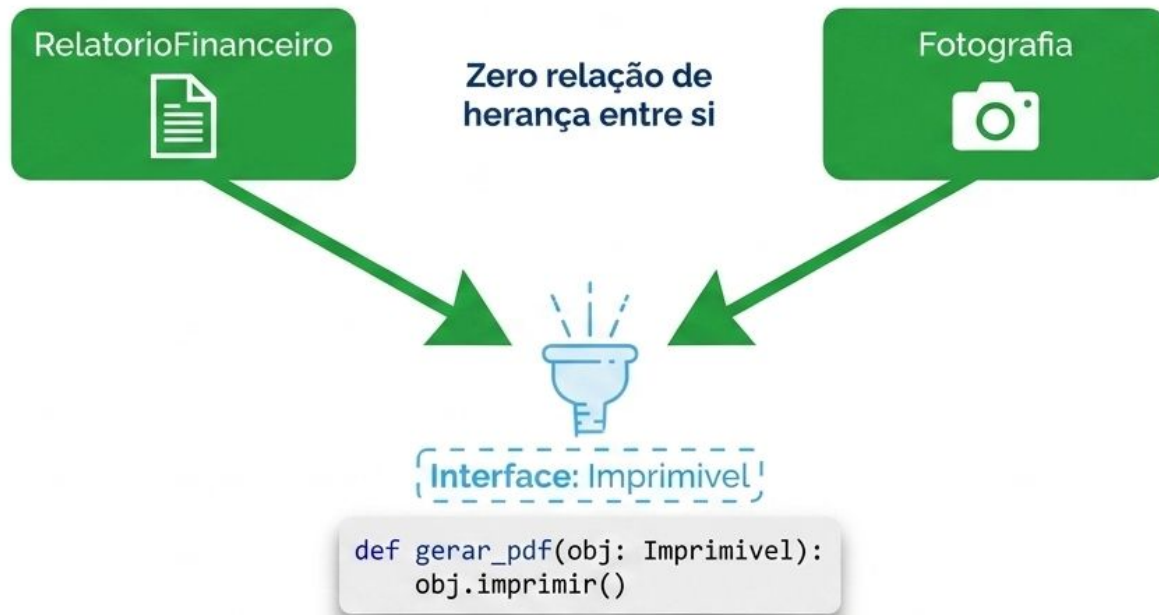
Boas Práticas de Nomenclatura:

Interfaces costumam receber nomes que indicam capacidade ou comportamento.

- Terminados em '-ável' ou '-ível' (Ex: Imprimivel, Serializavel, IE: Serializavel, Autenticavel).
- Ou prefixados com a letra 'I' maiúscula (Ex: Impressora, IDocumento).

O Poder do Polimorfismo com Interfaces

A maior vantagem da **Interface** é desvincular o **comportamento** da **hierarquia** da classe.



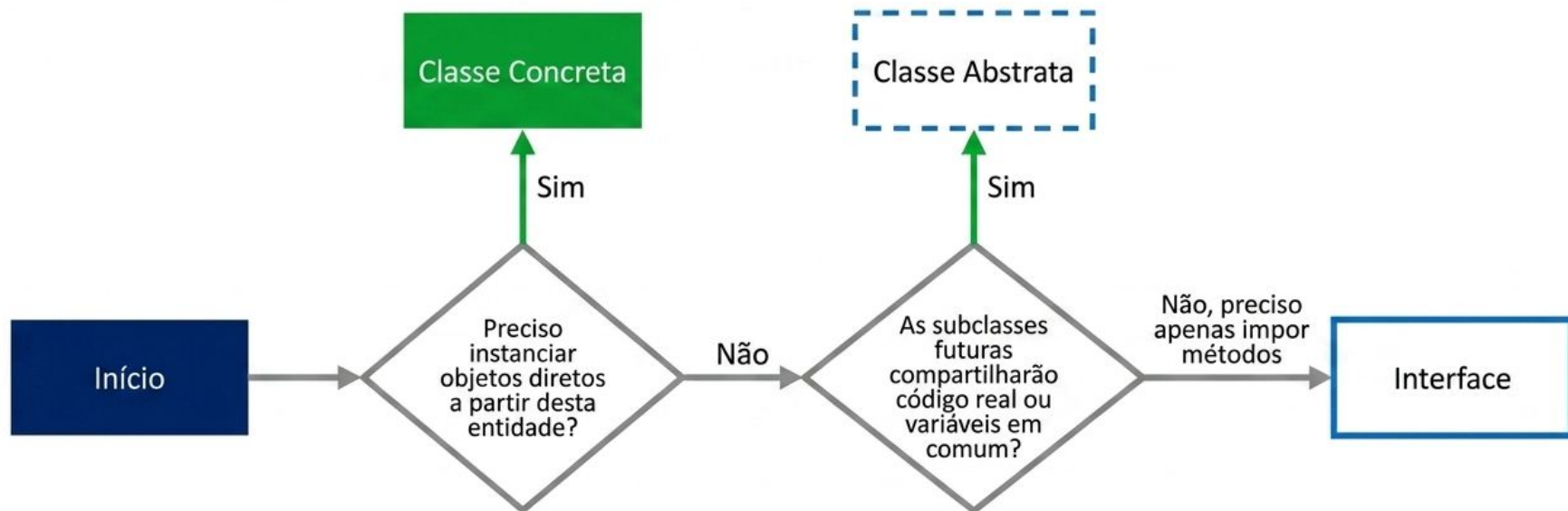
O sistema aceita ambos os objetos sem se importar com a sua classe base, pois **ambos garantem cumprir o contrato**.

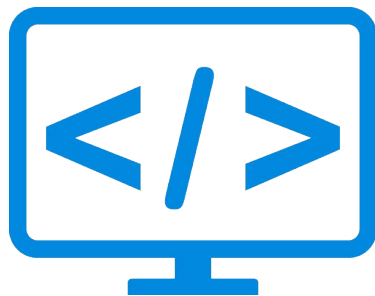
Matriz de Diagnóstico Arquitetural

	Classe Concreta	Classe Abstrata	Interface
Pode ser instanciada?	✓	✗	✗
Código implementado?	100% implementado	Misto (Abstrato + Real)	0% (Apenas assinaturas)
Possui estado (variáveis)?	✓	✓	✗
Objetivo Principal	Criar objetos	Estrutura base + Regras	Contrato puro



Fluxograma de Decisão





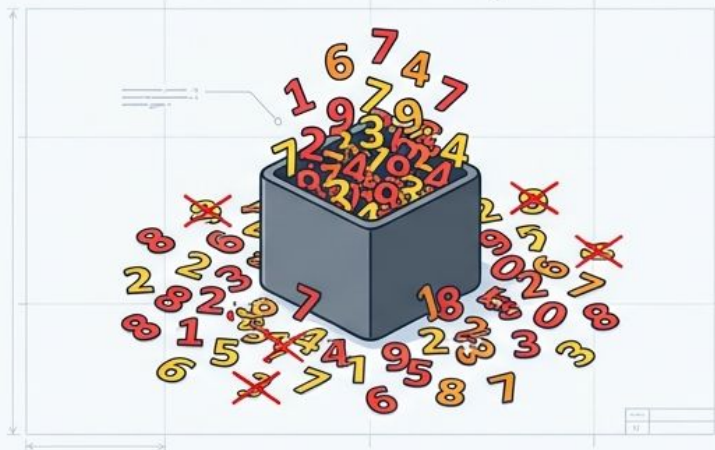
Coleções

Listas

- .Há problemas simples que não podem ser adequadamente resolvidos com as variáveis, estruturas de seleção e repetição
- .Exemplo: ler uma lista de números e depois imprimir esta lista na ordem inversa à que foi lida
- .Quanto são os números?
- .Como calcular a ordem inversa?

O Problema da Escala: Por que variáveis simples não bastam?

ANTES: Variável Simples

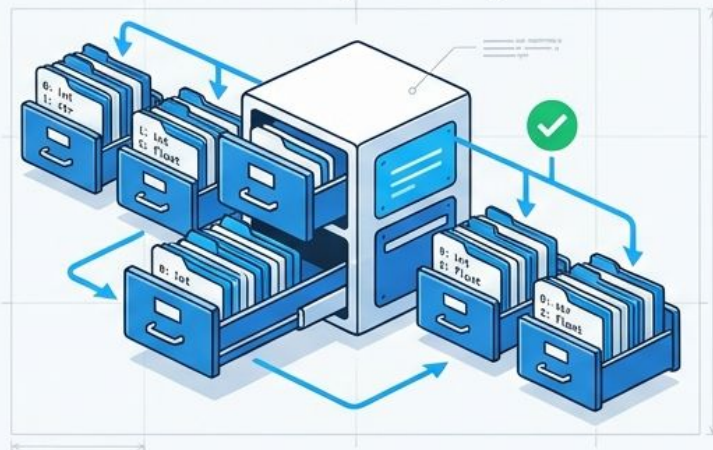


Variável Simples

Variáveis tradicionais armazenam apenas um valor por vez. Resolver problemas reais exige múltiplas variáveis isoladas, o que quebra a lógica do código.

```
// Exemplo:  
int score = 85;  
// Erro ao tentar: score = 85, 90, 72;  
// ✗ FALHA DE ESCALA
```

DEPOIS: Coleções em Python



Coleções em Python

Coleções agrupam múltiplos dados sob um único nome, permitindo o processamento em massa e a manipulação inteligente de informações.

```
// Exemplo: scores = [85, 90, 72, 95, 88]  
// ✓ SOLUÇÃO ESCALÁVEL  
// Processamento em massa:  
for s in scores: process(s)
```

Listas

- Uma variável do tipo **lista** em Python podem conter uma sequência de elementos (valores), e não apenas um (número, texto, etc)
- São criadas com uma sequência de valores entre colchetes e separados por vírgula

```
for i in [1, 2, 3, 4]:  
    print(i)
```

1
2
3
4

```
minha_lista = [4, 5, 3.8, 'texto']  
for i in minha_lista:  
    print(i)
```

4
5
3.8
texto

Listas

- Uma lista em Python é uma estrutura de dados heterogênea, pois suas posições podem ser ocupadas por valores de tipos distintos.
- É possível **acessar** um valor de uma lista através do uso de um índice ou posição.
- Esse índice é acessado colocando o nome da variável da lista seguido de uma expressão numérica entre colchetes
- O primeiro elemento da lista está no índice **zero**.

```
minha_lista = [4, 5, 3.8, 'texto']  
print(minha_lista[0])  
print(minha_lista[1])  
print(minha_lista[2])  
print(minha_lista[3])
```

```
4  
5  
3.8  
texto
```

Listas

- .Criando listas vazias
- .Descobrimo o tamanho de uma lista: função len()
- .Convertendo uma string em lista de caracteres: função list()

```
lista = []
```

```
lista = [1, 3, 10]  
print(len(lista))
```

3

```
texto = "abc"  
lista = list(texto)  
print(lista)
```

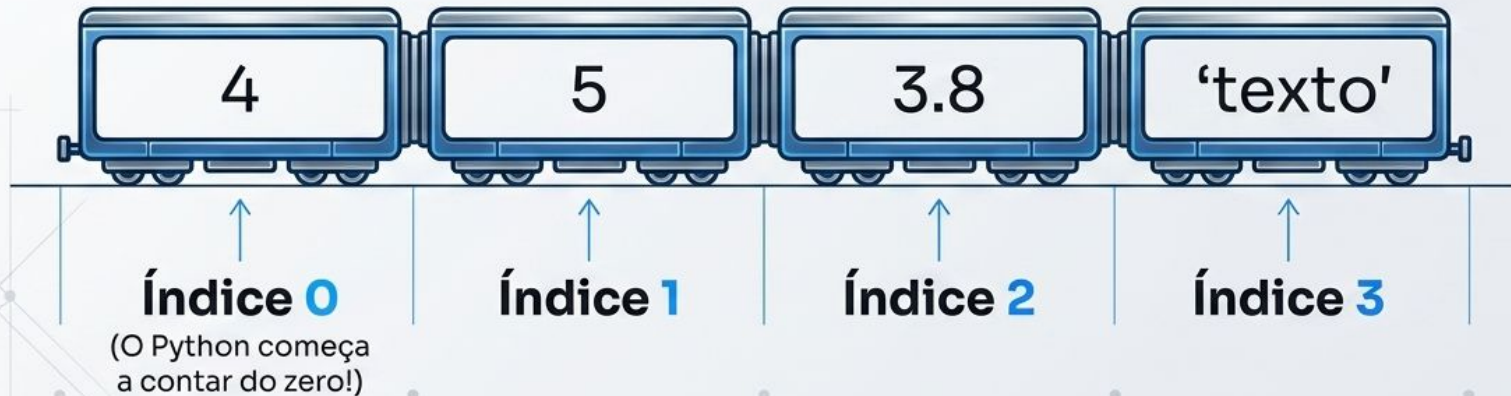
```
['a', 'b', 'c']
```

Anatomia de uma Lista

```
minha_lista = [4, 5, 3.8, 'texto']
```

Colchetes definem a lista. Elementos são separados por vírgula.

Heterogênea. Suporta inteiros, decimais e textos na mesma estrutura.



O Kit de Ferramentas Essencial

Criando uma lista vazia

```
lista = []
```

Descobrimo o tamanho

```
lista = [1, 3, 10]  
print(len(lista))
```

3

Texto para Lista

```
texto = "abc"  
lista = list(texto)  
print(lista)
```

['a', 'b', 'c']

Listas

- Adicionando elementos na lista:
- Função `append()`

```
minha_lista = [4, 5, 3.8, 'texto']  
print(minha_lista[0])  
print(minha_lista[1])  
print(minha_lista[2])  
print(minha_lista[3])  
minha_lista.append('testando')  
print(minha_lista[4])
```

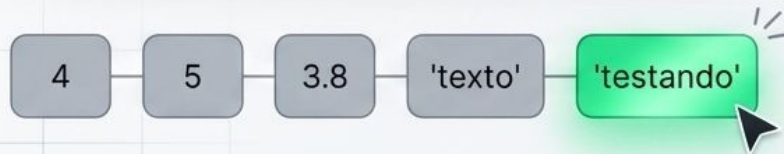
```
4  
5  
3.8  
texto  
testando
```

Modificando Listas: Adição de Elementos

append()

Adiciona o elemento sempre ao final da lista.

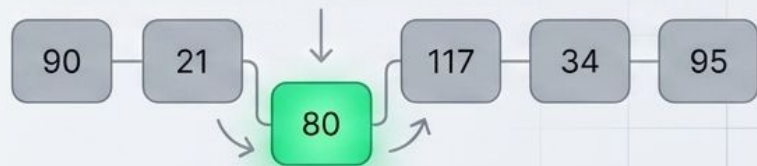
```
minha_lista = [4, 5, 3.8, 'texto']  
minha_lista.append('testando')
```



insert(posição, valor)

Empurra os elementos existentes e insere o novo valor em um índice exato.

```
lista = [90, 21, 117, 34, 95]  
lista.insert(2, 80)
```



Listas

.Exemplo: ler uma lista de números e depois imprimir esta lista na ordem inversa à que foi lida

```
QUANT = 5
minha_lista = []
for i in range(QUANT):
    lido = input('[{0}]:'.format(i))
    minha_lista.append(lido)
for i in range(QUANT - 1, -1, -1):
    print(minha_lista[i])
```

```
[0]:1
[1]:2
[2]:3
[3]:4
[4]:5
5
4
3
2
1
```

Listas

.Função reversed()

.Retorna uma cópia da lista com os elementos ordenados do último para o primeiro

```
QUANT = 5
minha_lista = []
for i in range(QUANT):
    lido = input('[{0}]:'.format(i))
    minha_lista.append(lido)
for elemento in reversed(minha_lista):
    print(elemento)
```

O Desafio da Ordem Inversa



```
Entrada lida: [1, 2, 3, 4, 5]
```

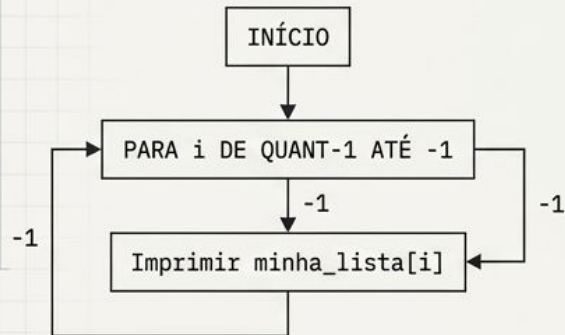
```
Saída desejada: 5, 4, 3, 2, 1
```

Como capturamos dados em uma ordem, mas processamos e imprimimos na ordem exatamente oposta?

Existem duas formas de pensar: a lógica matemática manual e o “Jeito Pythonic”.

Duas Abordagens: Lógica vs. Python

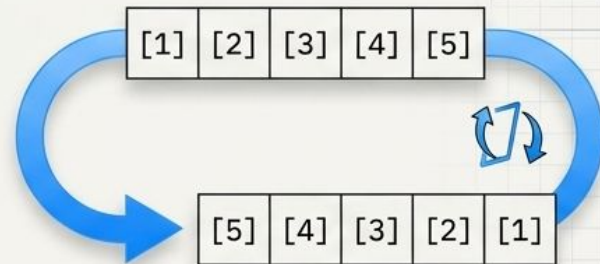
Iteração Manual



```
for i in range(QUANT - 1, - 1, - 1):  
    print (minha_lista[i])
```

Usa matemática para forçar os índices de trás para frente. Essencial para entender como os índices funcionam.

Função Nativa



```
for elemento in reversed(minha_lista):  
    print (elemento)
```

A função `reversed()` retorna uma cópia invertida sem matemática complexa. Mais limpo, mais rápido e menos suscetível a erros.

Listas

.Função insert()

.Insere um elemento em determinado índice da lista

```
lista = [90, 21, 117, 34, 95]  
lista.insert(2, 80)  
print(lista)
```

```
[90, 21, 80, 117, 34, 95]
```


Listas

.Função insert()

.Criando a lista já invertida

```
QUANT = 5
minha_lista = []
for i in range(QUANT):
    lido = input('[{0}]:'.format(i))
    minha_lista.insert(0, lido)
for elemento in minha_lista:
    print(elemento)
```

Listas

.Removendo elementos

.Função remove() remove a primeira ocorrência de um elemento de uma lista

```
lista = [8, 4, 9, 6]          [8, 9, 6]
lista.remove(4)               [7, 9, 6, 5]
print(lista)                  [7, 9, 5]

lista = [9, 7, 9, 6, 5]
lista.remove(9)
print(lista)

# A remoção pode ocasionar em erros caso
# o elemento não se encontre na lista.
# Para isso, pode-se verificar se ele está
# ou não na lista antes de remover
if 6 in lista:
    lista.remove(6)

print(lista)
```

Modificando Listas: Remoção Segura

```
lista.remove(4)
```

Remove a primeira ocorrência exata do elemento "4".

Prática de Segurança

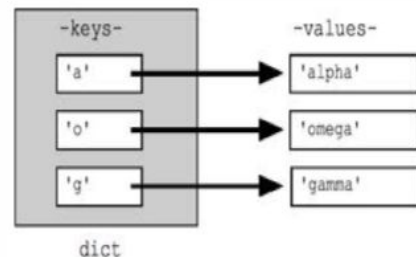
Tentar remover um elemento que não existe quebra o código. Verifique sempre a existência do valor antes de agir:

```
if 6 in lista:  
    lista.remove(6)
```

O operador "in" age como um radar preventivo antes da execução.

Dicionários

- .Estrutura que permite que outros tipos de valores além de inteiros sejam usados como índices
- .Os dicionários são representados por **chaves** `{}`
- .Cada elemento é representado por uma **chave** e um **valor**, no seguinte formato:
`{chave : valor}`
- .As chaves podem ser de qualquer tipo, exceto listas e dicionários. Os valores podem ser de qualquer tipo



Dicionários

.Exemplo: imagine uma entrevista com um grupo de pessoas e, entre outras coisas, precisa saber o ano em que elas nasceram. No final, deve ser impresso a quantidade de pessoas que nasceram em cada ano.

```
nascimentos = {}  
while True:  
    ano = input('Qual o ano de nascimento?')  
    if ano == '':  
        break  
    nascimentos[ano] = nascimentos.get(ano, 0) + 1  
for ano in nascimentos:  
    print('Ano {0}: {1} pessoas '.format(ano, nascimentos[ano]))
```

Aplicação Prática: Contagem de Frequência

Entrevistamos pessoas. Quantas nasceram em cada ano?

```
nascimentos[ano] = nascimentos.get(ano, 0) + 1
```

Busque ou crie a chave para este ano...

...e defina o valor como...

...o valor atual deste ano (ou retorne **ZERO** se ele ainda não existir)...

...e some um a este valor.

O método `.get()` é o truque mestre em Python para contar ocorrências sem gerar erros de “chave não encontrada”.

Dicionários

.Para imprimir as chaves do dicionário:

.Para imprimir os valores das chaves:

```
dic = {'Pedro':1, 'Maria':4, 'João':9, 'Rui':6}
for elemento in dic:
    print(elemento)
```

```
dic = {'Pedro':1, 'Maria':4, 'João':9, 'Rui':6}
for elemento in dic:
    print(dic[elemento])
```


A Evolução: Por que Dicionários?



Listas são incríveis, mas dependem de números (0, 1, 2) para encontrar dados.

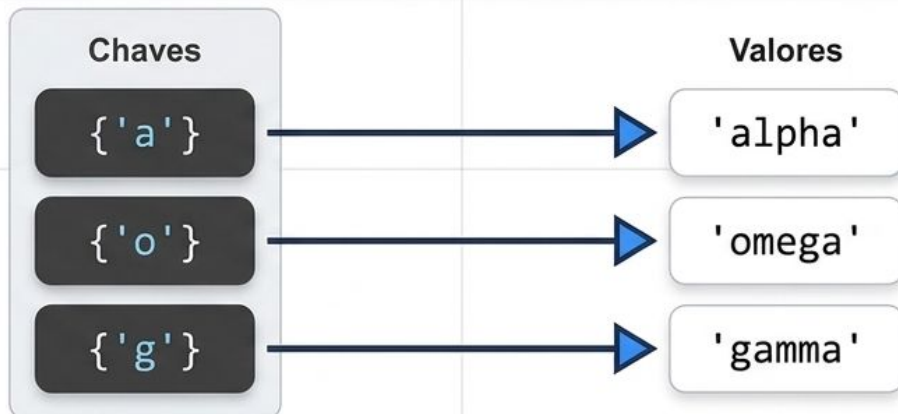
E se você não souber a posição do dado?

E se você quiser buscar informações por um "nome" ou "rótulo"?

Dicionários permitem que outros tipos de valores (textos, dados imutáveis) sejam usados como índices. Onde a Lista usa a **posição**, o **Dicionário** usa a **identidade**.

Anatomia de um Dicionário { }

```
dicionario = {'a': 'alpha', 'o': 'omega', 'g': 'gamma'}
```



Regra 1:

Usa chaves { } em vez de colchetes [].



Regra 2:

Cada elemento é um par {chave : valor}.



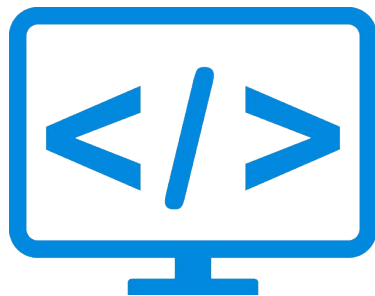
Regra 3:

Valores podem ser qualquer coisa. Chaves NÃO podem ser listas ou outros dicionários.

Matriz de Decisão: Qual Estrutura Escolher?

	Listas []	Dicionários { }
Sintaxe	Colchetes [a, b, c]	Chaves {a: 1, b: 2}
Como acessar?	Por posição (Índices inteiros 0, 1, 2)	Por rótulo (Chave nomeada)
Melhor Caso de Uso	Filas, sequências lógicas, histórico, dados ordenados que repetem.	Associações, catálogos, contagem de elementos, buscar um dado rapidamente por nome.

Use Listas para ordem. Use Dicionários para associação e busca.



Tratamento de exceções

Introdução

- . Exceções são erros que ocorrem durante a execução de um programa
- . Quando ocorre um erro, o programa é interrompido e uma mensagem de erro é exibida



Introdução

- . Por que tratar exceções?
 - **Evita falhas inesperadas:** sem tratamento, um erro simples pode encerrar toda a aplicação
 - **Garante melhor experiência do usuário:** mensagens claras evitam confusão e frustração
 - **Facilita a depuração e manutenção:** erros tratados podem ser registrados (logs) para análise posterior
 - **Permite ações de recuperação:** como tentar uma alternativa, repetir uma operação ou limpar recursos

Introdução

- . Consequências de não tratar exceções:
 - O programa é interrompido abruptamente
 - Dados podem ser corrompidos ou perdidos
 - Arquivos ou conexões podem ficar abertos
 - O usuário vê uma mensagem técnica confusa

Exceções comuns

- **ZeroDivisionError:** ocorre ao tentar dividir um número por zero
- **ValueError:** ao passar um valor inválido para uma função
- **IndexError:** ao tentar acessar um índice inexistente em uma lista
- **KeyError:** ao acessar uma chave inexistente em um dicionário

ZeroDivisionError

```
resultado = 10 / 0
```

ValueError

```
numero = int("dez")
```

IndexError

```
lista = [1, 2, 3]  
print(lista[5])
```

KeyError

```
dados = {"nome": "Ana"}  
print(dados["idade"])
```

Erro de Lógica vs Erro de Execução

- Qual a diferença entre erro de lógica e erro de execução?

Erro de Lógica

- O programa roda sem gerar exceções, mas produz resultados errados
- Difícil de detectar: o código “parece funcionar”
- Exige análise cuidadosa da lógica do algoritmo

```
# Objetivo: calcular média de 2 notas  
media = nota1 + nota2 / 2  
# ERRO: precedência de operadores
```

Erro de Execução

- Ocorre durante a execução do programa
- Gera uma interrupção imediata, a menos que seja tratado
- Pode ser detectado com try-except

```
numero = int("abc") # ValueError
```


Erro de Lógica vs Erro de Execução

- Nem todo erro está relacionado a exceções
- Tratamento de exceções não corrige erros de lógica, mas ajuda a lidar com falhas em tempo de execução



Estrutura básica do try-except

- Em python, usamos os blocos try-except para capturar e tratar as exceções
- Sintaxe:

```
try:  
    # Bloco de código que pode causar erro  
    operacao_risco()  
except TipoDeErro:  
    # Bloco executado se o erro ocorrer  
    tratar_erro()
```



Estrutura básica do try-except

. Exemplo

```
try:
    numero = int(input("Digite um número: "))
    resultado = 10 / numero
    print(f"Resultado: {resultado}")
except ValueError:
    print("Erro: entrada inválida.")
except ZeroDivisionError:
    print("Erro: divisão por zero.")
```

Tenta executar o que está dentro do bloco try

Se der erro, o except correspondente será executado

Usando except genérico

- . Vantagens:
 - Evita que o programa quebre completamente
 - Pode ser útil em blocos amplos ou quando não se conhece todos os tipos de erro possíveis
- . Desvantagens:
 - Oculta a causa real do erro, dificultando a depuração
 - Torna o código menos previsível e menos seguro
 - Prejudica o feedback para o usuário ou desenvolvedor

```
try:  
    operacao()  
except:  
    print("Ocorreu um erro.")
```

Usando except genérico

- . Boa prática:
 - Use except específico sempre que possível
 - Se usar except genérico, registre o erro com **logging** para diagnóstico
 - Logging: <https://docs.python.org/3/library/logging.html>

Múltiplos excepts e o uso de else

- Usar múltiplos excepts quando diferentes tipos de erro exigem tratamentos diferentes

```
try:  
    # Código com risco de erro  
    operacao()  
except ValueError:  
    print("Erro de valor inválido.")  
except ZeroDivisionError:  
    print("Erro: divisão por zero.")  
except Exception:  
    print("Erro desconhecido.")  
else:  
    print("Operação realizada com sucesso.")
```

O **primeiro except compatível** com o erro será executado

O bloco **else** só é executado se nenhum erro ocorrer no bloco try

Permite separar claramente tratamento de erros da lógica principal

O bloco finally

- . O bloco finally é executado **sempre, com ou sem exceção**
- . Ideal para encerrar recursos, como:
 - Fechar arquivos
 - Encerrar conexões
 - Liberar memória ou recursos externos

```
try:
    arquivo = open("dados.txt", "r")
    # Leitura do arquivo
except FileNotFoundError:
    print("Arquivo não encontrado.")
finally:
    print("Encerrando processo...")
    arquivo.close() # mesmo que tenha ocorrido erro
```

Exceções em Funções

- É possível (e recomendado) tratar exceções dentro da própria função para torná-la mais robusta e reutilizável

```
def dividir(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError:  
        print("Erro: divisão por zero.")  
        return None  
print(dividir(10, 0)) # Resultado: None
```


Exceções Aninhadas

- . É possível ter try-except dentro de outro bloco try
- . Útil quando há várias operações em sequência, com riscos diferentes

```
try:
    x = int(input("Digite um número: "))
    try:
        resultado = 10 / x
        print(f"Resultado: {resultado}")
    except ZeroDivisionError:
        print("Divisão por zero.")
except ValueError:
    print("Entrada inválida.")
```

Usando raise para lançar erros

- A palavra-chave **raise** em Python é usada para lançar uma exceção manualmente
- Útil quando você deseja forçar a interrupção do programa ao detectar uma situação inválida

```
def definir_idade(idade):  
    if idade < 0:  
        raise ValueError("Idade não pode ser negativa.")  
    print(f"Idade registrada: {idade}")  
definir_idade(-5)  
>>> ValueError: Idade não pode ser negativa.
```

Usando raise para lançar erros

- **raise** pode ser usado com erros internos do Python (ValueError, TypeError, etc.)
- Também pode ser usado com exceções personalizadas (criação de classes que herdam de Exception)

```
def definir_idade(idade):  
    if idade < 0:  
        raise ValueError("Idade não pode ser negativa.")  
    print(f"Idade registrada: {idade}")  
definir_idade(-5)  
>>> ValueError: Idade não pode ser negativa.
```

Exceções Personalizadas

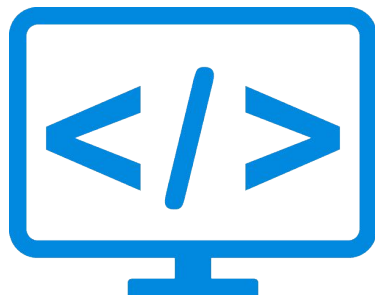
- . Pode-se criar exceções personalizadas para representar erros específicos do seu domínio de aplicação
- . Permite criar mensagens de erro mais claras e descritivas
- . Facilita a manutenção e a depuração de sistemas complexos

```
class SaldoInsuficienteError(Exception):  
    pass
```



Exceções Personalizadas

```
def sacar(saldo, valor):  
    if valor <= 0:  
        raise ValueError("O valor do saque deve ser positivo.")  
    if valor > saldo:  
        raise SaldoInsuficienteError("Saldo insuficiente para realizar o saque.")  
    return saldo - valor  
# Teste  
try:  
    saldo_atual = sacar(500, 700)  
except ValueError as ve:  
    print(f"Erro de valor: {ve}")  
except SaldoInsuficienteError as se:  
    print(f"Erro de saldo: {se}")
```



Serialização e persistência de objetos

Introdução

- . Durante a execução de um programa, os dados ficam em **memória**
- . Quando um programa termina ou o computador é desligado, os dados da memória são **perdidos**
- . Para armazenar dados permanentemente, é necessário grava-los em um **arquivo**

Introdução

- . Trabalhar com arquivos é semelhante a trabalhar com livros:
 - Para utilizar, tem que **abri-lo**
 - Ao final, é necessário **fecha-lo**
 - Enquanto o livro estiver aberto, é possível tanto **ler** quanto **escrever** nele.

Abrindo um arquivo

- Função **open()**:
 - Recebe dois argumentos: o nome do arquivo e o modo
 - Modos:
 - **w** para sobrescrever (se o arquivo não existir, cria)
 - **r** para ler
 - **a** para escrever a partir do final (não apaga o conteúdo existente)
 - **r+** para leitura e sobrescrita
 - **a+** para leitura e escrita

Abrindo um arquivo

- No exemplo abaixo, se não existir o arquivo teste.txt, ele será **criado** (somente no modo escrita!)

```
- f = open("teste.txt", "w")  
- print(f)  
- # <_io.TextIOWrapper name='teste.txt'  
  mode='w' encoding='UTF-8'>
```

Escrevendo em um arquivo

- Função **write()**:
 - Escreve no arquivo aberto
- Função **close()**:
 - Fecha o arquivo

```
- f.write("Escrevendo no arquivo")  
- f.write(" para depois fecha-lo")  
- f.close()
```

Lendo em um arquivo

. Função `read()`

```
- f = open("teste.txt" , "r")  
- texto = f.read()  
- print(texto)  
- texto = f.read(10)  
- print(texto)
```

Escrevendo em um arquivo

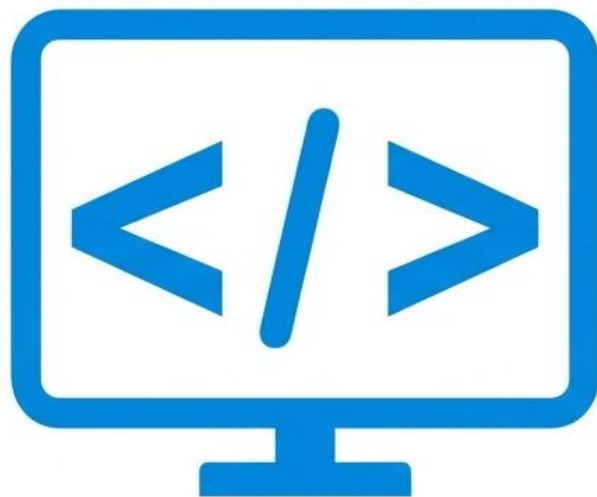
- Escrevendo mais de uma linha:

```
- f = open("teste2.txt", "w")  
- f.write("Linha 1\nLinha 2\nLinha 3")  
- f.close()  
- f = open("teste2.txt", "r")  
- print(f.readline())  
- # Linha 1  
- print(f.readlines())  
- # ['Linha 1', 'Linha 2', 'Linha 3']
```

Gravando variáveis

- A escrita em um arquivo deve ser em **string**

```
- x = 52
- f.write(str(x))
- carros = 52
- f.write("Foram vendidos {} carros".format(carros))
- f.write("Em %d dias fizemos %f milhões %s." %
(34,6.1,'reais'))
```

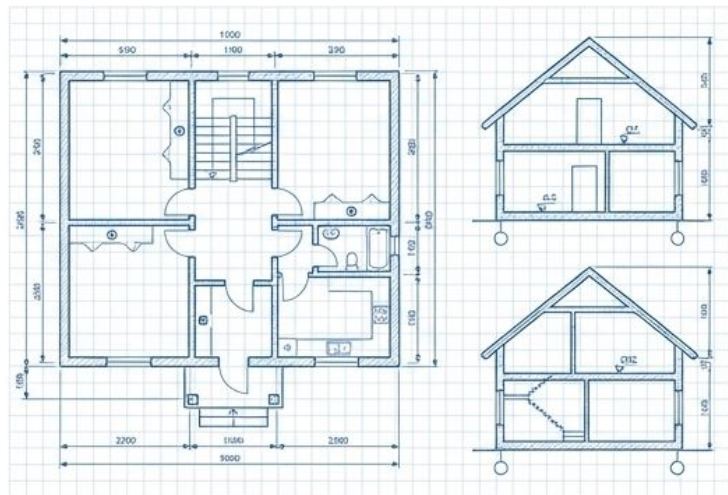


Leitura e Compreensão de Diagrama de Classes

O Manual de Tradução: Do Projeto UML para a Implementação em Python

PROGRAMAÇÃO ORIENTADA A OBJETOS

A Planta Baixa do Software (UML)



O Projeto

A Unified Modeling Language (UML) é a linguagem visual universal da engenharia de software. Ela define 'o que' construir antes de nos preocuparmos com o 'como'.

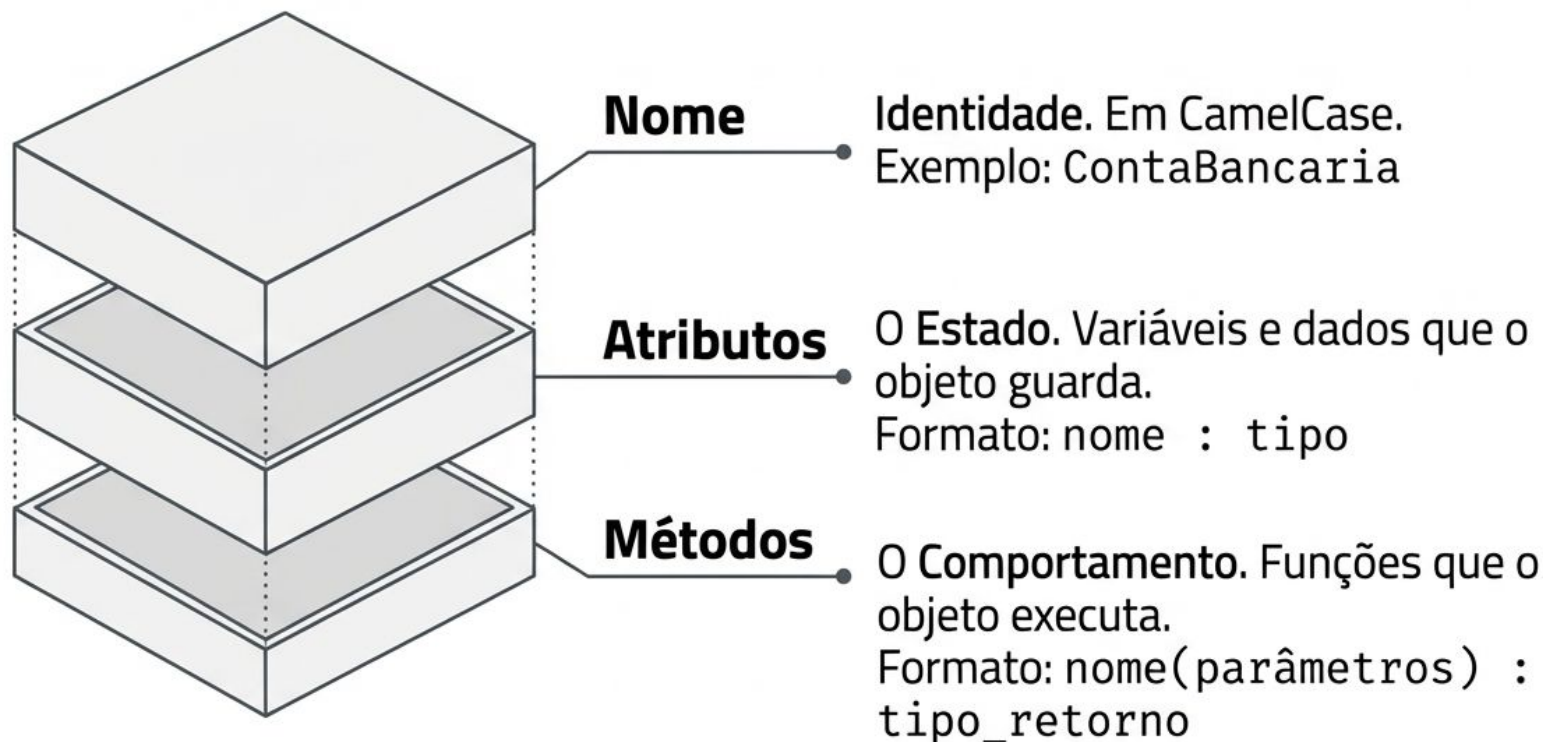


A Construção

O código (Python, Java, C++) é a execução materializada. Ler um diagrama UML com fluência significa saber exatamente qual código escrever.

UML não é código, é contrato. Se você entende o diagrama, a tradução para a Programação Orientada a Objetos torna-se quase automática.

Anatomia de uma Classe UML



Tradução Base: Estrutura da Classe

Pessoa
- nome: String - idade: int
+ falar()



```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def falar(self):
        print(f"Olá, sou {self.nome}")
```

A Matriz de Visibilidade

Símbolo UML	Visibilidade	Significado	Convenção em Python
+	Público	Acessível por qualquer classe.	<code>self.atributo</code> (Padrão)
-	Privado	Restrito à própria classe.	<code>self.__atributo</code> (Dunder)
#	Protegido	Acessível pela classe e suas filhas (herança).	<code>self._atributo</code> (Underscore)
~	Pacote	Acessível dentro do mesmo pacote/módulo.	(Sem equivalente estrito no Python)

Exemplo Prático:

UML: - senha: String → Python: `self.__senha`

Relacionamentos: Associação e Multiplicidade

Classes não vivem isoladas. Uma **Associação** (linha contínua) indica que objetos de uma classe "conhecem" ou "usam" objetos de outra.



Multiplicidade

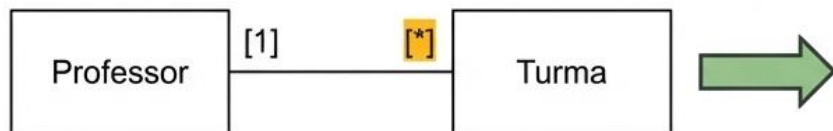
1 : Exatamente um.

0..1 : Zero ou um (Opcional).

* ou 0..* : Zero ou muitos
(Gera uma lista).

1..* : Pelo menos um a muitos.

Traduzindo Associação e Multiplicidade



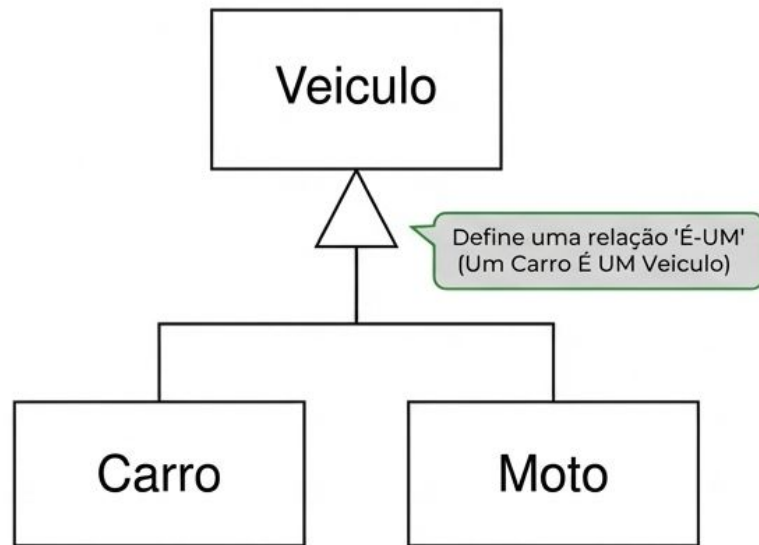
```
class Professor:
    def __init__(self, nome):
        self.nome = nome
        # O multiplicador '*' exige uma
        coleção (Lista)
        self.turmas = []

    def adicionar_turma(self, turma):
        self.turmas.append(turma)
```

Integridade Estrutural: Herança (Generalização)

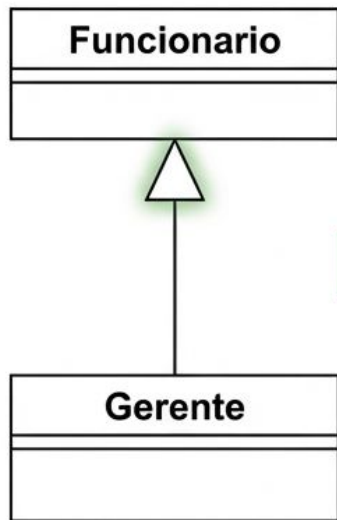
Representada por uma seta com triângulo vazio apontando para a classe 'Pai' (Superclasse).

A classe filha herda automaticamente atributos e métodos da classe pai, promovendo o reaproveitamento de código.





Traduzindo Herança

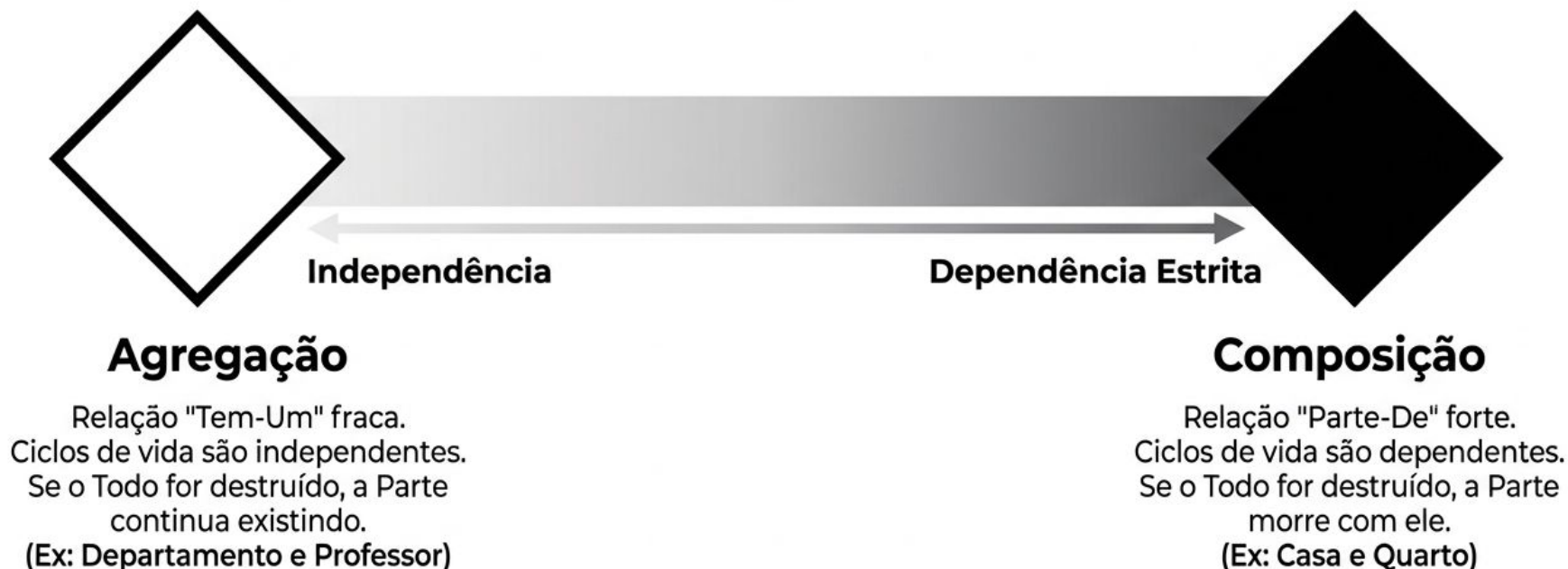


```
class Funcionario:
    def __init__(self, nome, salario):
        self.nome = nome
        self.salario = salario
```

A seta indica que Gerente herda de Funcionario

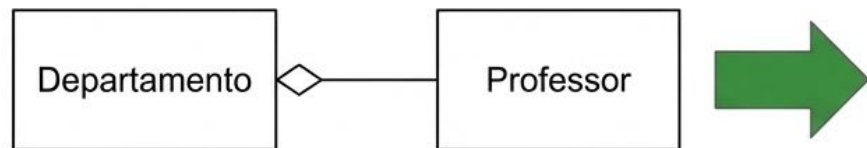
```
class Gerente(Funcionario):
    def __init__(self, nome, salario, setor):
        # Acionando o construtor da classe Pai
        super().__init__(nome, salario)
        self.setor = setor
```

O Espectro de Dependência



Traduzindo Agregação (Independência)

Como o Professor existe independentemente do Departamento, nós o recebemos como um parâmetro já instanciado.

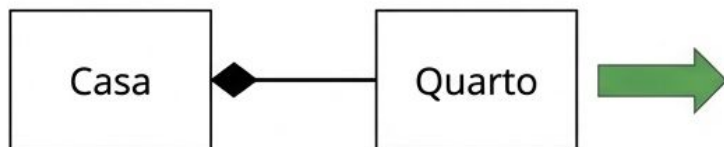


```
class Departamento:
    # O professor é "injetado" de fora
    def __init__(self, nome, professor):
        self.nome = nome
        self.professor = professor

prof_carlos = Professor("Carlos")
# O departamento usa um objeto já existente
dept_ti = Departamento("TI", prof_carlos)
```

Traduzindo Composição (Dependência)

Como o Quarto só existe se a Casa existir, nós o instanciamos ***dentro*** do construtor da classe Casa.

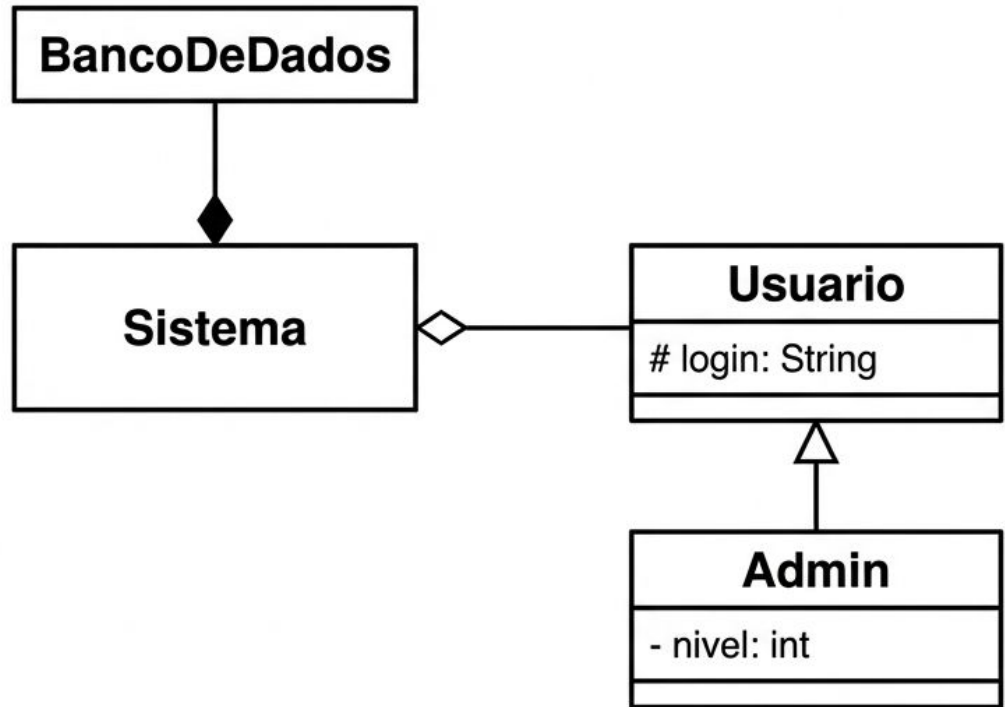


```
class Casa:
    def __init__(self, endereco):
        self.endereco = endereco
        # O quarto nasce e morre junto com a casa
        self.quartos = [Quarto(), Quarto()]

# Se a casa_praia for apagada da memória,
# seus quartos também serão.
casa_praia = Casa("Rua das Conchas")
```

A Pedra de Roseta: Lendo o Sistema Completo

Aqui temos a planta arquitetônica completa. Conseguimos ver a hierarquia de usuários, as dependências estritas do sistema central e os componentes opcionais. No próximo slide, a tradução literal.



O Sistema Construído

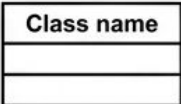
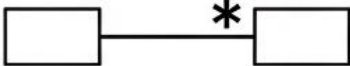
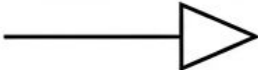
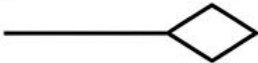
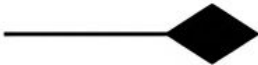
```
class Usuario:
    def __init__(self, login):
        self._login = login # Protegido (#)

class Admin(Usuario): # Herança (Seta vazia)
    def __init__(self, login, nivel):
        super().__init__(login)
        self.__nivel = nivel # Privado (-)

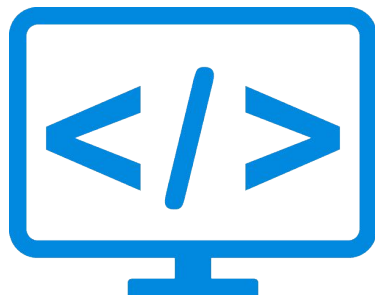
class BancoDeDados:
    pass # Estrutura omitida

class Sistema:
    def __init__(self, usuario_logado):
        # Composição: Sistema cria seu próprio BD
        self.bd = BancoDeDados()
        # Agregação: Recebe um Usuário já existente
        self.sessao_atual = usuario_logado
```

Resumo: O Guia de Bolso do Tradutor

Símbolo UML (Abstracto)	Regra Python (Concreto)
	Gera uma classe, com variáveis no <code>__init__</code> e métodos próprios.
	Ditam os prefixos em Python (nenhum, <code>__</code> , <code>_</code>).
	Gera uma lista (<code>self.lista = []</code>).
	Exige herança (<code>class Filha(Pai):</code>) e <code>super()</code> .
	O objeto é passado por parâmetro (<code>__init__(self, obj)</code>).
	O objeto é instanciado internamente (<code>self.obj = NovaClasse()</code>).

A planta baixa está lida. As ferramentas estão prontas. Agora, é só codificar.



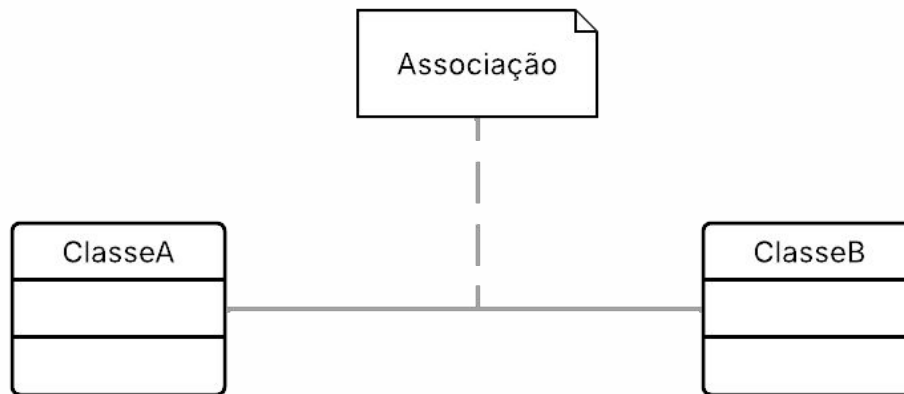
Relacionamentos entre Classes

Introdução

- Conforme Alan Kay, objetos trabalham juntos para alcançar um resultado
- Precisam se relacionar
- Formas de relacionamento:
 - Associação
 - Dependência
 - Herança ou generalização
 - Realização
- Cardinalidade (multiplicidade) e navegabilidade

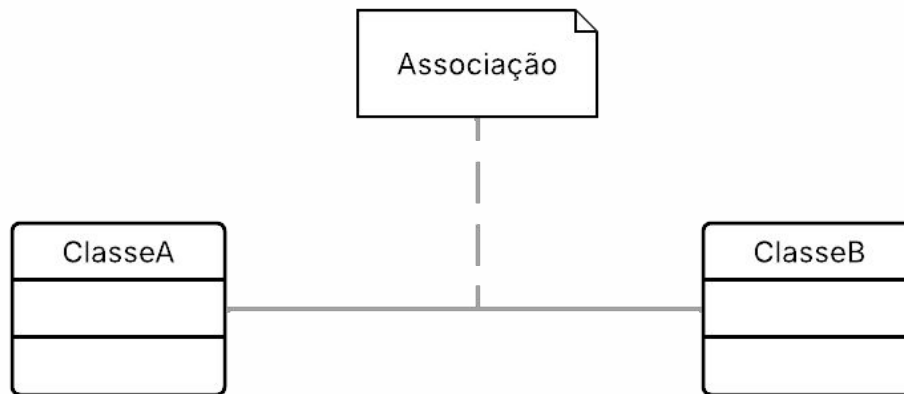
Associação

- Forma mais simples e abstrata de definir que duas classes se relacionam
- Não define exatamente como se relacionam, apenas define que se relacionam de alguma forma



Cardinalidade

- Também conhecido como multiplicidade
- Define as quantidades mínima e máxima de objetos envolvidos na associação



Cardinalidade

Nome	Simbologia na UML
Apenas Um	1..1 (ou 1)
Zero ou Muitos	0..* (ou *)
Um ou Muitos	1..*
Zero ou Um	0..1
Intervalo Específico	$l_i..l_s$

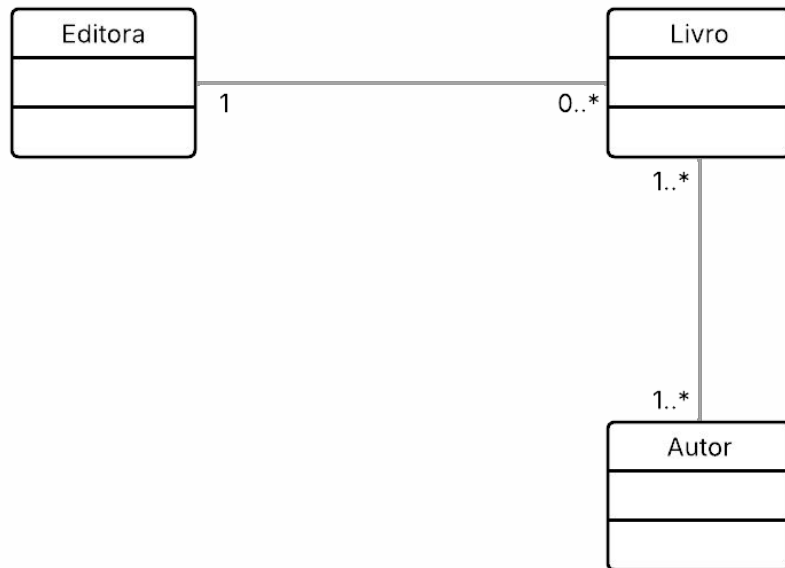


Conectividade

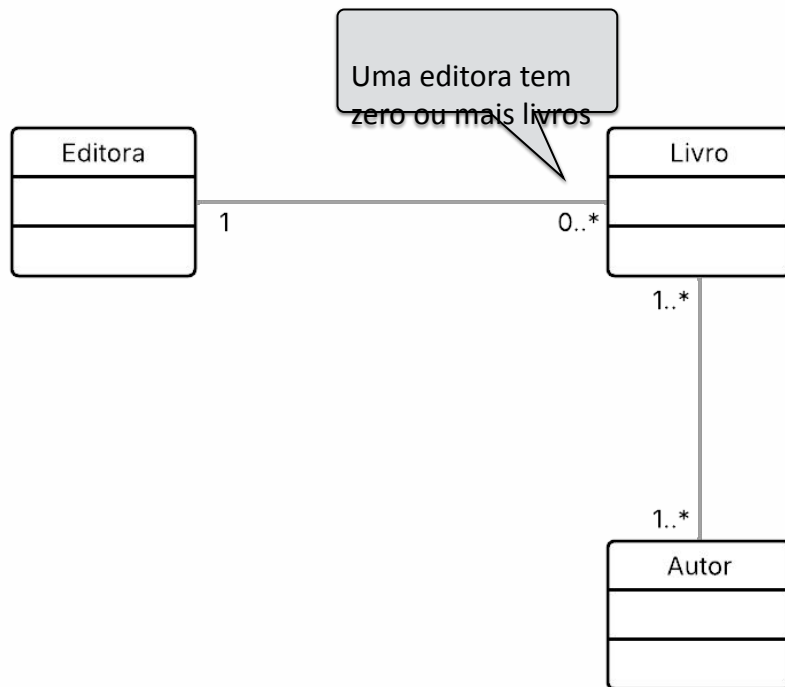
Conectividade	Em um extremo	No outro extremo
Um para um	0..1 1	0..1 1
Um para muitos	0..1 1	* 1..* 0..*
Muitos para muitos	* 1..* 0..*	* 1..* 0..*



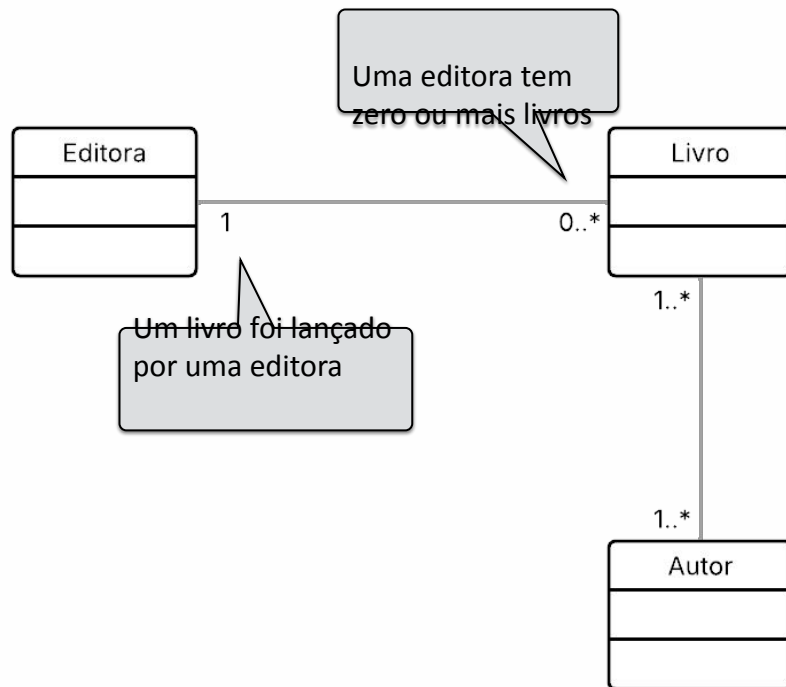
Exemplo



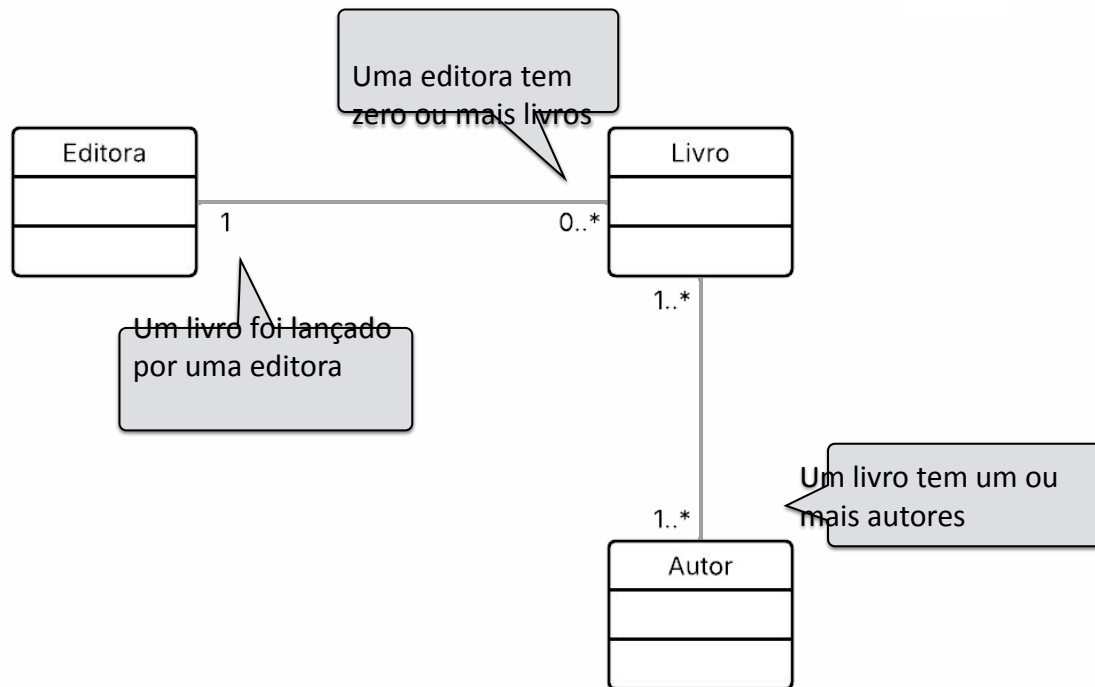
Exemplo



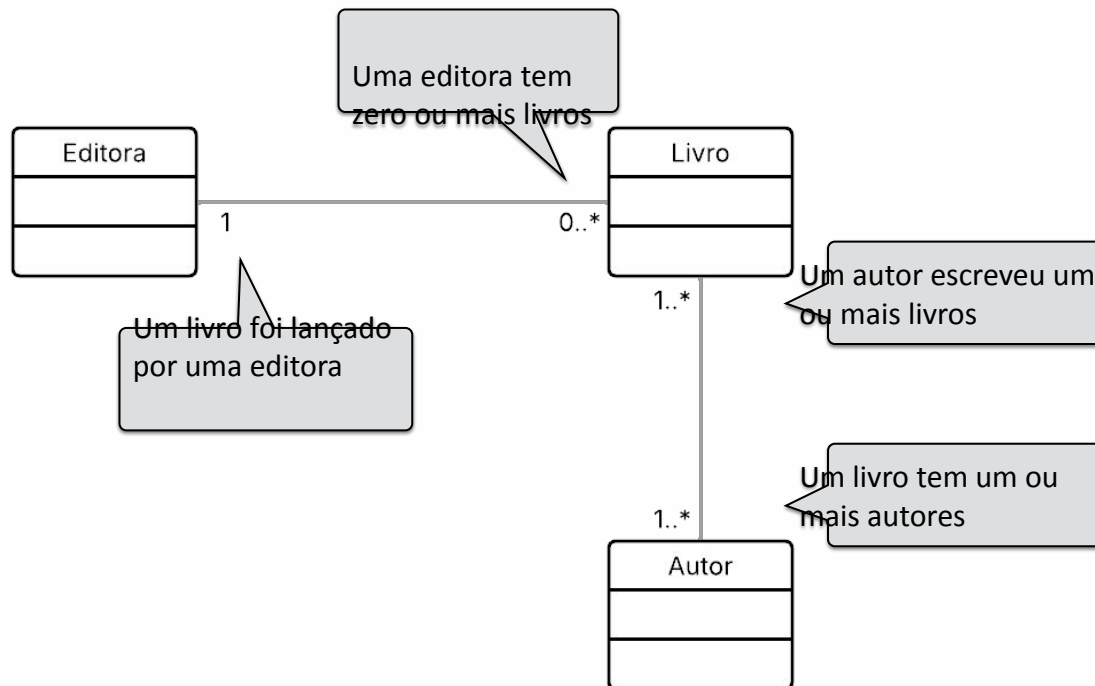
Exemplo



Exemplo

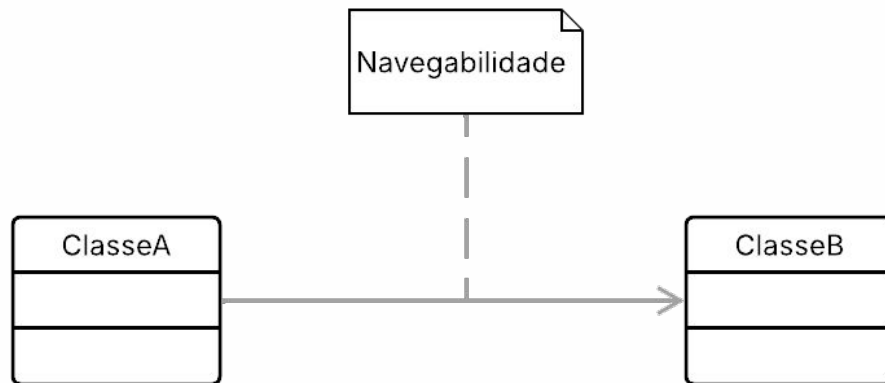


Exemplo

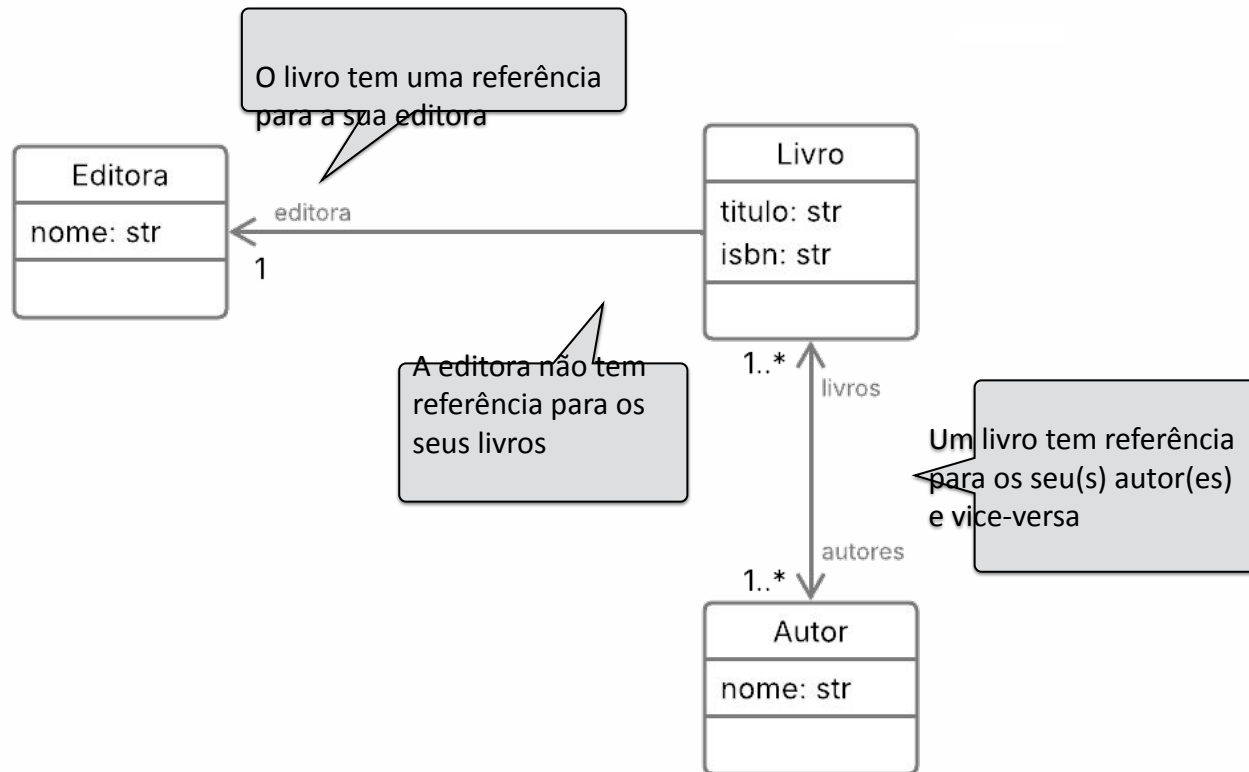


Navegabilidade

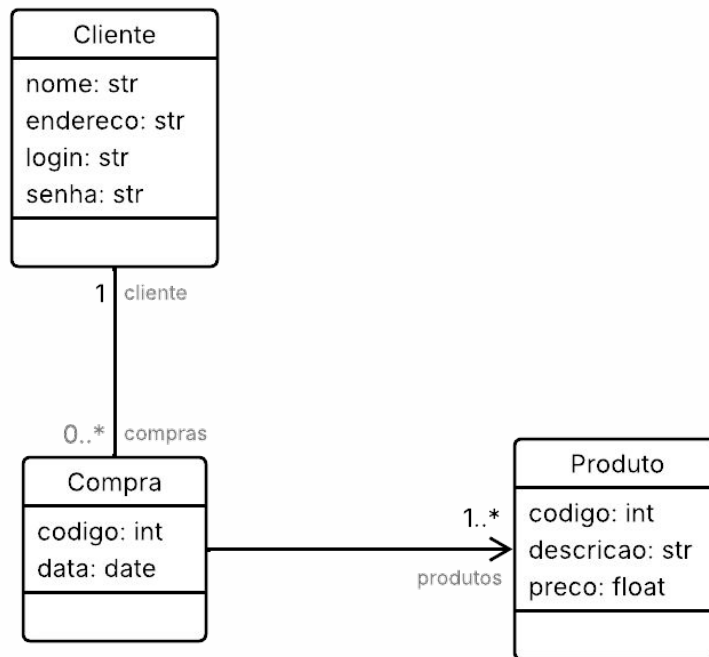
- Indica as referências que uma classe possui para outra(s) classe(s)
- Pode ser unidirecional ou bidirecional



Navegabilidade

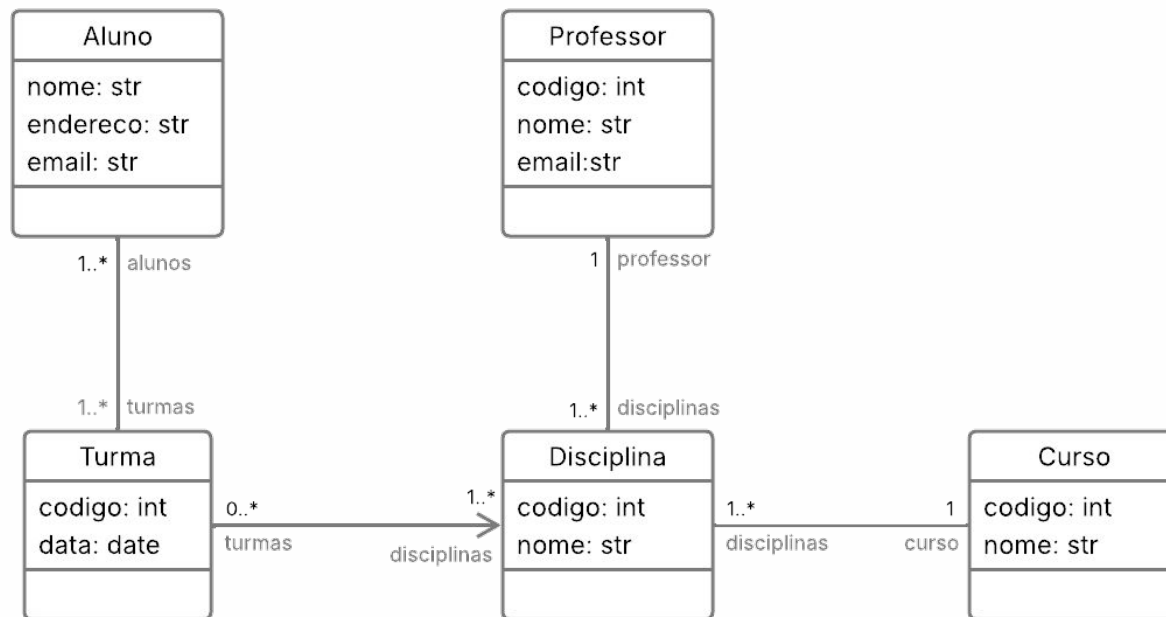


Exemplo



Exercício

- Implemente estas classes em Python. Faça um arquivo de testes.



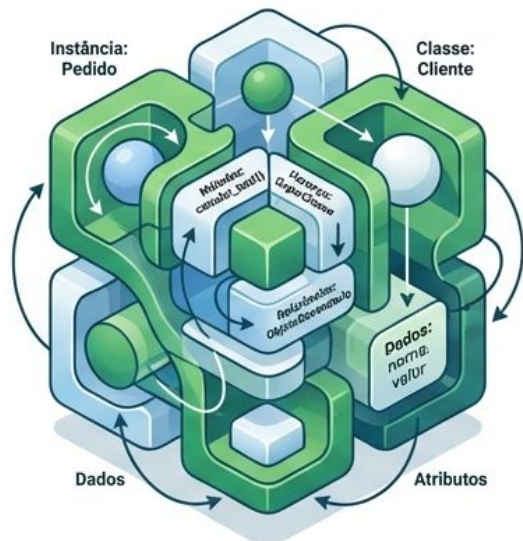
Persistência de Objetos com Peewee e MySQL

Uma abordagem prática para mapeamento objeto-relacional (ORM) em Python.



O CONFLITO DE PARADIGMAS

MEMÓRIA DO PROGRAMA (POO)



Classes em Python possuem métodos, herança e referências aninhadas.

ARMAZENAMENTO RELACIONAL

ID	NOME	VALOR	FK_CLIENTE_ID	DATA_PEDIDO
1	"Produto A"	150.00	101	"2024-10-27"
2	"Serviço B"	200.00	102	"2024-10-27"
3	"Produto C"	200.00	103	"2024-10-27"
4	"Produto D"	300.00	104	"2024-10-27"
5	"Produto E"	200.00	101	"2024-10-27"
6	"Produto F"	300.00	102	"2024-10-27"
7	"Serviço G"	200.00	103	"2024-10-27"
8	"Serviço H"	150.00	102	"2024-10-27"
9	"Serviço U"	200.00	102	"2024-10-27"

Bancos de dados relacionais armazenam apenas valores escalares organizados em tabelas planas.



A Consequência (Impedance Mismatch): Salvar um objeto complexo exige quebrar sua estrutura manualmente em pedaços simples para o banco de dados e reconstruí-lo ao ler.

O Papel do Mapeamento Objeto-Relacional (ORM)



Um ORM converte **automaticamente** dados entre sistemas de **tipos incompatíveis**, mapeando Classes para Tabelas e Instâncias para Linhas.

1

Fluxo Nativo

Permite manipular o banco de dados usando sintaxe orientada a objetos, sem a necessidade de escrever queries SQL extensas.

2

Manutenibilidade

Separa a lógica de negócios da lógica do banco, resultando em um código mais limpo e fácil de manter.

3

Segurança

Abstrai a execução das queries, prevenindo proativamente ataques de injeção de SQL (SQL Injection).

Por que Escolhemos o Peewee?

	Peewee	SQLAlchemy	Tortoise-ORM
Curva de Aprendizado	Baixa (Ideal para Iniciantes)	Alta	Média
Tamanho e Complexidade	Leve e Minimalista	Complexo e Robusto	Focado em Assincronicidade
Melhor Caso de Uso	Projetos pequenos/médios e Ensino	Sistemas Enterprise	Aplicações Async de Alta Concorrência



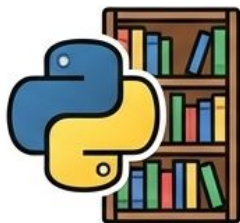
A escolha didática: O Peewee oferece uma abstração simples e intuitiva do banco de dados, permitindo foco no aprendizado de POO sem a sobrecarga de ferramentas voltadas para aplicações Enterprise complexas.

Preparando o Ambiente de Desenvolvimento



1. O Servidor

O Peewee não cria o banco nativamente. É necessário ter o MySQL Server rodando (via instalador standalone, XAMPP ou Docker).



2. A Biblioteca ORM

A ferramenta leve e intuitiva que fará a ponte.

```
pip install peewee
```



3. O Driver de Conexão (DB-API)

O mensageiro de baixo nível. O Peewee precisa deste driver para se comunicar especificamente com bancos MySQL.

```
pip install pymysql
```

Estabelecendo a Ponte com o Servidor

```
database_config.py

from peewee import *

db = MySQLDatabase(
    'nome_do_banco',
    user='root',
    password='sua_senha',
    host='localhost',
    port=3306,
    autoconnect=False
)
```

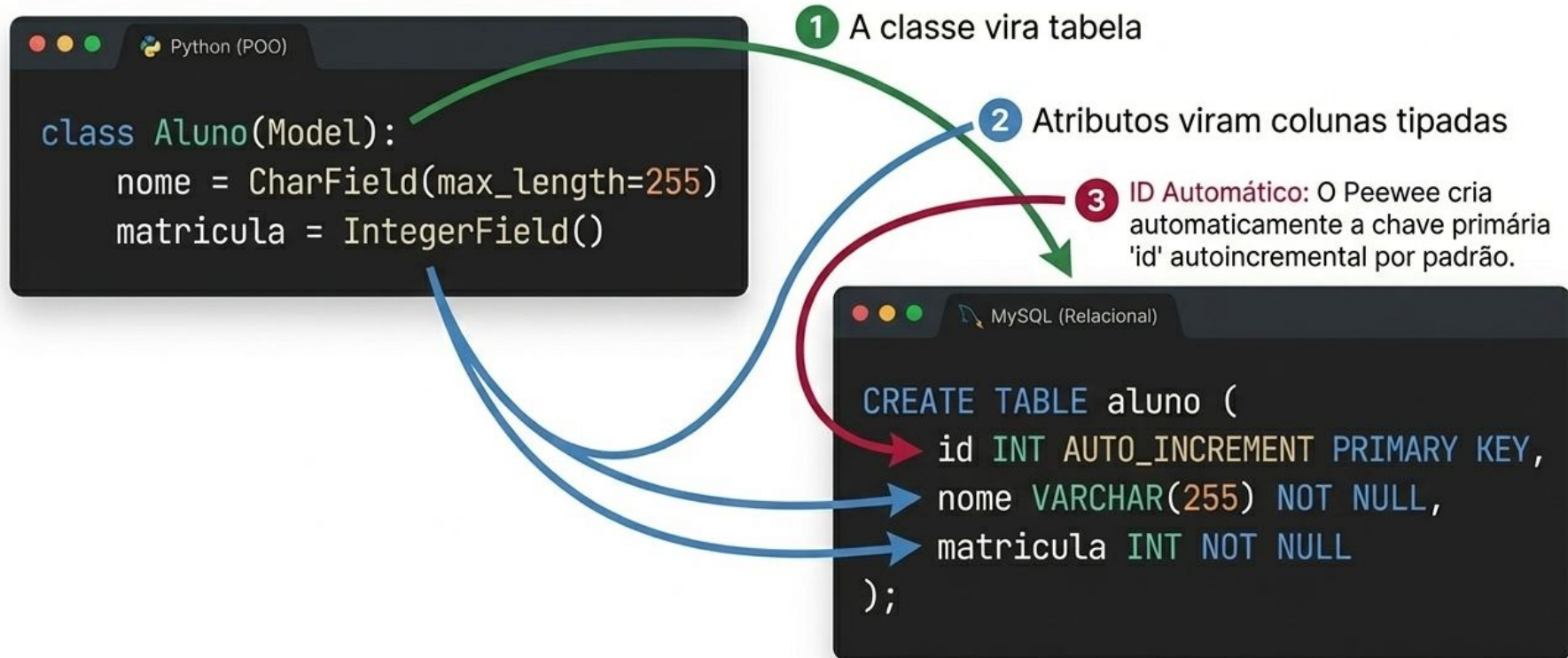
A classe específica do Peewee para o driver MySQL.

Mapeia a porta padrão (3306) e os dados de acesso.



Impede que a conexão seja aberta automaticamente na instância (boa prática para gerenciar o ciclo de vida manualmente).

Mapeando Classes para Tabelas (Modelos)



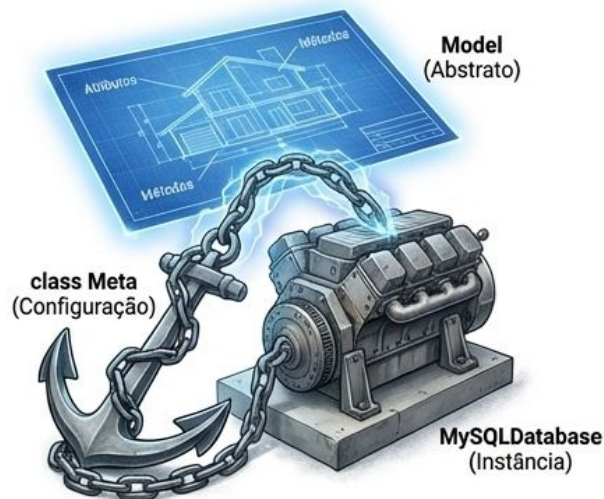
A Classe Interna Meta: O Âncora de Configuração

```
models.py

class BaseModel(Model):
    class Meta:
        database = db

class Aluno(BaseModel):
    nome = CharField()
    # A herança aplica a classe Meta automaticamente
```

- A classe Meta fornece metadados e configurações cruciais para o Modelo.
- database: Define em qual banco de dados específico esta tabela existirá no servidor.
- **Boa Prática:** Criar um **BaseModel** com a classe Meta centraliza a configuração. Todos os outros modelos (como **Aluno**) apenas herdam dele, mantendo o código limpo (DRY).



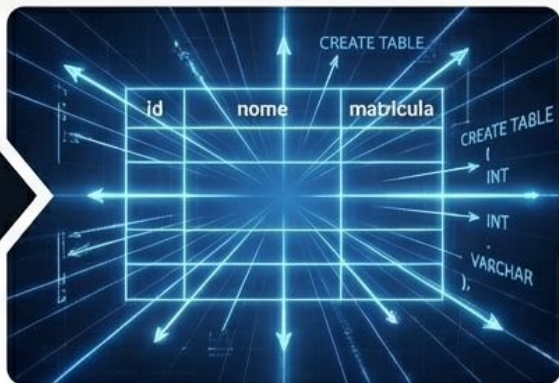
- A classe Meta fornece metadados e configurações cruciais para o Modelo.
- **database:** Define em qual banco de dados específico esta tabela existirá no servidor.
- **Boa Prática:** Criar um **BaseModel** com a classe Meta centraliza a configuração. Todos os outros modelos (como **Aluno**) apenas herdam dele, mantendo o código limpo (DRY).

Materializando o Esquema no Banco de Dados

```
db.connect()  
db.create_tables([Aluno, Curso, Professor])
```



Antes da Execução



Tradução de Classes para DDL

A blue-bordered box containing a table with three columns: 'id', 'nome', and 'matricula'. The table has five rows, with the first row highlighted in a darker blue. This represents the state of the database after the tables have been materialized.

id	nome	matricula

Tabela Materializada



Operação Segura: Por padrão, o Peewee adiciona internamente a cláusula IF NOT EXISTS. O código pode rodar múltiplas vezes sem apagar ou corromper dados já existentes.

Operações CRUD: Inserindo Dados (Create)

Abordagem 1: Inserção Direta

```
1 # Instancia e salva no banco em uma única operação
2 Aluno.create(
3     nome='Maria Joaquina', \
4     matricula=2023101
5 )
```

Abordagem 2: Instanciação e Salvamento

```
1 # Cria o objeto apenas na memória RAM
2 novo_aluno = Aluno(nome='João Pedro', matricula=2023102)
3
4 # ... Lógica adicional (ex: validações) ...
5
6 # Efetiva a gravação no banco de dados
7 novo_aluno.save()
```



	id	nome	matricula
▶	3	Maria Joaquina	2023101

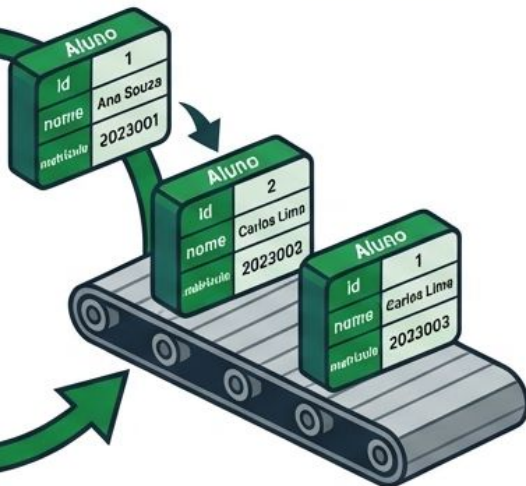
Ambos os métodos confirmam (commit) as alterações automaticamente na configuração padrão.

Operações CRUD: Consultando Dados (Select)

```
1 # SELECT * FROM aluno WHERE matricula > 2023000
2 alunos_recentes = Aluno.select().where(Aluno.matricula > 2023000)
3
4 for aluno in alunos_recentes:
5     print(f'Nome: {aluno.nome} | Matrícula: {aluno.matricula}')
```

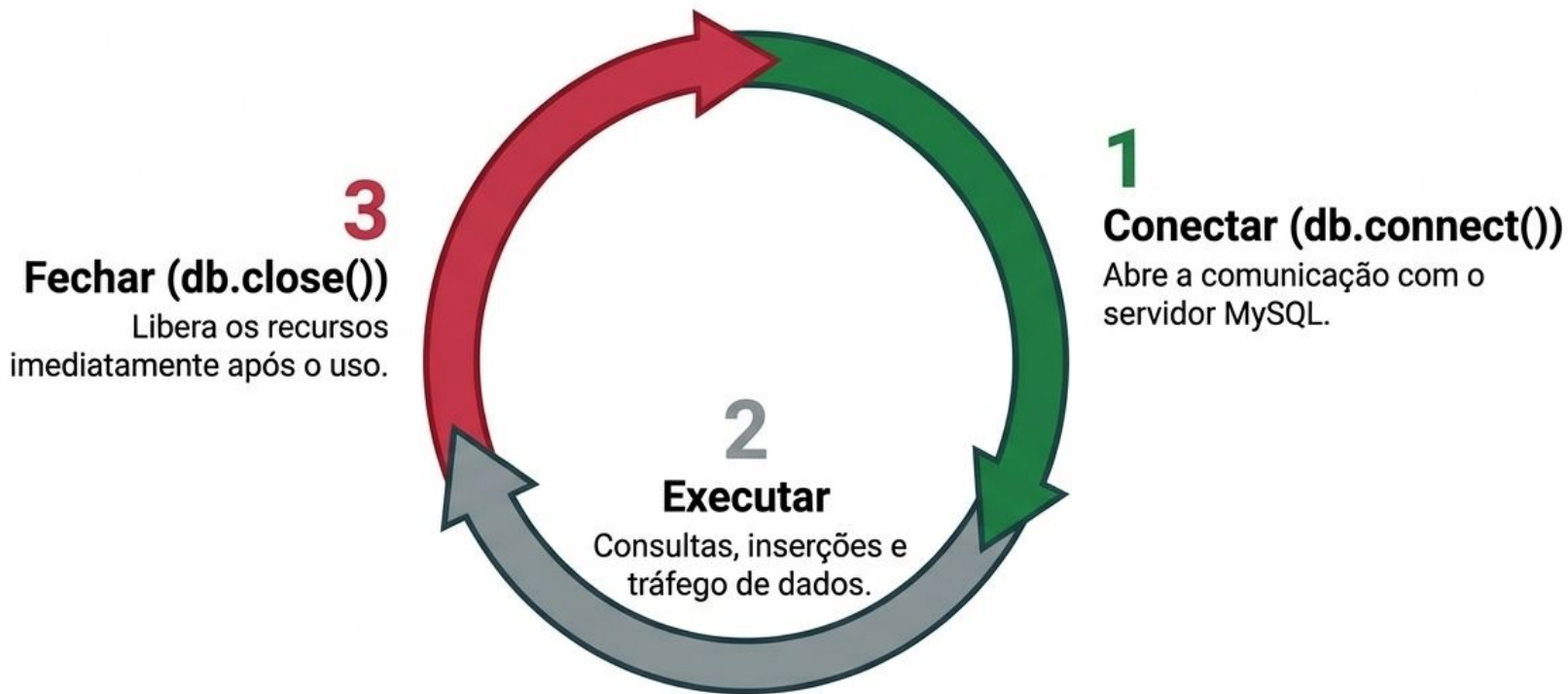
aluno

id	nome	matricula
1	Ana Souza	2023001
2	Carlos Lima	2023002
3	Carlos Linno	2023003
...	...	...



- A query **não retorna linhas de texto brutas**, mas sim uma **coleção iterável de instâncias reais** da classe **Aluno**.
- **Acesso Elegante**: Você acessa as colunas da tabela utilizando a **notação de ponto nativa do Python** (`aluno.nome`), mantendo o **encapsulamento** e o **design da POO** intactos.

Boas Práticas: O Ciclo de Vida da Conexão



Aviso Crítico: Nunca deixe conexões penduradas! O MySQL fecha silenciosamente conexões inativas por tempo limite. Tentar reutilizar uma conexão abandonada horas depois resultará no erro fatal: "MySQL Server going away".

O "Jeito Pythônico": Gerenciamento Avançado

```
# O Gerenciador de Contexto ('with') garante segurança absoluta
with db:
    # A conexão é ABERTA automaticamente ao entrar no bloco
    alunos = Aluno.select()
    for aluno in alunos:
        print(aluno.nome)

# A conexão é FECHADA automaticamente ao sair do bloco,
# mesmo se ocorrer um erro ou exceção na execução!
```



O uso do Context Manager (with db:) é a forma mais segura, robusta e recomendada. Ele elimina a necessidade de chamar manualmente os métodos de abrir e fechar, prevenindo vazamentos de recursos em caso de falhas catastróficas na aplicação.

Visão Geral da Arquitetura de Persistência

1. Lógica da Aplicação



Objetos na Memória
RAM (Python)

2. Modelos Peewee



Camada de Tradução
ORM e Classe Meta

3. Driver DB-API



Mensageiro Binário
(pymysql)

4. Servidor MySQL



Armazenamento
Físico em Disco

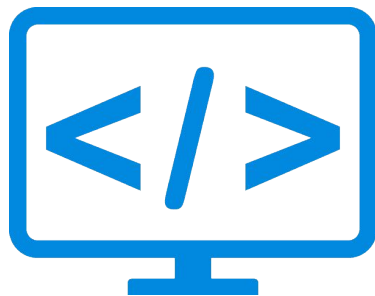
O Grande Truque: Toda a complexidade estrutural entre as etapas 2, 3 e 4 é completamente **invisível** para a Lógica da Aplicação (Etapa 1), garantindo um código limpo e fortemente desacoplado.

Conclusão: Objetos Persistidos com Sucesso

- ✓ Compreendemos o Paradigma ORM e sua capacidade de ponte.
- ✓ Configuramos uma conexão segura via MySQLDatabase.
- ✓ Mapeamos o esquema relacional usando Classes e Herança.
- ✓ Realizamos rotinas CRUD preservando objetos nativos do Python.
- ✓ Implementamos rotinas seguras para o ciclo de vida das conexões.

A verdadeira fluência em Programação Orientada a Objetos ocorre quando dominamos não apenas como os objetos vivem na memória, mas como eles sobrevivem ao desligamento da máquina.

Mãos ao código.



Introdução ao Tkinter

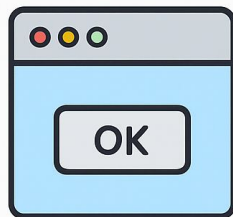
O que é uma Interface Gráfica (GUI)?

- GUI (Graphical User Interface) é uma interface visual que permite ao usuário interagir com um programa por meio de elementos gráficos, como janelas, botões, caixas de texto, menus, etc.
- Características:
 - Uso de ícones, imagens, botões e menus
 - Não exige digitação de comandos manuais
 - Mais intuitiva e amigável, principalmente para iniciantes
- Exemplos:
 - Navegadores de internet (Chrome, Firefox)
 - Programas de edição de texto (Word, Notepad)
 - Aplicativos de celular

GUI x CLI

GUI vs. CLI

GUI



- Visual e intuitiva
- Usa botões e ícones
- Mais fácil para iniciantes

CLI



- Baseada em texto
- Usa comandos digitados
- Exige mais conhecimento

GUI x CLI

Facilidade de uso	Alta – visual e intuitiva	Baixa – requer memorização
Velocidade	Menor para tarefas repetitivas	Maior para usuários experientes
Recursos necessários	Mais recursos gráficos e memória	Leve e rápido
Aprendizado	Mais acessível	Curva de aprendizado maior

O que é o Tkinter?

- .Tkinter é o módulo padrão de Python para criação de interfaces gráficas (GUI)
- .Fornece uma camada de acesso à biblioteca gráfica Tcl/Tk, que é multiplataforma e de código aberto
- .Seu nome vem de “Tk Interface” → Interface do Python com a biblioteca gráfica Tk



O que é o Tkinter?

- A biblioteca Tk foi desenvolvida em 1991 por John Ousterhout para a linguagem Tcl
- O Tkinter surgiu como interface oficial entre Python e Tk, e foi **incorporado à linguagem** desde as versões iniciais do Python (1.5)
- Hoje é mantido pela própria comunidade do Python e vem instalado por padrão com o Python
- Características:
- Multiplataforma: funciona no Windows, macOS e Linux
- Simples e leve, ideal para pequenos sistemas e protótipos
- Integra-se bem com outros recursos da linguagem Python

O que é o Tkinter?

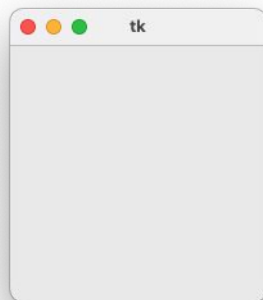
.Limitações

- .Interface mais simples se comparada a frameworks modernos
- .Pouco personalizável em estilos e temas
- .Não indicado para grandes sistemas gráficos ou aplicações com visual profissional avançado
- .Suporte limitado a responsividade/adaptação a diferentes tamanhos de tela

Tkinter é uma excelente opção para aprender, praticar e desenvolver aplicações leves e funcionais com interface gráfica.

Importando Tkinter

```
import tkinter as tk  
janela = tk.Tk()  
janela.mainloop()
```



← Renomeia a biblioteca para
“tk” no contexto do
arquivo

← Cria a janela
principal

← Inicia o **loop de
eventos**.

Sem ele, a janela fecha
imediatamente.

Primeira Janela

```
import tkinter as tk
# Criação da janela principal
janela = tk.Tk()
# Título da janela
janela.title("Minha Primeira Janela")
# Dimensões da janela (largura x altura)
janela.geometry("400x300")
# Início do loop de eventos
janela.mainloop()
```



Restringindo redimensionamento

.É possível restringir o redimensionamento da janela:

. `janela.resizable(largura, altura)`

.largura e altura são valores booleanos (True ou False):

.True permite redimensionar

.False bloqueia o redimensionamento

.Use `.resizable(False, False)` quando quiser manter o tamanho fixo, ideal para interfaces simples como calculadoras ou alertas

Centralizando janela

- .Centralizar a janela oferece uma experiência visual mais profissional e agradável
- .Tkinter não centraliza automaticamente a janela criada
- .Código para centralizar janela manualmente (próximo slide)

Centralizando janela

```
import tkinter as tk
janela = tk.Tk()
largura = 400
altura = 300
# Obtendo as dimensões da tela do computador
largura_tela = janela.winfo_screenwidth()
altura_tela = janela.winfo_screenheight()
# Calculando coordenadas para centralizar a janela
x = (largura_tela // 2) - (largura // 2)
y = (altura_tela // 2) - (altura // 2)
# Definindo o tamanho e a posição
janela.geometry(f"{largura}x{altura}+{x}+{y}")
janela.mainloop()
```

Centralizando janela

```
import tkinter as tk
janela = tk.Tk()
largura = 400
altura = 300
# Obtendo as dimensões da tela do computador
largura_tela = janela.winfo_screenwidth()
altura_tela = janela.winfo_screenheight()
# Calculando coordenadas para centralizar a janela
x = (largura_tela // 2) - (largura // 2)
y = (altura_tela // 2) - (altura // 2)
# Definindo o tamanho e a posição
janela.geometry(f"{largura}x{altura}+{x}+{y}")
janela.mainloop()
```

Obtém a largura total da tela

Obtém a altura total da tela

O geometry recebe dimensões + posição no formato: "LARGURAxALTURA+POS_X+POS_Y"

Adicionando ícone

- O método **.iconbitmap()** define o ícone da janela principal da aplicação
- O arquivo deve estar no formato **.ico**
- O caminho pode ser:
- Relativo: "meu_icone.ico"
- Absoluto: "C:/imagens/icone.ico"
- Se o caminho estiver errado ou o arquivo for inválido, nenhuma alteração é aplicada (ou pode gerar erro em alguns sistemas)
- Pode gerar incompatibilidade em Linux e MacOS
- Nesses casos, usar bibliotecas adicionais como o Pillow.

Adicionando ícone

```
import tkinter as tk
janela = tk.Tk()
janela.title("Janela com Ícone")
janela.iconbitmap("icone.ico") # Substitua pelo
nome do seu arquivo
janela.geometry("300x200")
janela.mainloop()
```

Comportamento ao fechar janela

- .Quando o usuário clica no botão de fechar [X](normalmente no canto superior da janela), o Tkinter:
- .Encerra imediatamente o loop principal (mainloop())
- .Fecha a janela e termina o programa
- .Não há mensagem de confirmação
- .Recursos usados (arquivos, conexões, etc.) são encerrados apenas se o programador tratá-los

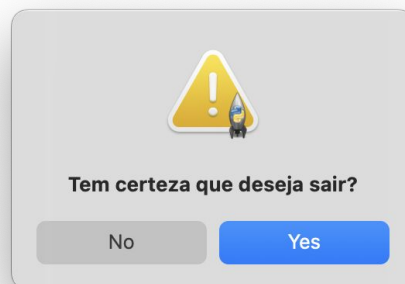
Comportamento ao fechar janela

.Interceptando o fechamento da janela

```
from tkinter import messagebox
def ao_fechar():
    resposta = messagebox.askyesno("Sair", "Tem
    certeza que deseja sair?", icon=messagebox.WARNING)
    if resposta:
        janela.destroy()
janela.protocol("WM_DELETE_WINDOW", ao_fechar)
```

Comportamento ao fechar janela

.Interceptando o fechamento da janela



Comportamento ao fechar janela

- "WM_DELETE_WINDOW" representa o evento de fechar a janela
- O método associado (ao_fechar) será chamado antes da janela fechar
- Pode ser usado para confirmar saída, salvar dados ou evitar perdas
- Útil em aplicações com formulários, editores de texto ou quando há risco de perder informações não salvas

Erros comuns

.Ícone não aparece ou dá erro

.Causa: caminho ou nome do arquivo .ico incorreto.

.Verifique:

.O arquivo existe?

.Está na mesma pasta do script?

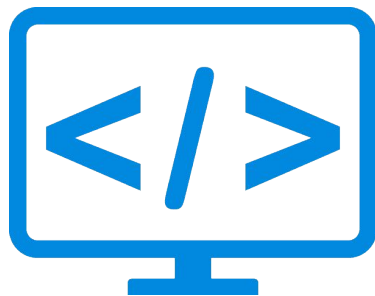
.Está com extensão .ico (e não .png, .jpg, etc.)?

.Janela fecha imediatamente

.Causa: ausência do `mainloop()`

Erros comuns

- .Texto ou widgets cortados
- .Causa: janela pequena demais ou não redimensionável
- .Solução:
 - .Ajuste `geometry("largura x altura")`
 - .Ex: `janela.geometry("500x400")`



Tkinter - Widgets Básicos

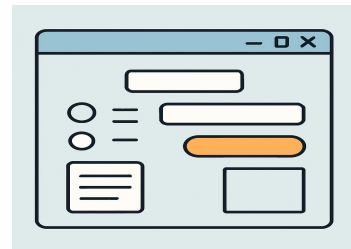
O que são widgets no Tkinter?

- .Widgets são elementos visuais da interface gráfica
- .Cada componente interativo de uma janela é um widget:
 - .Botões, rótulos, campos de texto, menus, etc
- .Exemplos:
 - .Label: para exibir textos
 - .Entry: para entrada de dados (como campos de formulário)
 - .Button: para ações e interatividade
 - .Frame, Checkbutton, Radiobutton, Listbox, entre outros

O que são widgets no Tkinter?

.Importância:

- .Permitem a interação do usuário com o programa
- .São a base da construção de interfaces gráficas com o Tkinter
- .Organizados e combinados, eles formam janelas funcionais e completas
- .Os widgets são como peças de montagem de uma interface, em que podemos posicionar, configurar e conectar esses widgets com funções para criar uma experiência visual



Label

- .Label é um widget do Tkinter usado para exibir textos ou imagens na interface gráfica
- .Ele serve como um rótulo estático, ou seja, não é interativo
- .Usos comuns:
 - .Mostrar títulos, instruções, mensagens ou resultados
 - .Exibir textos fixos ou dinâmicos com valores vindos de outras partes do programa
 - .Trabalhar junto com widgets como Entry e Button para guiar o usuário

Label

.Características:

- .Não recebe clique nem entrada do usuário
- .Pode ser personalizado com cor, fonte, alinhamento, bordas, etc
- .É simples de usar e muito presente em quase todas as interfaces Tkinter

Label

```
import tkinter as tk
janela = tk.Tk()
janela.title("Janela")
janela.geometry("500x400")
rotulo = tk.Label(janela, text="Bem-vindo!")
rotulo.pack()
janela.mainloop()
```

- O label precisa ser empacotado com `pack()` para ser exibido
- Sem o `pack()`, o label é criado mas não aparece na interface

Label

.O widget Label permite alterar aparência do texto e do fundo usando parâmetros adicionais

```
import tkinter as tk
janela = tk.Tk()
rotulo = tk.Label(
    janela,
    text="Mensagem personalizada",
    font=("Helvetica", 16, "italic"),
    fg="white",
    bg="darkblue"
)
rotulo.pack()
janela.mainloop()
```

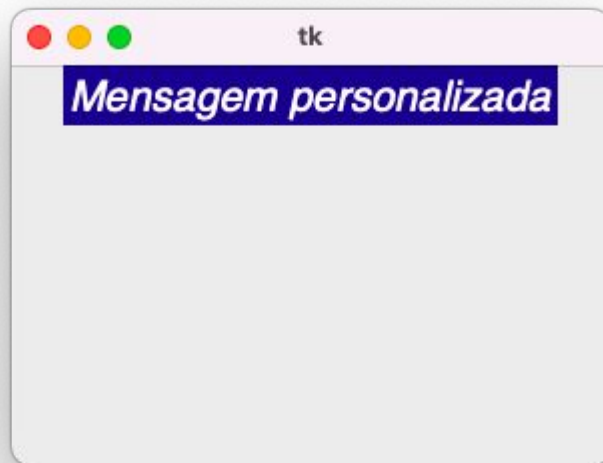


font: define o estilo da fonte

fg: define a cor do texto

bg: define a cor de fundo

Label



Entry

- .O Entry é um widget do Tkinter usado para entrada de dados do tipo texto
- .Ele permite que o usuário digite informações diretamente na interface gráfica
- .Usos comuns:
 - .Formulários (nome, e-mail, idade, etc.)
 - .Campos de login e senha
 - .Inserção de dados para processamento posterior

Entry

.Características

- .Aceita texto de uma única linha (para múltiplas linhas, usa-se Text)
- .É possível recuperar o conteúdo digitado com o método .get()
- .Pode ser combinado com Label e Button para criar interações completas

```
import tkinter as tk
janela = tk.Tk()
janela.title("Exemplo com Entry")
janela.geometry("400x300")
# Criar o campo de entrada
entrada = tk.Entry(janela)
entrada.pack()
janela.mainloop()
```

Entry

- .Um campo de texto aparece na janela
- .O usuário pode digitar livremente nesse campo
- .É possível adicionar argumentos como **width**, **font**, **fg** e **bg** para personalizar o Entry



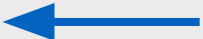
Recuperando dados do Entry

.O método **.get()** é usado para obter o texto digitado pelo usuário em um campo Entry

```
import tkinter as tk
def exibir_texto():
    texto = entrada.get()
    print("Texto digitado:", texto)
janela = tk.Tk()
entrada = tk.Entry(janela)
entrada.pack()
botao = tk.Button(janela, text="Mostrar",
                  command=exibir_texto)
botao.pack()
janela.mainloop()
```

Recuperando dados do Entry

```
import tkinter as tk
def exibir_texto():
    texto = entrada.get()
    print("Texto digitado:", texto)
janela = tk.Tk()
entrada = tk.Entry(janela)
entrada.pack()
botao = tk.Button(janela, text="Mostrar",
                  command=exibir_texto)
botao.pack()
janela.mainloop()
```



Função para exibir o texto



Usuário digita no entry

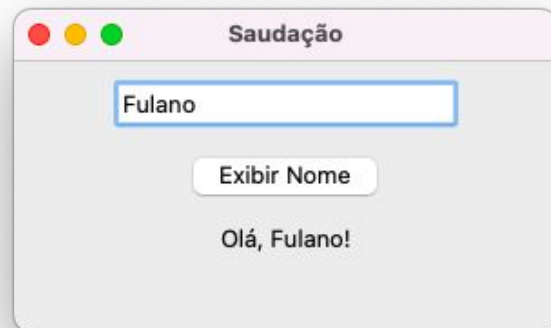


Ao clicar no botão, executa a função exibir_texto

Exemplo Completo

```
import tkinter as tk
def mostrar_nome():
    nome = entrada.get()
    rotulo_resultado.config(text=f"Olá, {nome}!")
janela = tk.Tk()
janela.title("Saudação")
janela.geometry("300x150")
entrada = tk.Entry(janela)
entrada.pack(pady=10)
botao = tk.Button(janela, text="Exibir Nome",
command=mostrar_nome)
botao.pack()
rotulo_resultado = tk.Label(janela, text="")
rotulo_resultado.pack(pady=10)
janela.mainloop()
```

Exemplo Completo



Button

- .O Button é um widget do Tkinter usado para executar ações quando clicado
- .Ele representa um botão interativo, como os vistos em janelas, sites ou aplicativos
- .Usos:
 - .Enviar ou confirmar dados
 - .Acionar funções, como cálculos, exibição de mensagens, navegação entre telas
 - .Encerrar ou reiniciar programas
 - .Atualizar informações na interface

Button

- Características
- Pode ter texto, imagem ou ambos
- Deve estar associado a uma função com o parâmetro **command**
- Pode ser estilizado com bg, fg, font, width, height etc
- **command** é o parâmetro usado para conectar o botão a uma função
- Importante: a função é passada sem parênteses (nome_da_funcao e não nome_da_funcao())
- Sempre que o botão for clicado, a função será executada automaticamente

Button

```
import tkinter as tk
def acao():
    print("O botão foi clicado!")
janela = tk.Tk()
botao = tk.Button(janela, text="Clique aqui",
command=acao)
botao.pack()
janela.mainloop()
```

Estilizando o Button

.O widget Button permite personalização visual

```
botao = tk.Button(  
    janela,  
    text="Enviar",  
    width=15,  
    height=2,  
    fg="white",  
    bg="blue",  
    font=("Helvetica", 12, "bold")  
)
```

```
botao.pack()
```

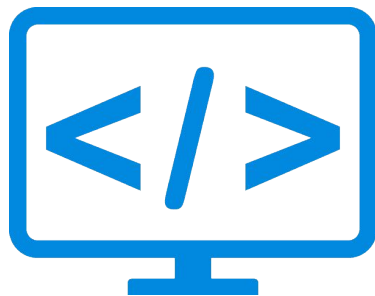
Largura

Altura

Cor da fonte

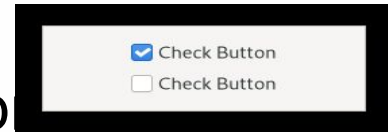
Cor de fundo

Características da fonte



Widgets Básicos - Parte 2

Checkbox



- .O Checkbox é um widget do Tkinter que representa uma caixa de seleção
- .Permite que o usuário ligue ou desligue uma opção (como marcar ou desmarcar)
- .Características:
 - .Permite múltiplas seleções
 - .Pode estar em dois estados: selecionado (1) ou não selecionado (0)
 - .Pode ser vinculado a uma variável de controle como `IntVar()` ou `BooleanVar()`

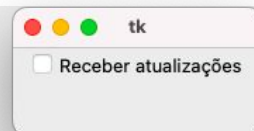
Checkbox

- .Quando o usuário pode escolher várias opções ao mesmo tempo
- .Ex: "Deseja receber notificações?", "Aceita os termos?", "Quais idiomas você fala?"

Checkbox

- .Quando o usuário marca o checkbox, `var.get()` retorna 1. Quando desmarca, retorna 0
- .Com `BooleanVar()`, retorna `True/False`

```
import tkinter as tk
janela = tk.Tk()
var = tk.IntVar()
check = tk.Checkbutton(janela, text="Receber
atualizações", variable=var)
check.pack()
janela.mainloop()
```

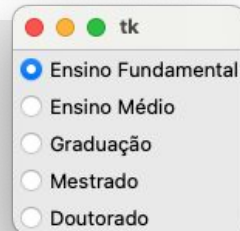


Radiobutton

- O Radiobutton permite criar um grupo de opções onde o usuário escolhe apenas uma
- Todos os Radiobuttons do grupo estão associados à mesma variável de controle (`IntVar()` ou `StringVar()`)

Radiobutton

```
import tkinter as tk
janela = tk.Tk()
opcao = tk.StringVar()
opcao.set("EF")
tk.Radiobutton(janela, text="Ensino Fundamental",
variable=opcao, value="EF").pack(anchor='w')
tk.Radiobutton(janela, text="Ensino Médio",
variable=opcao, value="EM").pack(anchor='w')
tk.Radiobutton(janela, text="Graduação",
variable=opcao, value="G").pack(anchor='w')
tk.Radiobutton(janela, text="Mestrado",
variable=opcao, value="M").pack(anchor='w')
tk.Radiobutton(janela, text="Doutorado",
variable=opcao, value="D").pack(anchor='w')
janela.mainloop()
```



Recuperando valores com get()

.Para recuperar o valor escolhido pelo usuário para um Checkbutton ou Radiobutton, deve-se usar o método **.get()**

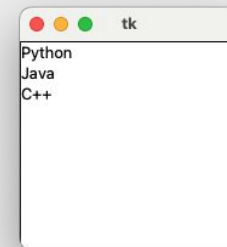
```
import tkinter as tk
janela = tk.Tk()
genero = tk.StringVar()
genero.set("M")
tk.Radiobutton(janela, text="Masculino",
variable=genero, value="M").pack()
tk.Radiobutton(janela, text="Feminino",
variable=genero, value="F").pack()
def mostrar_selecao():
    print("Gênero selecionado:", genero.get())
tk.Button(janela, text="Exibir",
command=mostrar_selecao).pack()
janela.mainloop()
```

Listbox

- A Listbox é um widget do Tkinter usado para exibir uma lista de itens
- Permite que o usuário visualize, role e selecione itens da lista
- Características:
 - Suporta seleção única ou múltipla de itens
 - Pode exibir listas longas com barra de rolagem (scrollbar)
 - Ideal para apresentar listas de opções, dados, registros ou escolhas

Listbox

```
import tkinter as tk
janela = tk.Tk()
lista = tk.Listbox(janela)
lista.insert(1, "Python")
lista.insert(2, "Java")
lista.insert(3, "C++")
lista.pack()
janela.mainloop()
```



Listbox

```
import tkinter as tk
janela = tk.Tk()
janela.title("Lista de Linguagens")
lista_linguagens = ["Python", "Java", "C++", "JavaScript",
"Ruby"]
lista_tela = tk.Listbox(janela)
lista_tela.pack()
for item in lista_linguagens:
    lista_tela.insert(tk.END, item)
janela.mainloop()
```



Listbox

.A Listbox permite definir como o usuário pode selecionar os itens:

.**single**: Permite selecionar apenas um item por vez (padrão)

.**multiple**: Permite selecionar vários itens, clicando em cada um

.**extended**: Permite selecionar vários itens, segurando Shift (intervalo) ou Ctrl (seleção múltipla livre)

.**browse**: Semelhante ao SINGLE, mas permite mover a seleção com o mouse pressionado

Listbox

```
import tkinter as tk
janela = tk.Tk()
janela.title("Seleção de Linguagens")
lista = tk.Listbox(janela, selectmode="extended")
lista.pack()
linguagens = ["Python", "Java", "C++",
"JavaScript", "Ruby"]
for item in linguagens:
    lista.insert(tk.END, item)
janela.mainloop()
```



Listbox

.Para recuperar os elementos selecionados de uma Listbox:

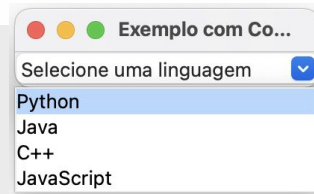
```
selecao = listbox.curselection()  
for i in selecao:  
    print(listbox.get(i))
```


Combobox

- .A Combobox é um widget que exibe uma lista suspensa de opções.
- .Permite que o usuário:
 - .Selecione uma opção da lista, ou
 - .Digite manualmente um valor (opcional).
- .É semelhante ao menu suspenso usado em formulários de sites
- .Ocupa menos espaço que uma Listbox
- .É mais elegante para listas curtas e escolhas únicas
- .Deixa a interface mais limpa e organizada

Combobox

```
import tkinter as tk
from tkinter import ttk
janela = tk.Tk()
janela.title("Exemplo com Combobox")
opcoes = ["Python", "Java", "C++", "JavaScript"]
combobox = ttk.Combobox(janela, values=opcoes)
combobox.pack()
combobox.set("Selecione uma linguagem")
janela.mainloop()
```



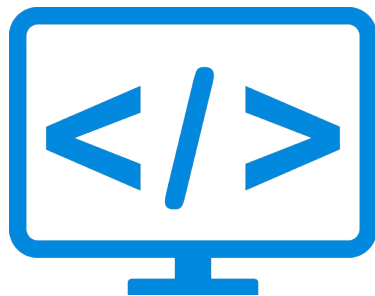
.O método **.set()** define um texto inicial ou uma opção padrão exibida na Combobox antes da seleção

Combobox

- .No Combobox, o método **.get()** permite recuperar a opção selecionada
- .O método **.bind()** permite reagir a mudanças na seleção
- .Exemplo (próximo slide)

Combobox

```
import tkinter as tk
from tkinter import ttk
def mostrar_selecao(event):
    print("Selecionado:", combobox.get())
janela = tk.Tk()
janela.title("Combobox com .get() e .bind()")
opcoes = ["Python", "Java", "C++", "JavaScript"]
combobox = ttk.Combobox(janela, values=opcoes)
combobox.pack()
combobox.bind("<<ComboboxSelected>>", mostrar_selecao)
janela.mainloop()
```



Gerenciamento de Layout

O que é um gerenciador de layout?

- .Um gerenciador de layout é o sistema que o Tkinter usa para organizar e posicionar os widgets (botões, labels, caixas de texto, etc.) dentro de uma janela ou de um Frame
- .Funções do gerenciador de layout:
 - .Controla onde e como cada widget será exibido
 - .Define o alinhamento, tamanho, espaçamento e comportamento da interface
 - .Permite que a janela se adapte ao conteúdo ou mantenha posições fixas

O que é um gerenciador de layout?

- .O Tkinter oferece três gerenciamentos principais
- .**pack()**: posicionamento sequencial (empilhado)
- .**grid()**: organização em linhas e colunas
- .**place()**: posicionamento absoluto (x, y)

Por que organizar os widgets com layout?

- Garantir que os componentes da interface fiquem bem posicionados, alinhados e adaptáveis dentro da janela da aplicação
- Benefícios:
 - Organização visual
 - Manutenção facilitada
 - Evita sobreposição e desalinhamento

pack()

.pack() é um gerenciador de layout que empilha widgets automaticamente dentro de um contêiner, um embaixo (ou ao lado) do outro

.Vantagens:

.Simplicidade: fácil de aprender e usar

.Ideal para interfaces pequenas ou verticais

.Posiciona os widgets de forma rápida sem configurar linhas e colunas

.Menos código para interfaces básicas

pack()

.Limitações:

- .Pouco flexível em layouts complexos
- .Não funciona bem quando se deseja alinhamentos em linha e coluna ao mesmo tempo
- .Difícil controle preciso sobre posição e alinhamento lateral de vários widgets
- .Não pode ser misturado com grid() no mesmo contêiner

.Quando usar:

- .Telas com poucos elementos, organizados em uma única direção (vertical ou horizontal)
- .Janelas de alerta, menus simples ou painéis rápidos

grid()

- grid() é um gerenciador de layout que organiza widgets em **linhas e colunas**, como em uma tabela
- Permite alinhamento preciso de elementos relacionados, como rótulos e campos de entrada
- Vantagens:
 - Ideal para criar formulários organizados (ex: login, cadastro)
 - Permite alinhar vários widgets horizontal e verticalmente
 - Suporta parâmetros como:
 - row, column: define a célula
 - colspan, rowspan: permite mesclar colunas/linhas
 - padx, pady: espaçamento
 - sticky: alinhamento dentro da célula (N, S, E, W)

grid()

.Limitações:

- .Não pode ser usado junto com pack() no mesmo contêiner
- .Em interfaces muito dinâmicas ou com muitos ajustes, pode demandar mais controle de layout

.Quando usar:

- .Sempre que precisar de alinhamento estruturado
- .Ideal para campos de entrada, rótulos e botões alinhados
- .Muito usado em painéis de configuração e formulários

grid()

.Sintaxe básica:

.row=n: de `widget.grid(row=n, column=m)`

.column=m: define a coluna (começa em 0)

grid()

```
import tkinter as tk
janela = tk.Tk()
tk.Label(janela, text="Usuário:").grid(row=0,
column=0)
tk.Entry(janela).grid(row=0, column=1)
tk.Label(janela, text="Senha:").grid(row=1,
column=0)
tk.Entry(janela, show="*").grid(row=1, column=1)
tk.Button(janela, text="Entrar").grid(row=2,
column=0, columnspan=2)
janela.mainloop()
```

O elemento ocupará a primeira linha, primeira coluna



O columnspan faz a mesclagem de colunas. Nesse caso, duas colunas



grid()



grid()

- Além de row e column, o método .grid() possui parâmetros adicionais que permitem um controle mais refinado da posição e aparência dos widgets na grade
- **padx, pady**: adiciona espaçamento externo (padding) horizontal e vertical
- **columnspan**: faz o widget ocupar múltiplas colunas
- **rowspan**: faz o widget ocupar múltiplas linhas
- **sticky**: alinha o widget dentro da célula, usando direções: N, S, E, W

grid()

```
import tkinter as tk
janela = tk.Tk()
janela.geometry("300x200")
janela.title("Tela de Login")
tk.Label(janela, text="Usuário:").grid(row=0,
column=0, padx=10, pady=10, sticky="E")
tk.Entry(janela).grid(row=0, column=1, padx=10,
pady=10, sticky="W")
tk.Label(janela, text="Senha:").grid(row=1,
column=0, padx=10, pady=10, sticky="E")
tk.Entry(janela, show="*").grid(row=1, column=1,
padx=10, pady=10, sticky="W")
tk.Button(janela, text="Entrar").grid(row=2,
column=0, columnspan=2, pady=20)
janela.mainloop()
```

grid()



Tela de Login

Usuário:

Senha:

place()

.place() é um gerenciador de layout que posiciona widgets **exatamente em posições específicas** dentro da janela, usando coordenadas (x, y) ou proporções (relx, rely)

.Vantagens:

- .Permite posicionamento preciso de cada widget na tela
- .Suporta tamanhos fixos com width e height
- .Aceita posicionamento relativo, útil para adaptação em diferentes tamanhos de tela
- .relx=0.5 centraliza horizontalmente
- .rely=1.0 coloca no final vertical da janela

place()

- Limitações:
- Sem ajuste automático: widgets não se adaptam ao conteúdo nem ao redimensionamento da janela
- Exige mais cálculos manuais para distribuir os elementos corretamente
- Mais difícil de manter em interfaces grandes ou com muitos componentes
- Quando usar:
- Em interfaces onde o posicionamento fixo é essencial, como:
- Simulações visuais
- Painéis personalizados
- Interfaces que imitam layout gráfico pronto (mockups)

place()

.Sintaxe básica:

•x: `widget.place(x=a, y=b, width=c, height=d)`

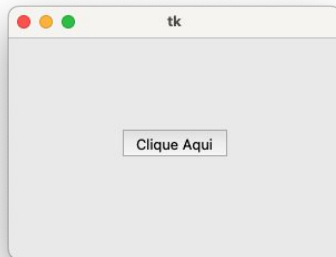
•y: posição vertical

•width: largura em pixels

•height: altura em pixels

place()

```
import tkinter as tk  
janela = tk.Tk()  
janela.geometry("300x200")  
botao = tk.Button(janela, text="Clique Aqui")  
botao.place(x=100, y=80, width=100, height=30)  
janela.mainloop()
```



relx e rely para posicionamento relativo

- **relx** e **rely** são parâmetros do método `.place()` que posicionam widgets de forma proporcional ao tamanho da janela
- Seus valores variam de 0.0 (0%) a 1.0 (100%) do espaço disponível

relx e rely para posicionamento relativo

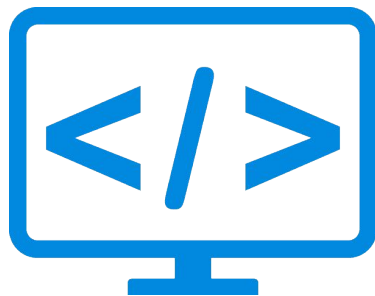
```
import tkinter as tk
janela = tk.Tk()
janela.geometry("300x200")
botao = tk.Button(janela, text="Clique Aqui")
botao.place(relx=0.5, rely=0.5, anchor="center")
janela.mainloop()
```

O elemento será centralizado,
pois está posicionado no meio
(50%)

da largura e altura

O anchor="center" é usado para
centralizar o widget em torno
do
ponto relativo.





Menus e Barra de Ferramentas

Menus e Barra de Ferramentas

.Menus:

.São barras localizadas no topo da janela que agrupam comandos e opções, geralmente organizados em categorias como Arquivo, Editar, Ajuda, etc.

.Toolbars (Barras de Ferramentas):

.São atalhos visuais, compostos por botões (com ou sem ícones), normalmente localizados logo abaixo do menu ou na lateral da janela.



O que são menus e toolbars?

Importância na interface de aplicativos

Menus e Barra de Ferramentas

- .Tornam a interface mais organizada e intuitiva
- .Facilitam o acesso às funções principais do sistema
- .São padrão em aplicações desktop, oferecendo familiaridade ao usuário
- .Melhoram a produtividade, permitindo que tarefas sejam executadas com menos cliques
- .Menus são ideais para ações organizadas e hierárquicas, enquanto a barra de ferramentas oferece acesso rápido às ações mais usadas

Menu

```
import tkinter as tk
janela = tk.Tk()
janela.title("Exemplo de Menu")
janela.geometry("400x300")
menu_bar = tk.Menu(janela)
janela.config(menu=menu_bar)
menu_arquivo = tk.Menu(menu_bar)
menu_bar.add_cascade(label="Arquivo", menu=menu_arquivo)
menu_arquivo.add_command(label="Novo")
menu_arquivo.add_command(label="Abrir")
menu_arquivo.add_separator()
menu_arquivo.add_command(label="Sair", command=janela.quit)
menu ajuda = tk.Menu(menu_bar)
menu_bar.add_cascade(label="Ajuda", menu=menu_ajuda)
menu_ajuda.add_command(label="Sobre")
janela.mainloop()
```

Cria a barra de menus e a adiciona à janela principal

Cria um menu na barra de menus

Adiciona a opção "Arquivo" à barra de menu

Cria itens de menu e um separador

Ligando itens de menu a funções

- .O parâmetro **command** permite associar uma função a um item do menu
- .Sempre que o usuário clicar na opção, a função será executada automaticamente
- .Exemplo (próximo slide)

Ligando itens de menu a funções

```
import tkinter as tk
def sobre():
    tk.messagebox.showinfo("Sobre", "Aplicativo de Exemplo")
def sair():
    janela.quit()
janela = tk.Tk()
janela.title("Menu com Funções")
menu_bar = tk.Menu(janela)
janela.config(menu=menu_bar)
menu_arquivo = tk.Menu(menu_bar, tearoff=0)
menu_bar.add_cascade(label="Arquivo", menu=menu_arquivo)
menu_arquivo.add_command(label="Sair", command=sair)
menu_ajuda = tk.Menu(menu_bar, tearoff=0)
menu_bar.add_cascade(label="Ajuda", menu=menu_ajuda)
menu_ajuda.add_command(label="Sobre", command=sobre)
janela.mainloop()
```

Ao clicar no menu, executa a
função sair()



Ao clicar no menu, executa a
função sobre()



Adicionando Atalhos de Teclado (Accelerators)

- Accelerators são atalhos de teclado exibidos no menu, indicando ao usuário uma maneira rápida de executar aquele comando
- O acelerador é apenas um texto visual, que precisa ser combinado com o método **bind()** para funcionar
- No exemplo abaixo, o `\t` adiciona um espaço alinhado à direita antes do atalho

```
menu.add_command(  
    label="Novo\tCtrl+N",  
    command=funcao_novo  
)
```

Adicionando Atalhos de Teclado (Accelerators)

- .O método **.bind()** permite associar eventos do teclado ou mouse a uma função
- .Com ele, é possível tornar os atalhos de teclado funcionais na interface
- .A função chamada precisa aceitar um parâmetro de evento, mesmo que não use:

```
janela.bind("<Control-n>", funcao)
```

```
def funcao(event):  
    pass
```


Adicionando Atalhos de Teclado (Accelerators)

```
import tkinter as tk
def novo_arquivo(event = None):
    print("Novo arquivo criado!")
janela = tk.Tk()
janela.title("Aceleradores")
menu_bar = tk.Menu(janela)
janela.config(menu=menu_bar)
menu_arquivo = tk.Menu(menu_bar)
menu_bar.add_cascade(label="Arquivo", menu=menu_arquivo)
menu_arquivo.add_command(label="Novo\tCtrl+N",
    command=novo_arquivo)
janela.bind("<Control-n>", novo_arquivo)
janela.mainloop()
```

← Ao usar **event=None** no parâmetro da função permite uso tanto por clique quanto por atalho.

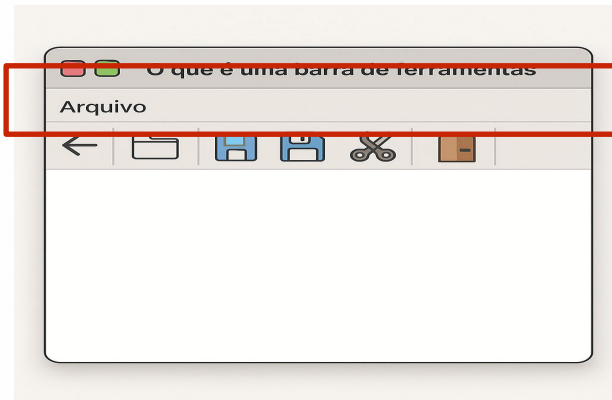
Barra de ferramentas

- .Uma barra de ferramentas (toolbar) é um elemento visual da interface gráfica que contém botões organizados horizontalmente ou verticalmente, geralmente com ícones representando ações rápidas
- .Fica normalmente posicionada logo abaixo da barra de menus
- .Oferecer acesso rápido e direto às funcionalidades mais usadas do aplicativo:
 - .Criar novo
 - .Salvar
 - .Recortar
 - .Fechar

Barra de ferramentas

.Vantagens:

- .Melhora a produtividade do usuário
- .Reduz o número de cliques para tarefas repetitivas
- .Deixa o sistema mais intuitivo e visual



Barra de ferramentas

- .É possível simular uma barra de ferramentas usando um **Frame** fixado no topo da janela
- .Dentro desse Frame, insere-se botões com ou sem ícones, que representam ações rápidas
- .Os arquivos de imagem devem estar no mesmo diretório do script ou deve ser informado o caminho correto
- .Usar preferencialmente o formato de imagem **.png**
- .Recomenda-se usar imagens pequenas (ex: 24x24 px ou 32x32 px) para visualização limpa na barra

Barra de ferramentas

```
import tkinter as tk
from tkinter import PhotoImage
def novo_arquivo():
    print("Novo arquivo criado.")
def salvar_arquivo():
    print("Arquivo salvo.")

janela = tk.Tk()
janela.title("Toolbar com Ícones")
janela.geometry("400x200")
toolbar = tk.Frame(janela, bg="#e0e0e0")
toolbar.pack(side="top", fill="x")
img_novo = PhotoImage(file="exemplo_tkinter_menu/add.png")
img_salvar = PhotoImage(file="exemplo_tkinter_menu/save.png")
img_sair = PhotoImage(file="exemplo_tkinter_menu/exit.png")
btn_novo = tk.Button(toolbar, image=img_novo, command=novo_arquivo)
btn_novo.pack(side="left", padx=5, pady=5)
btn_salvar = tk.Button(toolbar, image=img_salvar, command=salvar_arquivo)
btn_salvar.pack(side="left", padx=5, pady=5)
btn_sair = tk.Button(toolbar, image=img_sair, command=janela.quit)
btn_sair.pack(side="left", padx=5, pady=5)
janela.mainloop()
```

fill="x" faz com que o widget
preencha todo o espaço horizontal

Bibliografia

- MENEZES, Nilo Ney Coutinho. Introdução à programação com Python: algoritmos e lógica de programação
- para iniciantes. 3. ed. rev. ampl. São Paulo: Novatec, 2019. 328 p.
- MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. Algoritmos: lógica para desenvolvimento de
- programação de computadores. 29. ed. São Paulo: Érica, 2019. 368 p.
- PAIVA, Fábio Augusto Procópio de; SOUZA, Givanaldo Rocha de; NASCIMENTO, João Maria Araújo do;
- MARTINS, Rodrigo Siqueira. Introdução a Python com aplicações de sistemas operacionais: projeto. Natal: